CONTENTS

title: “SDS 04a: Sequence Flows (Inbound Channels)” date: “2026-07-03” status: “Draft-for-owner-review” source_of_truth: “All flows traced from actual co

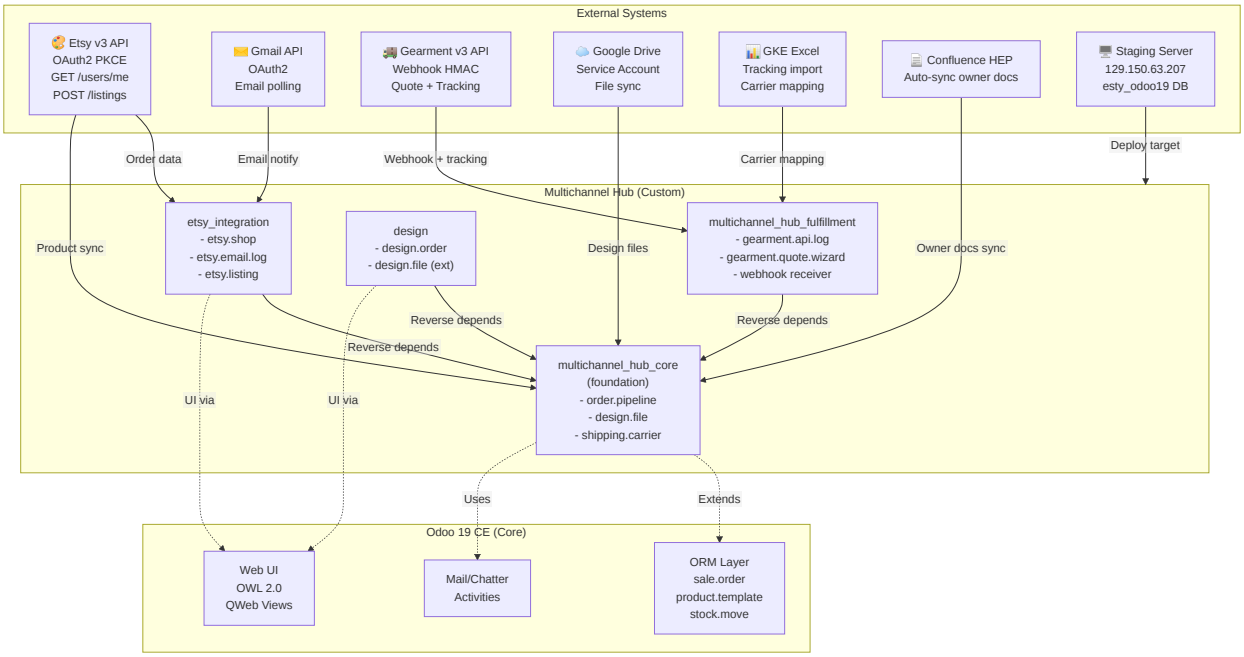
Part 1: System Architecture

Title: odoo19_esty System Context, Module Topology, and Deployment
Date: 2026-07-03
Status: Draft-for-owner-review
Version: 1.0
Source of Truth: Master Plan 006 tracking, .claude/plans/, custom_addons/ manifest files, deployment/ docker-compose

---

(a) System Context Diagram

The system orchestrates order ingest from Etsy, product design workflows, fulfillment coordination across Gearment (POD dropship) and internal VN production, and product catalog management (Phase 3).

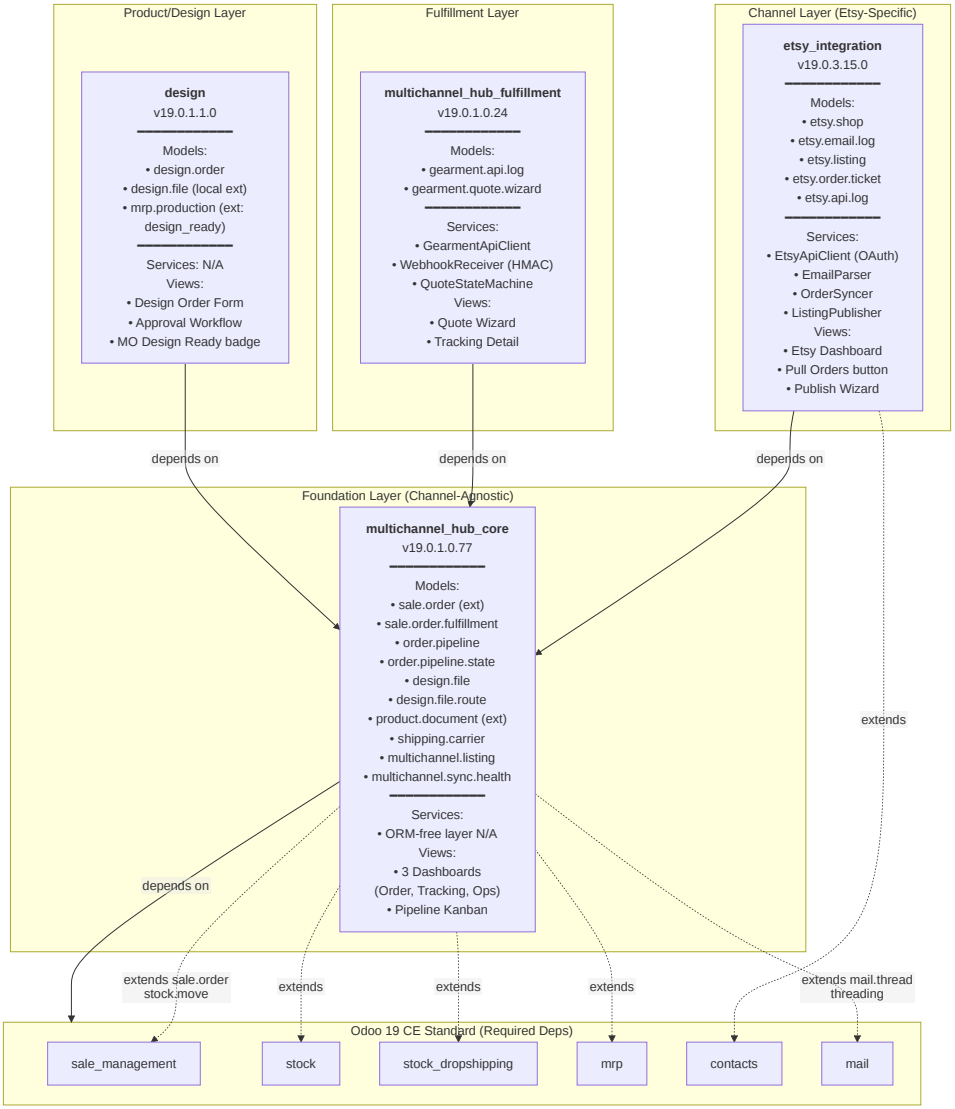


External Dependencies (marked by status):

- **Etsy v3 API** — ✅ Approved OAuth scopes (E1): `transactions_r/w`, `listings_r/w`, `shops_r`, `email_r`
- **Gmail API** — ✅ Legacy email fallback (Phase 2 cutover pending P2-07)
- **Gearment v3 API** — 🟨 Sandbox keys obtained; production credentials pending
- **Google Drive** — ✅ Service account JSON in `secrets/`
- **GKE Excel** — ✅ Nightly import via xlsx wizard (P2-05)
- **Confluence HEP** — ✅ Auto-synced via `.githooks/post-commit`
- **Staging Server** — ✅ Live (129.150.63.207, nightly DB restore)

(b) Module Topology Diagram

Four custom modules arranged in layers: foundation (multichannel_hub_core) → channel-specific (etsy_integration, design) + fulfillment (multichannel_hub_fulfillment) → consumers.



Dependency Flow (one-directional, acyclic):

- multichannel_hub_core → Standard Odoo modules (foundation depends on stdlib)
- etsy_integration → multichannel_hub_core (channel-specific depends on foundation)
- multichannel_hub_fulfillment → multichannel_hub_core (fulfillment depends on foundation)
- design → multichannel_hub_core (product/order design depends on foundation)

Why this structure? (ADR-003)

- Isolate channel-agnostic models (pipeline, fulfillment, carrier) in _core
- Keep Etsy-specific logic (API, email parsing, OAuth) in etsy_integration
- Reserve future channels (amazon_integration, website_channel) to inherit from _core + MHF
- Design module optional; can be disabled if order design workflows not needed

---

(c) Per-Module Responsibility Matrix

Module	Ownership	Key Responsibility	Critical Models	Test Coverage	Status
multichannel_hub_core	Architect + BA	Order pipeline routing, design file management, product design documents, unified dashboards, product catalog hub (Phase 3)	`sale.order.fulfillment`, `order.pipeline*`, `design.file`, `product.document`, `multichannel.listing`, `multichannel.sync.health`	80%+ (Phase 1 complete, ESTY-250)	✅ Phase 1 live + ESTY-250
etsy_integration	Etsy specialist + integrator	Etsy shop OAuth, email ingest (legacy), listing fetch, order creation, publish flow (Phase 3)	`etsy.shop`, `etsy.email.log`, `etsy.listing`, `etsy.api.log`	80%+ (Phase 0/1 regression green)	✅ P1 cutover ready
multichannel_hub_fulfillment	Fulfillment + Gearment specialist	Gearment API wrapper, webhook receiver, quote state machine, tracking sync	`gearment.api.log`, `gearment.quote.wizard`, webhook models (transient)	80%+ (P4-01 complete)	✅ P4-01 E2E pass
design	Product/production team	Design order document, approval workflow, file attachment, production routing, MO Design-Ready badge	`design.order`, `design.file` (extends _core), `mrp.production` (ext)	80%+ (P0 spec, ESTY-244 + ESTY-249)	✅ ESTY-244 + ESTY-249 live

---

(d) Layering Conventions: ORM-Free Service Layer

Pattern: Business logic lives in _core as ORM-free service classes; models in multichannel_hub_core provide ORM glue.

SERVICE LAYER STRUCTURE

```
custom_addons/
├── multichannel_hub_core/
│   ├── models/              # ORM models (sale_order.py, design_file.py, order_pipeline.py)
│   └── services/            # ORM-free business logic (order_pipeline_service.py, design_file_service.py)
├── etsy_integration/
│   ├── models/              # etsy.shop, etsy.email.log, etsy.listing
│   └── services/            # EtsyApiClient, EmailParser, OrderSyncer (ORM-free)
└── multichannel_hub_fulfillment/
    ├── models/              # gearment.api.log, gearment.quote.wizard
    └── services/            # GearmentApiClient, WebhookReceiver, QuoteStateMachine (ORM-free)
```

RATIONALE

1. **Testability**: Service methods can be unit-tested without Odoo ORM; no TransactionCase needed for parsing/crypto/API logic
2. **Reusability**: Services callable from wizards, crons, webhooks, browser-side JS (if APIs exposed)
3. **Encapsulation**: Models call services; services never call back to models (except constructor injection)
4. **Clarity**: ORM side effects (create, write, unlink) happen in model methods; logic-only methods in services

EXAMPLE: ORDERSYNCER

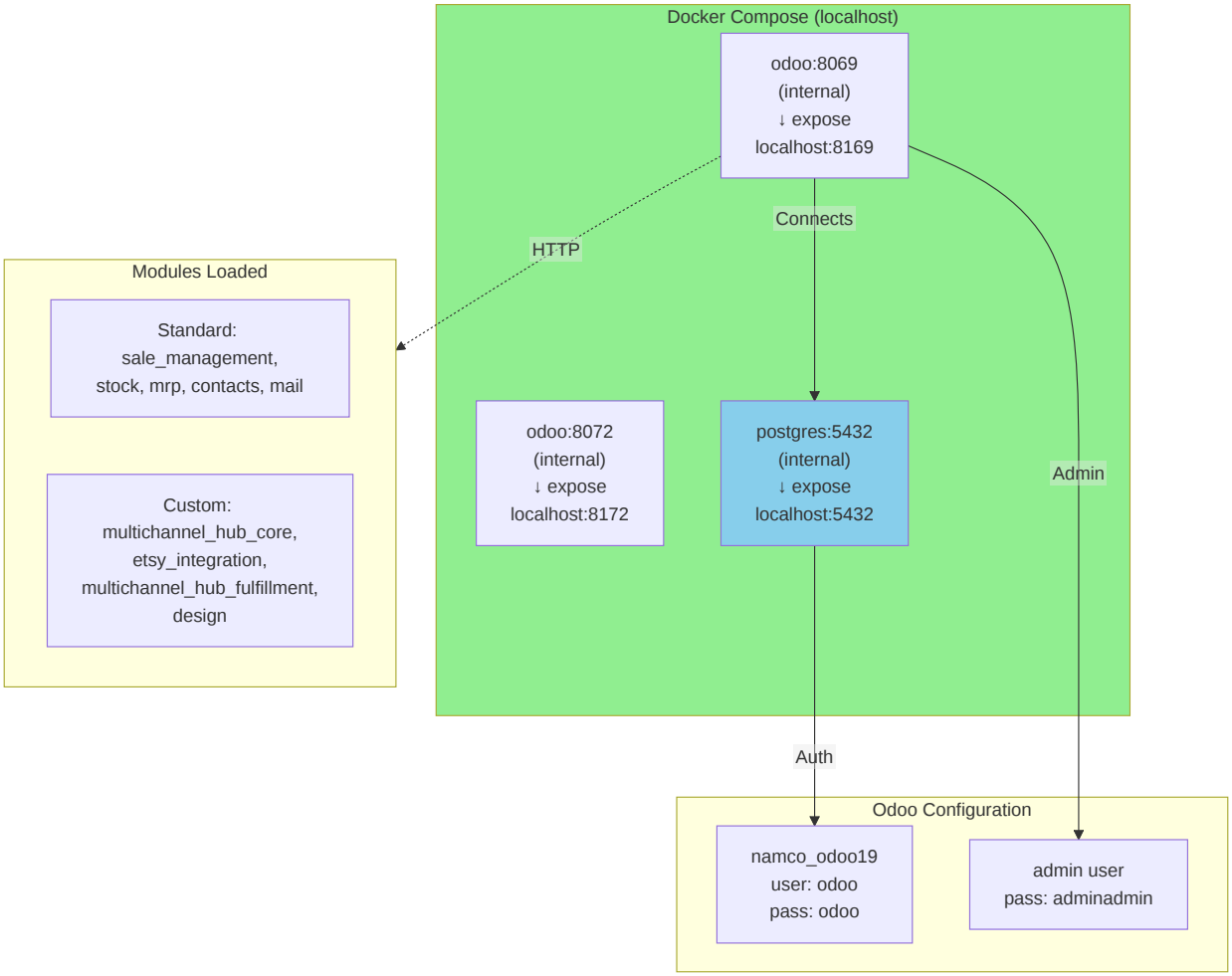
```
# services/order_syncer.py (ORM-free)
class OrderSyncer:
    def __init__(self, partner_repo, order_repo, dedup_service):
        self.partner_repo = partner_repo
        self.order_repo = order_repo
        self.dedup_service = dedup_service

    def sync_order(self, etsy_receipt_dict) -> dict:
        """Returns {order_dict, partner_dict, conflicts} — NO ORM calls."""
        partner = self.dedup_service.resolve_or_new(etsy_receipt_dict)
        order = self._build_order_dict(etsy_receipt_dict, partner)
        return {'order': order, 'partner': partner, 'conflicts': []}

# models/order_syncer_adapter.py (ORM model wrapper)
class OrderSyncerAdapter(models.TransientModel):
    _name = 'order.syncer.adapter'

    def sync_etsy_receipt(self, etsy_receipt_dict):
        """ORM wrapper calling ORM-free service."""
        service = OrderSyncer(
            partner_repo=PartnerRepository(self.env),
            order_repo=OrderRepository(self.env),
            dedup_service=PartnerDeduplicationService()
        )
        result = service.sync_order(etsy_receipt_dict)
        # ORM-side: create/write/link
        partner = self.env['res.partner'].create(result['partner'])
        order = self.env['sale.order'].create({**result['order'], 'partner_id': partner.id})
        return order
```

Applies to:

- Etsy API client (API calls, data mapping)
- Email parser (regex parsing, deduplication)
- Gearment webhook receiver (HMAC validation, state machine)
- Catalog import (Excel parsing, SKU derivation)

(e) Deployment View

DEVELOPMENT ENVIRONMENT (DOCKER)



Quick Start:

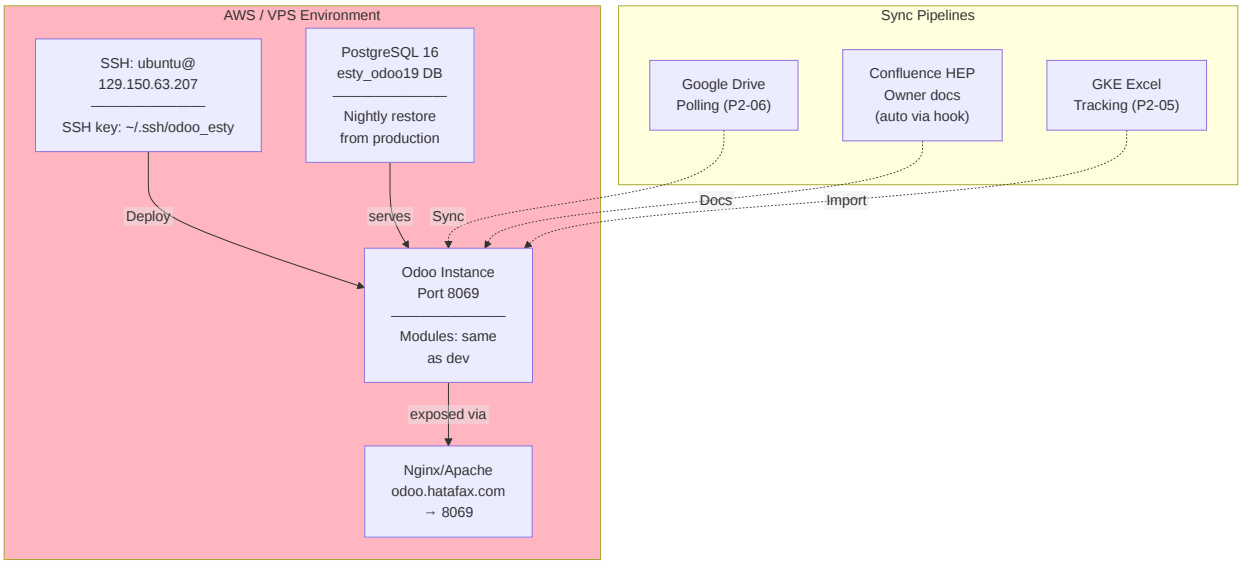
```
cd /home/odoo/odoo_dev/other_projects/odoo19_esty
docker compose build
docker compose up -d
# Wait 10-15s for Odoo startup
curl http://localhost:8169
```

Key Ports:

- **8169** — HTTP web interface (Odoo)
- **8172** — WebSocket (real-time dashboards)
- **5432** — PostgreSQL (direct client connections, testing)

Environment: See `odoo.conf` for settings (db_name, db_user, addons_path, etc.)

STAGING ENVIRONMENT (VPS)



Deployment Process (per `.claude/plans/006-implementation-playbook.md` Phase 5):

1. **Push to feature branch**: `feature/006-master-plan-coding`
2. **Local test**: Docker green (all tests pass, ruff clean)
3. **SSH to staging**: `ssh ubuntu@129.150.63.207`
4. **rsync modules**: `rsync -av --exclude=__pycache__ custom_addons/ ubuntu@129.150.63.207:/opt/odoo/custom_addons/` (per-module, never `--delete`)
5. **Upgrade module**: `docker exec namco_odoo19 odoo -d esty_odoo19 -u multichannel_hub_core --stop-after-init` (or per-module)
6. **Verify**: Browser test at `odoo.hatafax.com`, run E2E script `scripts/e2e_demo_drop_ship_ordertest2.py`
7. **Merge to main** (after E2E green and owner sign-off)

Database:

- Dev: `namco_odoo19` (docker-local, ephemeral if `docker compose down -v`)
- Staging: `esty_odoo19` (VPS, nightly restore from prod snapshot 2026-07-03)

Secrets (never in git):

- `.env` file on staging (GEARMENT_API_KEY, GEARMENT_WEBHOOK_HMAC_SECRET, ETSY_OAUTH_CLIENT_*) — mount via Docker secret or chown root:101
- `secrets/` directory (Google Drive service account JSON) — git-ignored

Logs:

- Dev: `docker logs -f namco_odoo19`
- Staging: SSH to VPS, `tail -f /var/log/odoo/namco_odoo19.log` (or docker logs if containerized)

DEPLOYMENT DIRECTORY STRUCTURE

```
deployment/
├── docker-compose.yml        # Dev orchestration (3 services: odoo, postgres, redis if async)
├── Dockerfile                # Odoo container layer (built on odoo19-ce-base)
├── Dockerfile.base           # Base image (Odoo 19 CE + system deps)
├── odoo.conf                 # Odoo server config (db_name, addons_path, log_level)
├── entrypoint.sh             # Container init script (db init, module install)
├── requirements.txt          # Python deps (openpyxl, requests, etc. — bundled in Odoo)
└── secrets/
    ├── .env.example          # Template (never commit actual .env)
    └── gearment_webhook_secret  # HMAC secret (git-ignored)
```

(f) Key Architectural Decision Records (ADR Digest)

Detailed ADRs live in `specs/006-master-plan/adrs/`. Quick reference table below.

ID	Title	Decision One-Liner	Consequences	Status
ADR-001	Odoo 19 CE Base	Use Odoo 19 CE (no Enterprise modules)	All code must run on CE; no mrp_workorder Enterprise dep; use standard Dropship/MTO	✅ Decided 2026-04-01
ADR-003	Four-Module Decomposition	Split etsy_integration → multichannel_hub_core + etsy_integration + multichannel_hub_fulfillment + design	Channel-agnostic _core allows future amazon_channel, website_channel without code duplication; clear deps (one-way)	✅ Decided 2026-04-15
ADR-004	No Enterprise Modules	No `enterprise_*`, no features gated to Enterprise tier	Rely on Dropship/MTO/quality/web_studio from CE; no timeline-gating on Enterprise features	✅ Decided 2026-04-15
ADR-005	Unified Carrier Model	Single `shipping.carrier` master; per-channel carriers inherit/extend	No redundant carrier seed per channel; tracking unified in sale.order.fulfillment	✅ Decided 2026-05-02
ADR-006	GDrive Primary	Design files + mockups + product images primary → GDrive API; ir.attachment (filestore) ≤2MB fallback	Large filestore writes (>2MB) routed to GDrive; images ≤2MB cached locally for speed	✅ Decided 2026-05-06
ADR-007	Fulfillment Delegation	`sale.order.fulfillment` (reverse-delegated Many2one from sale.order) unifies tracking/label/production state	Single dashboard source; label_status (Selection → Many2one); tracking_number indexed; bus.bus live updates	✅ Decided 2026-05-07
ADR-010	Hybrid Dropship+MTO	Product category field switches pipeline between Gearment dropship and Internal MTO	Shared order.pipeline; distinct states per route; no per-order toggle (category-driven)	✅ Decided 2026-05-12
ADR-014	Central Product Hub (Phase 3)	Odoo as catalog source of truth; Excel bidirectional sync + Etsy API publish chain	Phase 3 new scope; revises Phase 2 mindset (Etsy→Odoo ingest only) to (Odoo↔Etsy bidirectional)	✅ Decided 2026-05-23
ADR-015	Standard-Odoo-First	Before custom field/model, grep standard Odoo addons; if standard exists and fits, use it (owner approval required for custom)	Reduces custom code; leverages CE features; owner veto = work stalls until approval	✅ Decided 2026-05-27
ADR-016	Shop Brand Voice 3-Tier	Listing title/description/image fallback: (1) etsy.listing override, (2) product.template, (3) etsy.shop brand defaults	Reduces per-listing manual overrides; consistent brand across Etsy shop	✅ Implemented P-ENH-ESTY-190
ADR-017	Listing Currency Display	Per-listing currency widget + nightly exchange-rate cron (OpenExchangeRates API)	Displays Etsy-reported currency independent of Odoo company; audit-friendly	✅ Implemented P-ENH-ESTY-195
ADR-018a	Webhook HMAC v2 (Gearment)	Webhooks signed HMAC-SHA256(secret, url_path+nonce+timestamp+base64url(body)); verify before processing	Prevents replay attacks; header format: `X-Connect-Signature: algo=sha256 sig=<hex>`	✅ Implemented P4-01
ADR-018b	OAuth2 PKCE (Etsy)	Etsy OAuth grant via /users/me + PKCE state code; stateless client refresh; per-user shop scope ACL	No server-side session store; shop-level ACL enforced at model level (res.users.etsy_shop_ids M2M)	✅ Implemented P1-10/P1-11

Note: **ADR-018 collision** — both HMAC (Gearment) and OAuth (Etsy) numbered 018. Both documented here; Specs/015 consolidation to rename one variant (e.g., ADR-018-gearment-hmac, ADR-018-etsy-oauth).

Summary

The odoo19_esty system is built on Odoo 19 CE with four-module architecture (foundation + channel + fulfillment + design). Order ingest from Etsy flows through email parser (legacy, Phase 2 deprecates) and API polling (OAuth2 PKCE). Fulfillment routes orders to Gearment dropship (state machine + HMAC webhooks) or Internal MTO (standard Dropship route). Phase 3 adds central product hub (Odoo → Etsy publish via

API). All deployments (dev docker, staging VPS) share codebase; secrets in env vars or secrets/ (git-ignored). Minimum test coverage 80%; two-phase contract (DB schema + ORM unit). No hardcoded secrets, no Enterprise deps, no debug statements.

Next: See Part 2 (Data Model) for entity-relationship details and field inventory.

Part 1 authored 2026-07-03. Mermaid diagrams render in most markdown viewers (GitHub, Confluence, Obsidian). For PDF export, render via https://mermaid.live/SMARTPANTS_PRESERVED_1619 and save as PNG/SVG.

---

title: SDS 02a - Data Model (Domain Clusters 1-3) date: 2026-07-03 status: Draft-for-owner-review source_of_truth: Custom models verified from custom_addons/*.py code

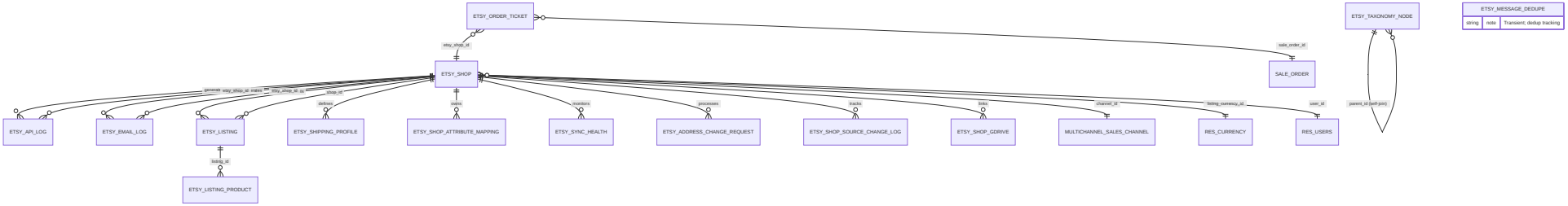
System Design Specification 02a: Entity-Relationship Data Model (Clusters 1-3)

This document provides ER (Entity-Relationship) diagrams and field inventories for all custom Odoo 19 CE models across the four custom modules. Split into two parts: **02a covers Clusters 1-3** (Etsy channel, Listings, Orders+fulfillment); **02b covers Clusters 4-5** (Design, Catalog), inheritance hierarchy, and state machines.

1. Domain Cluster 1: Etsy Channel Integration

The Etsy channel layer manages shop registration, OAuth token storage, email/API ingestion logs, and Etsy-side taxonomies and shipping profiles.

1.1 ER DIAGRAM (MERMAID)



1.2 MODEL REFERENCE TABLE

Model	_name	Inherits	Purpose
Etsy Shop	`etsy.shop`	None	Registration + OAuth + API config per shop
Etsy API Log	`etsy.api.log`	None	Request/response audit for API ingestion
Etsy Email Log	`etsy.email.log`	`mail.thread`, `mail.activity.mixin`	Email ingestion audit trail + chatter
Etsy Listing	`etsy.listing`	None	Read-only mirror of published listings on Etsy
Etsy Listing Product	`etsy.listing.product`	None	Variant/product link for each listing
Etsy Taxonomy Node	`etsy.taxonomy.node`	None	Etsy category tree (self-join hierarchy)
Etsy Shipping Profile	`etsy.shipping.profile`	None	Cached Etsy shipping profiles per shop
Etsy Shop Attribute Mapping	`etsy.shop.attribute.mapping`	None	Shop-level product.attribute → Etsy property_id overrides
Etsy Message Dedupe	`etsy.message.dedupe`	None	Transient; order-message dedup tracking
Etsy Sync Health	`etsy.sync.health`	None	Sync status + error log per shop
Etsy Address Change Request	`etsy.address.change.request`	`mail.thread`, `mail.activity.mixin`	Customer address override requests
Etsy Order Ticket	`etsy.order.ticket`	`mail.thread`, `mail.activity.mixin`	After-sales tickets (return/refund/reship) per order: type, reason, refund amount, approve/reject workflow with chatter audit
Etsy Shop Source Change Log	`etsy.shop.source.change.log`	None	Audit trail of sync_mode / active_source transitions
Etsy Shop GDrive Extension	`etsy.shop` (extended)	`etsy.shop`	File upload token + GDrive folder ID per shop

1.3 KEY FIELDS BY MODEL

etsy.shop

Field	Type	Purpose	Notes
name	Char	Shop name	Required, unique, indexed
active	Boolean	Active flag	Default: True
user_id	Many2one → res.users	Manager	Order visibility scope
etsy_oauth_access_token	Char	OAuth token	system-group only (P0-14)
etsy_oauth_refresh_token	Char	Refresh token	system-group only
etsy_oauth_token_expires_at	Datetime	Token expiry	Tracked during sync
etsy_api_shop_id	Char	Etsy numeric ID	Required before API use; indexed; system-group only
etsy_last_receipt_sync_at	Datetime	Sync watermark	Incremental cutoff; system-group only
sync_mode	Selection	`email_only` / `api_only`	Adapter selector (legacy; superseded by active_source)
active_source	Selection	`api` / `email`	Current upstream source; required
active_source_changed_at	Datetime	Last switch time	Auto-stamped on toggle
sync_audit_mode	Boolean	Read-only sync	Pilot validation mode
auto_recovery	Boolean	Auto-failover	When false, no recovery probe auto-switch
health_check_consecutive_failures	Integer	Failover counter	Reset to 0 on success
recovery_probe_consecutive_successes	Integer	Recovery counter	Reset to 0 on failure
default_taxonomy_id	Char	Taxonomy default	Etsy category; system-group only
default_shipping_profile_id	Char	Shipping default	Etsy profile ID (int64 as Char); system-group only
default_return_policy_id	Char	Return policy default	Etsy policy ID; system-group only
default_readiness_state_id	Char	Readiness state	Etsy processing-time profile; system-group only
listing_currency_id	Many2one → res.currency	Currency override	Shop listing currency (e.g., VND)
default_who_made	Selection	`i_did` / `someone_else` / `collective`	Etsy listing attribute
default_when_made	Char	Production timing	Etsy enum (e.g., `made_to_order`)
default_is_supply	Boolean	Supply flag	Etsy listing attribute
default_title	Char	Listing title	Brand voice tier-1 fallback
default_description	Text	Listing description	Brand voice tier-2 fallback
default_image_1920	Image	Hero image	Brand voice tier-3 fallback
weight_unit_pref	Selection	`oz` / `g`	Unit for item_weight; default: `oz`
dimensions_unit_pref	Selection	`cm` / `in`	Unit for dimensions; default: `cm`
order_ids	One2many → sale.order	Orders linked to this shop	Computed

Field	Type	Purpose	Notes
order_count	Integer	Count of orders	Computed
revenue_total	Float	Sum of order amounts	Computed

Constraints:

- SQL: `UNIQUE(name)` — Shop name unique across system

etsy.api.log

Field	Type	Purpose	Notes
etsy_shop_id	Many2one → etsy.shop	Shop reference	Required; foreign key
method	Char	HTTP method	GET, POST, PATCH, etc.
endpoint	Char	API endpoint	`/v3/application/...` path
request_body	Text	Serialized request	JSON or form-encoded body
response_status	Integer	HTTP status	200, 400, 401, 403, 404, 429, 500, etc.
response_body	Text	Full response	JSON response (capped); error details preserved
duration_ms	Integer	Latency	Request duration in milliseconds
created_at	Datetime	Timestamp	Auto-stamped

etsy.email.log

Field	Type	Purpose	Notes
etsy_shop_id	Many2one → etsy.shop	Shop reference	Required; foreign key
email_subject	Char	Email subject	From Etsy email
email_from	Char	Sender address	Etsy's mail address
email_body	Text	Email body	Full raw message
parsed_order_count	Integer	Orders extracted	Count of SO created/found
parse_status	Selection	`draft` / `parsing` / `linked` / `error`	Ingest lifecycle
parse_error	Text	Error details	Failure reason if parse_status=error
received_at	Datetime	Ingest time	When email arrived
create_date	Datetime	Log timestamp	Auto-stamped

Inherits: `mail.thread`, `mail.activity.mixin` — supports chatter + activities.

etsy.listing

Field	Type	Purpose	Notes
etsy_shop_id	Many2one → etsy.shop	Parent shop	Required
etsy_listing_id	Char	Etsy's listing ID	Primary key at Etsy; indexed
title	Char	Listing title	Read-only from Etsy
description	Text	Listing description	Read-only from Etsy
price	Float	Current price	Last synced
quantity	Integer	Quantity available	Inventory
state	Selection	`active` / `inactive` / `expired`	Etsy state
last_synced_at	Datetime	Sync timestamp	Read-only

etsy.taxonomy.node

Field	Type	Purpose	Notes
etsy_category_id	Char	Etsy numeric ID	Required; indexed
name	Char	Category name	e.g., "Clothing"
parent_id	Many2one → etsy.taxonomy.node	Parent category	Self-join hierarchy; NULL for roots
level	Integer	Tree depth	Computed
path	Char	Full path string	e.g., "Clothing > Women's Clothing"

etsy.shipping.profile

Field	Type	Purpose	Notes
etsy_shop_id	Many2one → etsy.shop	Parent shop	Required
etsy_profile_id	Char	Etsy shipping profile ID	Char (int64 overflow); indexed
name	Char	Profile name	e.g., "Domestic + Intl"
origin_country_iso	Char	ISO country code	Shipping origin
last_synced_at	Datetime	Sync timestamp	Read-only

---

2. Domain Cluster 2: Listings (Multichannel Intent & Attributes)

Listings layer sits between `product.template` (the product master) and channel-specific publishing (e.g., `etsy.listing`). It captures per-channel overrides for marketing copy, images, and metadata.

2.1 ER DIAGRAM (MERMAID)

2.2 MODEL REFERENCE TABLE

Model	_name	Inherits	Purpose
Multichannel Listing	`multichannel.listing`	None	Per-channel/per-shop listing intent (marketing overrides)
Multichannel Listing Attr. Mapping	`multichannel.listing.attribute.mapping`	None	Per-listing attribute → Etsy property_id override (Wave 2+)
Multichannel Enquiry	`multichannel.enquiry`	`mail.thread`, `mail.activity.mixin`	Customer inquiry (presales question)
Multichannel Enquiry (Etsy extend)	`multichannel.enquiry` (ext)	`multichannel.enquiry`	Etsy-specific fields (message ID, etc.)
Multichannel Product Image	`multichannel.product.image`	None	Hero + gallery images for product
Multichannel Sales Channel	`multichannel.sales.channel`	None	Channel master (etsy, website, etc.)

2.3 KEY FIELDS BY MODEL

multichannel.listing

Field	Type	Purpose	Notes
product_tmpl_id	Many2one → product.template	Product	Required; cascade delete; indexed
channel_id	Many2one → multichannel.sales.channel	Sales channel	Required; restrict delete; indexed
shop_ref	Char	Shop identifier	Etsy: shop slug (e.g., “jahandmadeart”); indexed
sequence	Integer	Display order	Per-template ordering; default 10
title	Char	Marketing override title	Empty → falls back to product.template.name
description	Text	Marketing override description	Empty → falls back to product.template.description_sale
image_1920	Image	Hero image override	Empty → falls back to product.template.image_1920
extra_image_ids	One2many → multichannel.product.image	Gallery images	Related (shared with product)
video_attachment_id	Many2one → ir.attachment	Video file	Single video per listing; private attachment only
state	Selection	`draft` / `ready` / `published` / `error`	Lifecycle
external_ref	Char	Channel-side ID	Etsy: listing_id; indexed; copy=False
last_synced_at	Datetime	Last publish time	Read-only
last_sync_error	Text	Publisher error	Computed from product.channel.status (read-only)
display_name	Char	Computed display	Format: “ProductName @ channel/shop”

Constraints:

- SQL UNIQUE (raw init()): `UNIQUE(product_tmpl_id, channel_id, COALESCE(shop_ref, ''))` — One listing per (product, channel, shop) combination.

multichannel.listing.attribute.mapping

Field	Type	Purpose	Notes
listing_id	Many2one → multichannel.listing	Parent listing	Required
attribute_id	Many2one → product.attribute	Product attribute	e.g., “Size”, “Color”
etsy_property_id	Char	Etsy property ID	Etsy numeric property; overrides shop default

multichannel.enquiry

Field	Type	Purpose	Notes
name	Char	Reference number	Auto-generated; indexed
partner_id	Many2one → res.partner	Customer	Required
sale_order_id	Many2one → sale.order	Related SO	Optional (inquiry may be pre-sale)
subject	Char	Inquiry topic	e.g., “Shipping question”
body	Text	Full inquiry text	Message body
state	Selection	`draft` / `replied` / `closed`	Lifecycle
channel_code	Char	Channel source	e.g., “etsy”

Inherits: `mail.thread`, `mail.activity.mixin` — supports chatter.

multichannel.product.image

Field	Type	Purpose	Notes
product_tmpl_id	Many2one → product.template	Product	Required
image	Image	Image binary	Hero or gallery image
sequence	Integer	Order in gallery	Hero first (sequence 0), then gallery

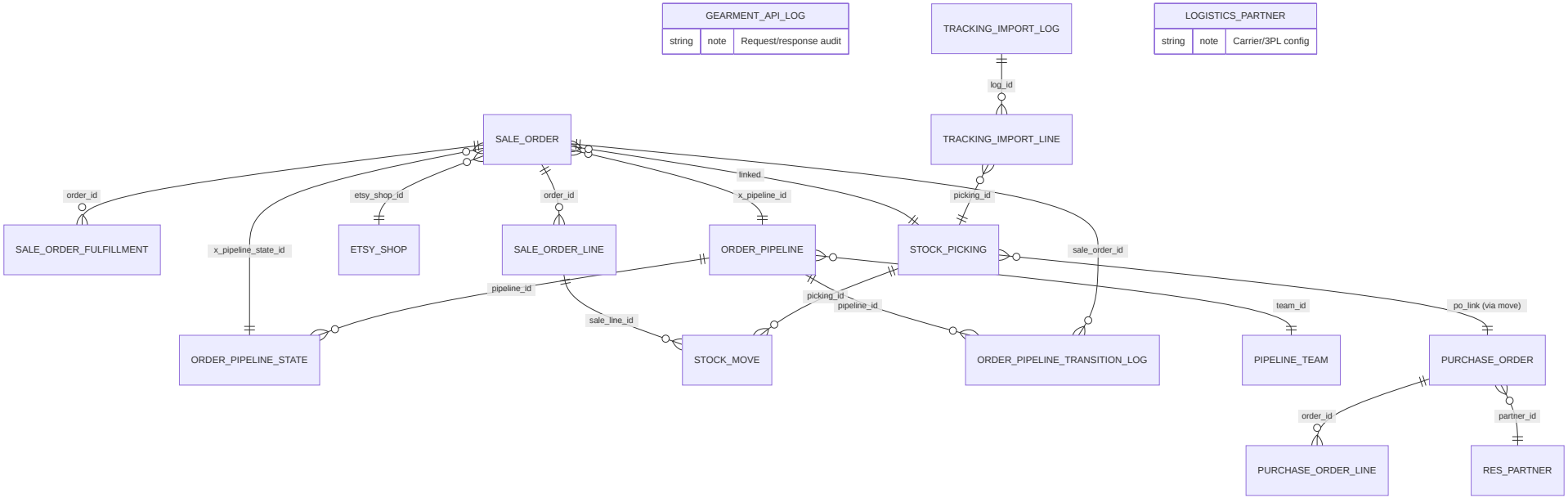
multichannel.sales.channel

Field	Type	Purpose	Notes
name	Char	Channel display name	e.g., “Etsy Shop 1”, “Company Website”
code	Char	Channel identifier	e.g., “etsy”, “website”; indexed
active	Boolean	Active flag	Default: True

3. Domain Cluster 3: Orders + Fulfillment (Pipeline, Purchase, Picking, Tracking)

The order-fulfillment layer connects `sale.order` across three inbound pathways (Etsy email, Etsy API, manual), stages them through a configurable pipeline (VN internal, Gearment POD, hybrid MTO), and routes them to warehouse (stock.picking) or suppliers (purchase.order, gearment API).

3.1 ER DIAGRAM (MERMAID)



3.2 MODEL REFERENCE TABLE

Model	_name	Inherits	Purpose
Sale Order	`sale.order` (ext)	`sale.order`	Etsy order ingestion + pipeline state
Sale Order Fulfillment	`sale.order.fulfillment` (core & ext)	`mail.thread`	Fulfillment orchestration (phiếu gửi hang)
Sale Order Line	`sale.order.line` (ext)	`sale.order.line`	Line-level fulfillment details
Stock Picking	`stock.picking` (ext)	`stock.picking`	Warehouse picking + ship label generation
Stock Move	`stock.move` (ext)	`stock.move`	Line-item inventory move
Purchase Order	`purchase.order` (ext)	`purchase.order`	Dropship PO to suppliers (Gearment, internal vendors)
Purchase Order Line	(std Odoo)	—	Line-item purchase detail
Gearment API Log	`gearment.api.log`	None	Gearment quote/order/tracking API audit
Tracking Import Log	`tracking.import.log`	`mail.thread`, `mail.activity.mixin`	Batch tracking-number ingestion
Tracking Import Line	`tracking.import.line`	None	Individual tracking number row
Order Pipeline	`order.pipeline`	`mail.thread`, `mail.activity.mixin`	Pipeline master (VN Internal, Gearment, Hybrid)
Order Pipeline State	`order.pipeline.state`	`mail.thread`, `mail.activity.mixin`	Pipeline stage (e.g., draft, quoted, confirmed, shipped)
Order Pipeline Transition Log	`order.pipeline.transition.log`	None	Audit trail of state changes
Pipeline Team	`pipeline.team`	`mail.thread`, `mail.activity.mixin`	Team responsible for a pipeline stage
Logistics Partner	`logistics.partner`	`mail.thread`	Carrier/3PL configuration (USPS, GKE, etc.)
Label Status Option	`label.status.option`	`mail.thread`	Master data for order label statuses (name, code, color, sequence, bucket); unique-name constraint

3.3 KEY FIELDS BY MODEL

sale.order (extensions)

Field	Type	Purpose	Notes
etsy_shop_id	Many2one → etsy.shop	Etsy shop source	NULL for non-Etsy orders
etsy_order_id	Char	Etsy numeric order ID	Etsy's unique ID; indexed
etsy_buyer_user_id	Char	Etsy buyer ID	Buyer's Etsy account ID
etsy_receipt_id	Char	Etsy receipt ID	Etsy receipt identifier; indexed
x_pipeline_id	Many2one → order.pipeline	Pipeline	e.g., "VN Internal", "Gearment POD"
x_pipeline_state_id	Many2one → order.pipeline.state	Current stage	e.g., "draft", "quoted", "confirmed", "shipped"
x_gearment_outbound_state	Selection	`draft` / `quoted` / `operator review` / `confirmed`	Gearment POD-specific state (if pipeline=Gearment)
x_gearment_po_id	Char	Gearment quote/order ID	External Gearment ID; indexed
x_production_notes	Text	Internal notes	Production team comments

sale.order.fulfillment

Field	Type	Purpose	Notes
name	Char	Document number	Auto-assigned; "SOF" prefix
order_id	Many2one → sale.order	Parent SO	Required
fulfillment_type	Selection	`internal prod` / `dropship` / `hybrid`	Production route
state	Selection	Lifecycle states	Computed from stock.picking, purchase.order
picking_ids	One2many → stock.picking	Warehouse picks	Outbound pickings for this fulfillment
purchase_order_ids	One2many → purchase.order	Supplier orders	Dropship POs linked to this fulfillment

Inherits: `mail.thread` — chatter enabled.

stock.picking (extensions)

Field	Type	Purpose	Notes
x_gearment_shipping_label_url	Char	Label download	URL to Gearment's shipping label PDF
x_logistics_partner_id	Many2one → logistics.partner	Carrier	USPS, GKE, etc.
x_tracking_number	Char	Carrier tracking	Auto-imported from tracking.import.log
x_tracking_imported_at	Datetime	Tracking sync time	When tracking was injected

purchase.order (extensions)

Field	Type	Purpose	Notes
x_sale_order_id	Many2one → sale.order	Linked SO	Dropship context
x_gearment_quote_id	Char	Gearment quote ID	If vendor is Gearment
x_gearment_order_id	Char	Gearment order ID	After PO confirmation
x_gearment_quote_state	Selection	Quote lifecycle	Mirrored from Gearment API
x_requested_delivery_date	Date	Target delivery	Customer-promised date

order.pipeline

Field	Type	Purpose	Notes
name	Char	Pipeline name	e.g., “VN Internal Production”
code	Char	Pipeline identifier	e.g., “vn_internal”, “gearment_pod”; indexed
active	Boolean	Active flag	Default: True
team_id	Many2one → pipeline.team	Responsible team	e.g., “Production Team”
state_ids	One2many → order.pipeline.state	Stages	Ordered list of workflow stages
transition_log_ids	One2many → order.pipeline.transition.log	Audit	State change history

Inherits: `mail.thread`, `mail.activity.mixin`.

order.pipeline.state

Field	Type	Purpose	Notes
name	Char	Stage name	e.g., “Quote Pending”
code	Char	Stage code	e.g., “quoted”, “confirmed”; indexed
pipeline_id	Many2one → order.pipeline	Parent pipeline	Required
sequence	Integer	Stage order	Determines workflow progression
allow_confirm	Boolean	Confirmable	If true, SO can be confirmed at this stage
auto_create_picking	Boolean	Auto-picking	If true, transition creates stock.picking
auto_create_invoice	Boolean	Auto-invoice	If true, transition triggers invoicing

order.pipeline.transition.log

Field	Type	Purpose	Notes
sale_order_id	Many2one → sale.order	Order reference	FK
pipeline_id	Many2one → order.pipeline	Pipeline context	FK
old_state_id	Many2one → order.pipeline.state	Previous stage	NULL if first transition
new_state_id	Many2one → order.pipeline.state	New stage	Required
changed_by_user_id	Many2one → res.users	Who changed	Auto-stamped
changed_at	Datetime	Transition time	Auto-stamped
change_type	Selection	`manual` / `automatic`	User action or system sync
notes	Text	Reason/details	Optional audit note

gearment.api.log

Field	Type	Purpose	Notes
purchase_order_id	Many2one → purchase.order	Context	Required FK
method	Char	HTTP method	GET, POST, PATCH
endpoint	Char	API endpoint	Gearment API path
request_body	Text	Request	JSON body
response_status	Integer	HTTP status	200, 400, 401, 409, 500, etc.
response_body	Text	Response	JSON response (truncated if large)
duration_ms	Integer	Latency	Milliseconds
created_at	Datetime	Timestamp	Auto-stamped

tracking.import.log

Field	Type	Purpose	Notes
name	Char	Import reference	e.g., “GKE Tracking Batch 2026-07-03”
import_source	Char	Source system	e.g., “gke”, “usps”, “manual”
import_file	Binary	Uploaded file	CSV or JSON
import_status	Selection	`draft` / `processing` / `completed` / `error`	Lifecycle
processed_count	Integer	Successfully imported	Count of tracking numbers linked
error_count	Integer	Failed rows	Count of failed imports
import_error	Text	Error details	First error message
imported_at	Datetime	Completion time	Auto-stamped

Inherits: `mail.thread`, `mail.activity.mixin`.

tracking.import.line

Field	Type	Purpose	Notes
log_id	Many2one → tracking.import.log	Parent import	Required
picking_id	Many2one → stock.picking	Target picking	Links tracking to warehouse pick
tracking_number	Char	Carrier tracking	e.g., “1Z999AA10123456784”
carrier_code	Char	Carrier identifier	e.g., “gke”, “usps”
import_status	Selection	`pending` / `linked` / `error`	Line status
import_error	Text	Error reason	If link failed

logistics.partner

Field	Type	Purpose	Notes
name	Char	Carrier/3PL name	e.g., “GKE Logistics”
code	Char	Carrier code	e.g., “gke”, “usps”; indexed
active	Boolean	Active flag	Default: True
api_endpoint	Char	API URL	For automated tracking/label pulls
api_key	Char	API credential	system-group only
tracking_number_format	Char	Regex pattern	Validate incoming tracking numbers

Inherits: `mail.thread`.

pipeline.team

Field	Type	Purpose	Notes
name	Char	Team name	e.g., “Production Team”
member_ids	Many2many → res.users	Team members	Users in this team
pipeline_ids	One2many → order.pipeline	Pipelines	Pipelines this team operates

Inherits: `mail.thread`, `mail.activity.mixin`.

---

4. Summary of Cluster 1-3 Relationships

Relationship	Source	Target	Multiplicity	Purpose
Shop → Orders	etsy.shop	sale.order	1:N	Each order ingested from a shop
Shop → Listings	etsy.shop	etsy.listing	1:N	Shop hosts multiple listings
Listing → Product	etsy.listing	etsy.listing.product	1:N	Listing variants
Product → Listing Intent	product.template	multichannel.listing	1:N	Multiple channels per product
Order → Fulfillment	sale.order	sale.order.fulfillment	1:N	May split into multiple fulfillments
Fulfillment → Picking	sale.order.fulfillment	stock.picking	1:N	Warehouse pick per fulfillment line
Order → Pipeline	sale.order	order.pipeline	N:1	Order assigned to one pipeline
Pipeline → States	order.pipeline	order.pipeline.state	1:N	Pipeline defines its workflow
Order State Change → Audit	order.pipeline.transition.log	sale.order	N:1	All transitions logged
Picking → Tracking	stock.picking	tracking.import.line	1:N	Multiple tracking numbers? (1:1 expected)
Purchase Order → Gearment Log	purchase.order	gearment.api.log	1:N	Each API call logged

---

Document Metadata

- **Date:** 2026-07-03
- **Status:** Draft for owner review
- **Odoo Version:** 19.0 CE
- **Module Scope:** etsy_integration, multichannel_hub_core, multichannel_hub_fulfillment (partial), design (partial)
- **Related SDS:** 02b-data-model.md (Clusters 4-5, inheritance hierarchy, state machines)

- **Code Verification:** All models, fields, and relationships verified against `/home/odoo/odoo_dev/other_projects/odoo19_esty/custom_addons/` source code on 2026-07-03.

---

title: SDS 02b - Data Model (Domain Clusters 4-5, Inheritance, State Machines) date: 2026-07-03 status: Draft-for-owner-review source_of_truth: Custom models verified from custom_addons/*.py code

System Design Specification 02b: Entity-Relationship Data Model (Clusters 4-5, Inheritance, State Machines)

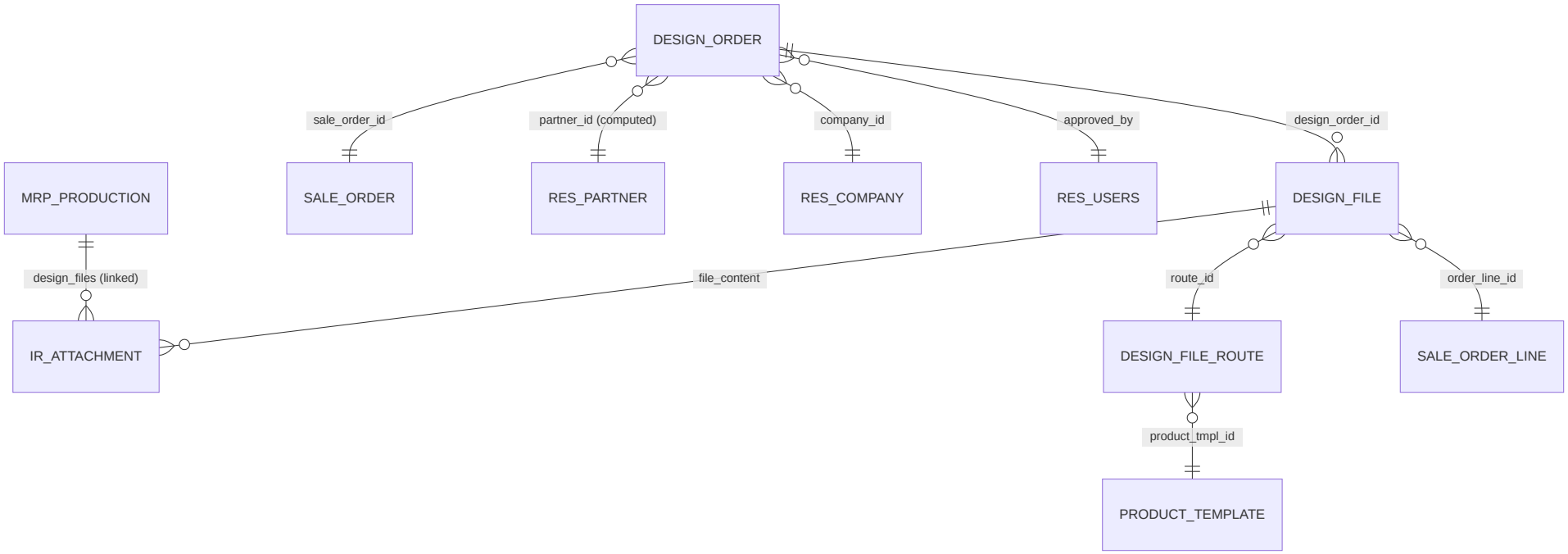
Continuation of **02a**. This document covers the Design cluster, Catalog/Product hub cluster, the full Odoo inheritance hierarchy, and state machines across all models.

---

5. Domain Cluster 4: Design Orders & Files (Document Control)

Design orders (`design.order`) are first-class documents parallel to `sale.order` and `mrp.production`. They track the approval flow for customer-submitted artwork, organize design files, and attach approved files to manufacturing orders.

5.1 ER DIAGRAM (MERMAID)



5.2 MODEL REFERENCE TABLE

Model	_name	Inherits	Purpose
Design Order	`design.order`	`mail.thread`, `mail.activity.mixin`	Document for design approval flow (phiếu design)
Design File (core)	`design.file`	`mail.thread`, `mail.activity.mixin`	Design file (artwork, mockup, proof)
Design File (design ext)	`design.file`	`design.file` (_inherit)	Extension with design_order_id link
Design File Route	`design.file.route`	`mail.thread`, `mail.activity.mixin`	Route template for design uploads (deprecated/archive)
Design File Upload Wizard	`design.file.upload.wizard`	—	Transient; file upload form
Sale Order (design ext)	`sale.order` (_inherit)	`sale.order`	Extensions: auto-create design orders
MO (design ext)	`mrp.production` (_inherit)	`mrp.production`	ESTY-249: computed `design_ready` + `design_order_id` (informational MO badge)

5.3 KEY FIELDS BY MODEL

design.order

Field	Type	Purpose	Notes
name	Char	Document reference	Auto-assigned from sequence; readonly after creation; tracked
sale_order_id	Many2one → sale.order	Parent SO	Required; cascade delete; indexed; tracked
partner_id	Many2one → res.partner	Customer	Stored, computed from sale_order_id.partner_id; readonly
company_id	Many2one → res.company	Company	Required; defaults to current company; indexed
state	Selection	`pending` / `proof_sent` / `approved` / `rejected`	Readonly after pending; tracked; indexed; default: `pending`
design_file_ids	One2many → design.file	Child files	All design files linked to this order
design_files_count	Integer	Total file count	Computed; readonly
approved_files_count	Integer	Approved file count	Computed; readonly
approved_by	Many2one → res.users	Approver	Readonly; null
approved_at	Datetime	Approval timestamp	Readonly; auto-stamped
rejection_reason	Text	Rejection details	Shown when state=rejected; tracked
create_date	Datetime	Creation timestamp	Auto-stamped
write_date	Datetime	Last modification	Auto-stamped

Constraints:

- SQL: `UNIQUE(sale_order_id)` — One design order per SO (enforced via init() raw SQL per project pattern).

design.file

Field	Type	Purpose	Notes
name	Char	File name	Display name; required
order_id	Many2one → sale.order	Header-level link	Design file at SO level (NULL if line-level)
order_line_id	Many2one → sale.order.line	Line-level link	Design file at line level (NULL if header-level)
design_order_id	Many2one → design.order	Parent order	Extension field (added by design module)
storage_mode	Selection	`small` / `url` / `gdrive`	Where the file is stored
file_url	Char	External URL	If storage_mode=url; clickable link
gdrive_preview_url	Char	GDrive preview	If storage_mode=gdrive; preview link
design_file	Binary	File binary	If storage_mode=small; binary data
file_size	Integer	File size	In bytes; computed
route_id	Many2one → design.file.route	Route (deprecated)	Legacy field; may be NULL
state	Selection	`draft` / `proof sent` / `approved` / `rejected`	File approval state; tracked
created_by_user_id	Many2one → res.users	Uploader	Who uploaded the file
created_at	Datetime	Upload time	Auto-stamped

Inherits: `mail.thread`, `mail.activity.mixin` — chatter per file.

design.file.route

Field	Type	Purpose	Notes
name	Char	Route name	e.g., “Custom T-Shirt Design”
product_tmpl_id	Many2one → product.template	Product template	Route scoped to a product
active	Boolean	Active flag	Default: True

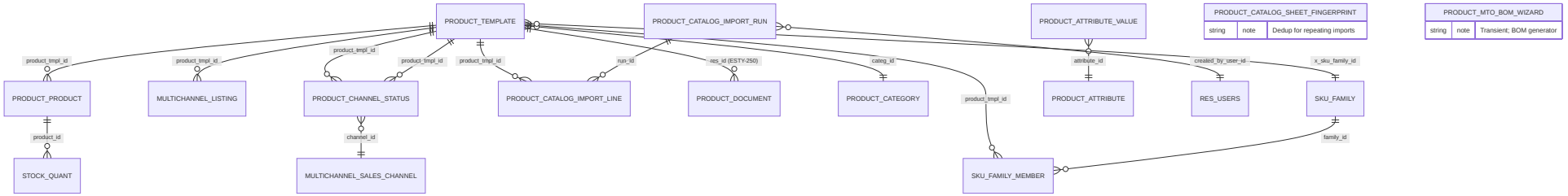
Note: This model is largely deprecated (use direct `design.file` links instead).

---

6. Domain Cluster 5: Catalog & Product Hub (SKU, Import, Channel Status)

The catalog layer manages product master data, channel-specific status tracking, bulk imports from spreadsheets, and SKU family groupings.

6.1 ER DIAGRAM (MERMAID)



6.2 MODEL REFERENCE TABLE

Model	_name	Inherits	Purpose
Product Template (core+ext)	`product.template`	`product.template` (ext x2)	Product master + channel SKU + import tracking
Product Product (core+ext)	`product.product`	`product.product` (_inherit)	Variant + Etsy link
Product Attribute Value (core+ext)	`product.attribute.value`	`product.attribute.value` (_inherit)	Variant attribute value + Etsy mapping
Product Attribute (ext)	`product.attribute` (_inherit)	`product.attribute`	Attribute definition + Etsy mapping
Product Document (ext)	`product.document` (_inherit)	`product.document`	Design files (AI/PSD/PDF) tagged for Original Design tab (ESTY-250)
Product Channel Status	`product.channel.status`	None	Per-channel publish status + error log
Product Catalog Import Run	`product.catalog.import.run`	None	Batch import execution record
Product Catalog Import Line	`product.catalog.import.line`	None	Individual row from XLSX import
Product Catalog Sheet Fingerprint	`product.catalog.sheet.fingerprint`	None	Dedup tracking for repeating imports
SKU Family	`mhc.sku.family`	None	Grouping of related SKUs
Product MTO BOM Wizard	`product.mto.bom.wizard`	—	Transient; BOM generation helper
Product Category (ext)	`product.category` (_inherit)	`product.category`	Category + pipeline assignment

6.3 KEY FIELDS BY MODEL

product.template (extensions)

Field	Type	Purpose	Notes
name	Char	Product name	Canonical product name
description_sale	Text	Sales description	Tier-1 fallback for listing description
image_1920	Image	Hero image	Tier-1 fallback for listing image
x_extra_image_ids	One2many → multichannel.product.image	Gallery images	Shared across all channels (Wave 2)
x_original_design_ids	One2many → product.document	Original design files	Design source files (AI/PSD/PDF) filtered via domain `x_is_original_design=True` (ESTY-250); distinct from generic Documents smart button
x_sku_family_id	Many2one → mhc.sku.family	SKU family	Logical grouping of related variants
categ_id	Many2one → product.category	Category	Odoo category + pipeline assignment
list_price	Float	List price (Monetary)	Standard Odoo selling price
standard_price	Float	Cost price	Standard Odoo cost
weight	Float	Weight	In kg (Odoo standard)
product_variant_ids	One2many → product.product	Variants	All variants of this product
product_variant_count	Integer	Variant count	Computed; readonly
attribute_line_ids	One2many → product.attribute.line	Variant attributes	Attributes that drive variants
x_etsy_category_id	Char	Etsy category	Cached Etsy taxonomy node ID (Wave 2)
x_etsy_shipping_profile_id	Char	Shipping profile	Cached Etsy profile ID (Wave 2)

product.product (extensions)

Field	Type	Purpose	Notes
product_tmpl_id	Many2one → product.template	Template	Parent template
default_code	Char	SKU	Standard Odoo SKU field
x_etsy_listing_ids	One2many → etsy.listing.product	Etsy listings	Listings carrying this variant

product.channel.status

Field	Type	Purpose	Notes
product_tmpl_id	Many2one → product.template	Product	Required; indexed
channel_id	Many2one → multichannel.sales.channel	Channel	Required; indexed
publish_state	Selection	`draft` / `published` / `error` / `out_of_stock`	Current status
external_ref	Char	Channel-side ID	e.g., Etsy listing_id
last_sync_at	Datetime	Last publish time	Read-only
last_sync_error	Text	Publisher error	Latest error message
is_active	Boolean	Active on channel	Should it be available?

Constraints:

- SQL UNIQUE: `UNIQUE(product_tmpl_id, channel_id)` — One status per (product, channel).

product.catalog.import.run

Field	Type	Purpose	Notes
name	Char	Import reference	e.g., “Catalog 2026-06 Q2”
import_file	Binary	Uploaded XLSX	Raw spreadsheet data
import_status	Selection	`draft` / `processing` / `completed` / `error`	Lifecycle
processed_count	Integer	Processed rows	Count of rows parsed
error_count	Integer	Failed rows	Count of validation failures
import_error	Text	Error details	First error message
created_by_user_id	Many2one → res.users	Uploader	Who initiated import
created_at	Datetime	Upload time	Auto-stamped
imported_at	Datetime	Completion time	Auto-stamped; NULL if not yet completed
import_line_ids	One2many → product.catalog.import.line	Detail rows	Each imported row

product.catalog.import.line

Field	Type	Purpose	Notes
run_id	Many2one → product.catalog.import.run	Parent run	Required; FK
product_tmpl_id	Many2one → product.template	Linked product	NULL if not yet matched
sku	Char	SKU from XLSX	Reference for matching
name	Char	Product name	From XLSX
description	Text	Product description	From XLSX
cost_price	Float	Cost	From XLSX
list_price	Float	Selling price	From XLSX
weight	Float	Weight	From XLSX (kg)
import_status	Selection	`pending` / `matched` / `created` / `error`	Line status
import_error	Text	Validation error	If import_status=error

mhc.sku.family

Field	Type	Purpose	Notes
name	Char	Family name	e.g., “Classic T-Shirt”
code	Char	Family code	e.g., “classic-t”; indexed
description	Text	Family description	Marketing overview
member_ids	One2many → product.template	Members	Products in this family
active	Boolean	Active flag	Default: True

product.attribute (extensions)

Field	Type	Purpose	Notes
name	Char	Attribute name	e.g., “Size”, “Color”
x_etsy_property_id	Char	Default Etsy property	Etsy property_id mapping (Wave 2)

product.attribute.value (extensions)

Field	Type	Purpose	Notes
name	Char	Attribute value	e.g., “Small”, “Red”
attribute_id	Many2one → product.attribute	Parent attribute	Required
x_etsy_value_id	Char	Etsy value ID	Etsy-side value mapping (Wave 2)

product.document (extensions, ESTY-250)

Field	Type	Purpose	Notes
x_is_original_design	Boolean	Original design tag	Tags document for display on product’s “Original Design” tab; untagged rows excluded
name	Char	File name	Inherited from ir.attachment via delegation
datas	Binary	File content	Inherited from ir.attachment; size cap enforced if x_is_original_design=True
res_model	Char	Resource model	Always `product.template` for products
res_id	Integer	Resource ID	product.template.id; indexed

Constraints:

- `@api.constrains('x_is_original_design', 'datas')` — Enforces size cap (default 10 MB) via multichannel_hub.large_file_threshold_bytes config parameter when x_is_original_design=True.

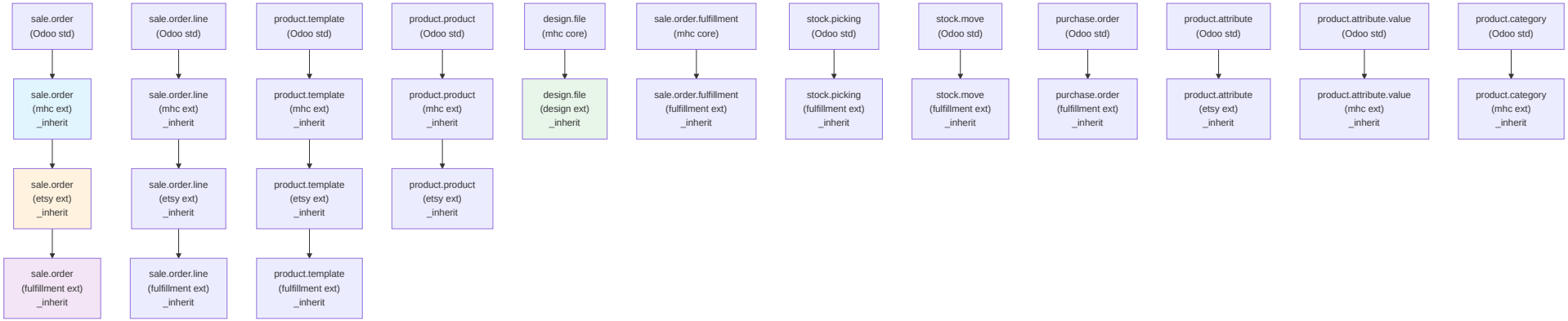
product.category (extensions)

Field	Type	Purpose	Notes
name	Char	Category name	Standard Odoo field
parent_id	Many2one → product.category	Parent category	Standard Odoo hierarchy
x_pipeline_id	Many2one → order.pipeline	Default pipeline	Products in this category default to this pipeline

---

7. Inheritance Hierarchy (Flowchart)

All custom model extensions and their inheritance chains:



Color Legend:

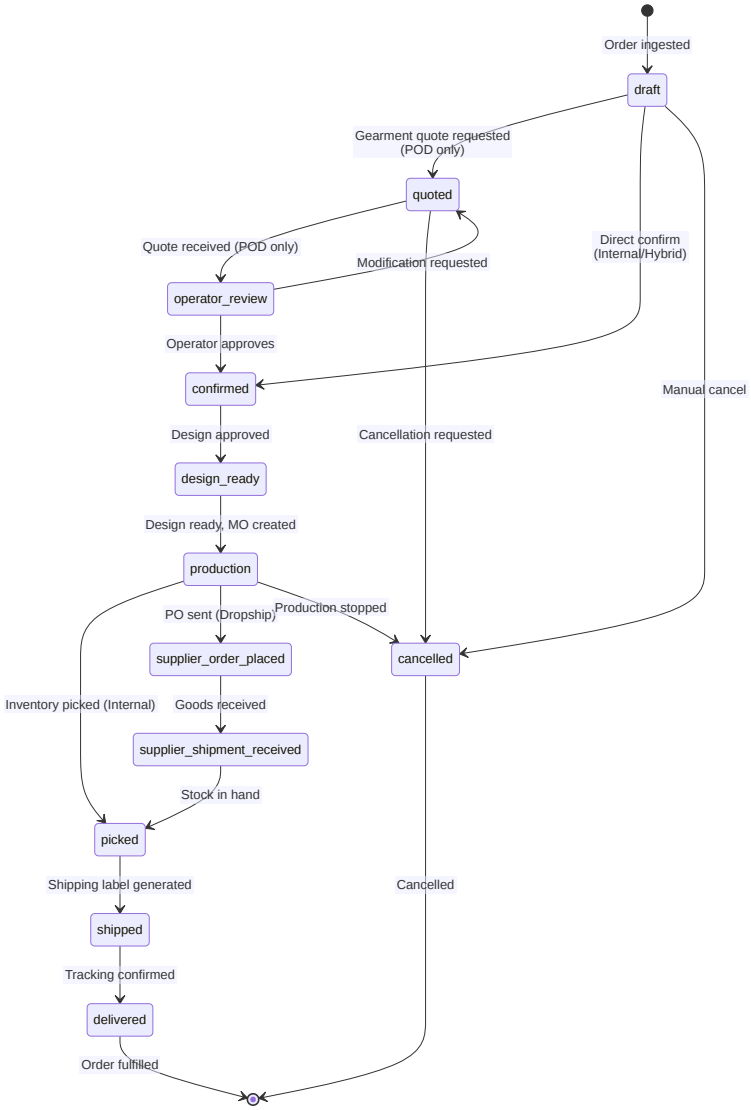
- Blue: `multichannel_hub_core` (mhc)
- Orange: `etsy_integration`
- Purple: `multichannel_hub_fulfillment`
- Green: `design`

---

8. State Machines (Mermaid)

8.1 SALE ORDER PIPELINE STATES

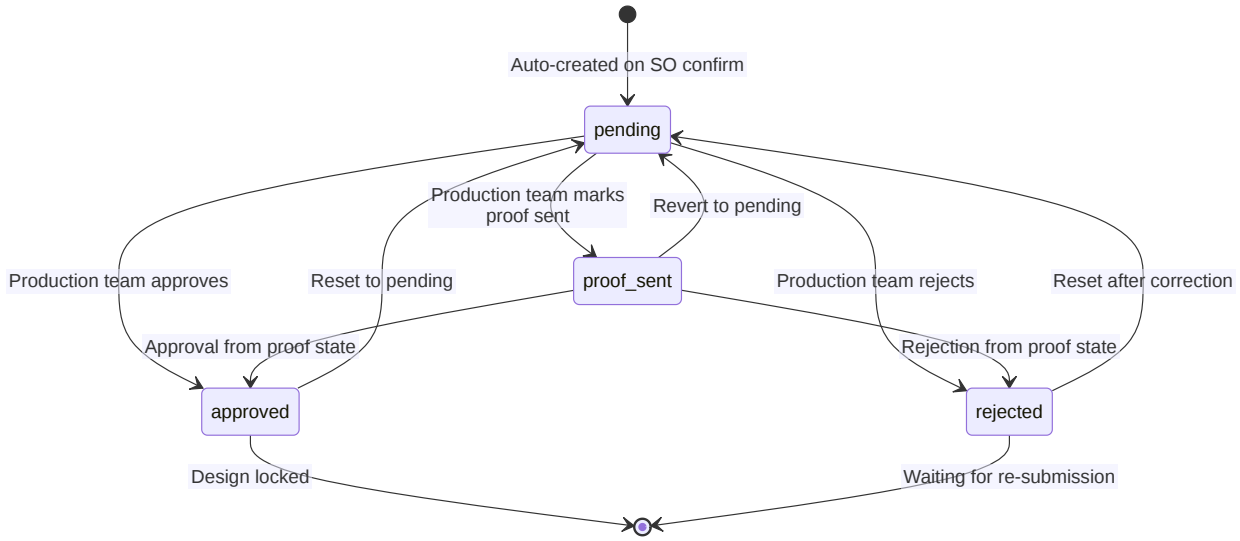
Generic pipeline structure (varies per pipeline instance; 3 example pipelines):



States by Pipeline:

Stage	Draft	Quoted	Operator Review	Confirmed	Design Ready	Production	Picked	Shipped	Delivered
VN Internal	✓	-	-	✓	✓	✓	✓	✓	✓
Gearment POD	✓	✓	✓	✓	✓	✓	✓	✓	✓
Hybrid MTO	✓	-	-	✓	✓	✓	✓	✓	✓

8.2 DESIGN ORDER STATES

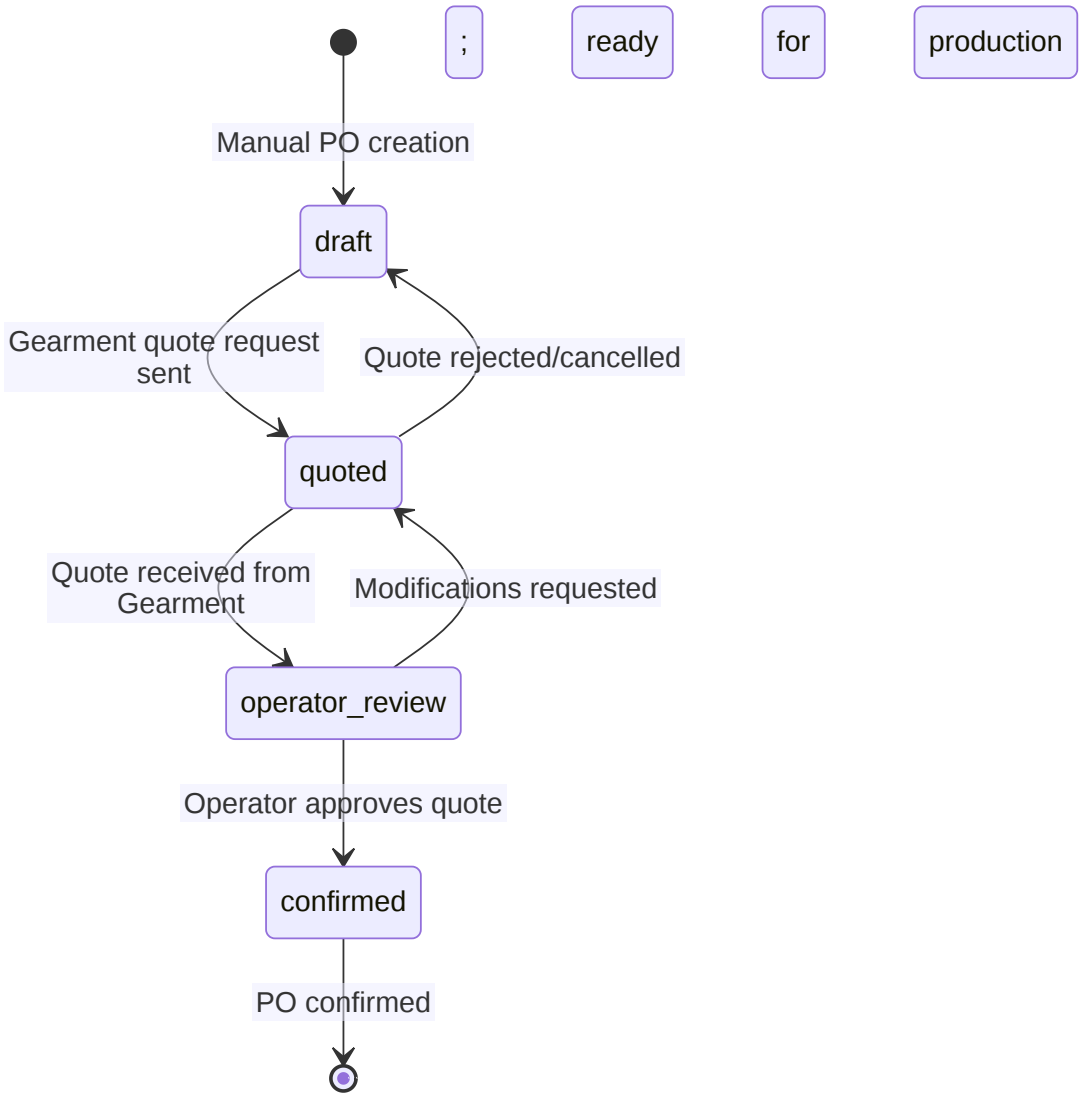


State Transitions:

- **pending → proof_sent**: Production team marks design as “proof sent to customer”
- **pending/proof_sent → approved**: Design approved; attaches files to MO; advances SO to `design_ready` stage
- **pending/proof_sent → rejected**: Requires non-empty rejection_reason; tracked in chatter
- **approved/rejected → pending**: Production team resets (for revisions)

8.3 GEARMENT POD ORDER STATES (X_GEARMENT_OUTBOUND_STATE)

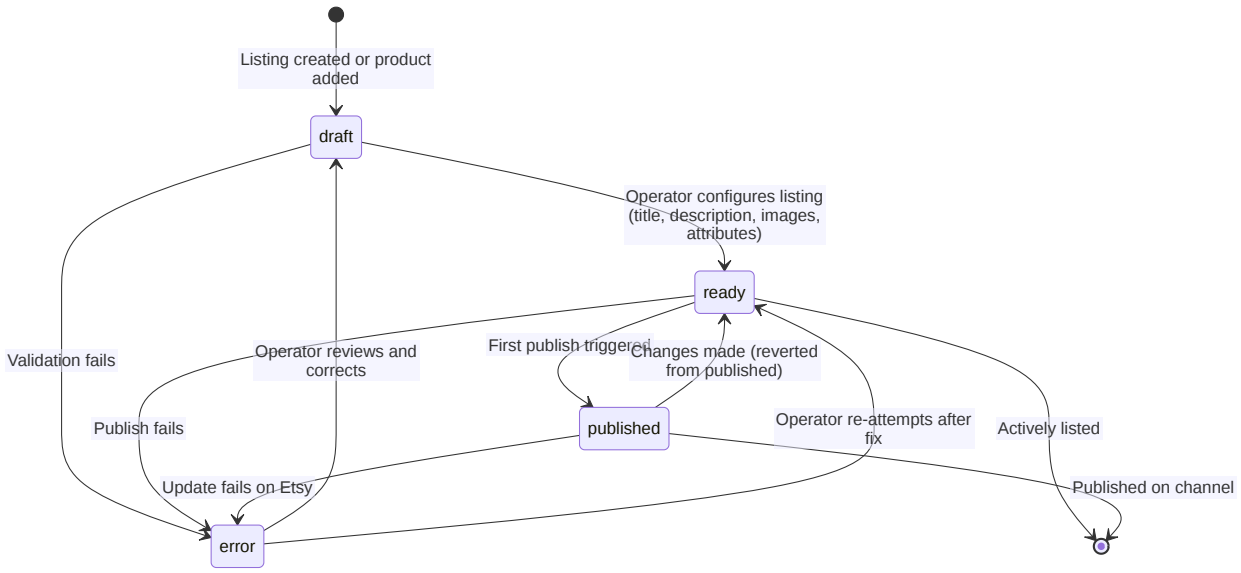
POD-specific substate (only active when `x_pipeline_id` = Gearment POD pipeline):



Relationship to sale.order.x_pipeline_state_id:

- `x_gearment_outbound_state` = draft → sale.order state = confirmed (pending quote)
- `x_gearment_outbound_state` = quoted → sale.order state = quoted (awaiting operator review)
- `x_gearment_outbound_state` = operator_review → sale.order state = operator_review
- `x_gearment_outbound_state` = confirmed → sale.order state = confirmed (production-ready)

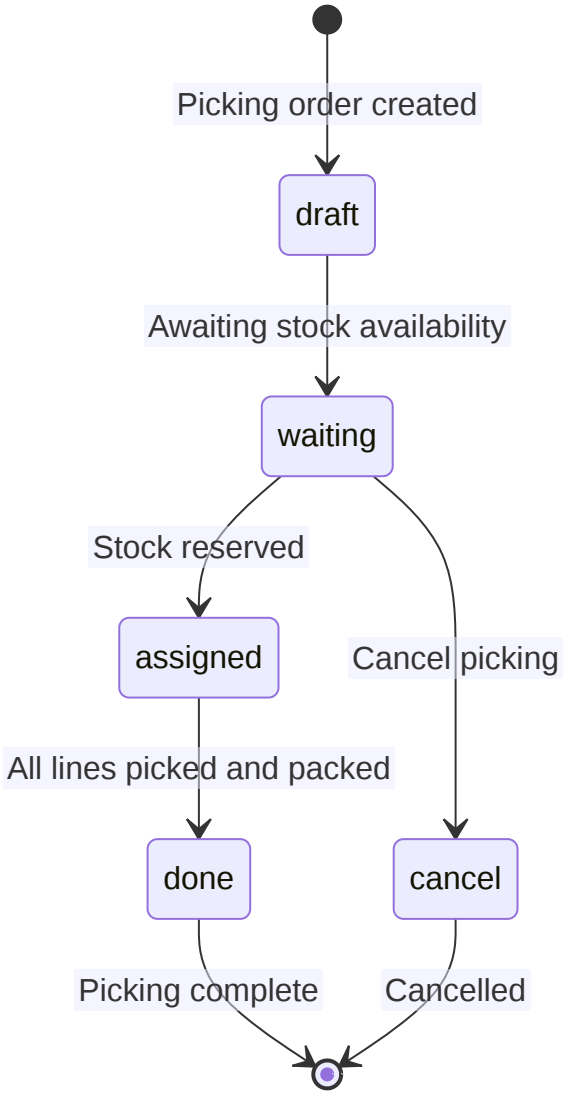
8.4 MULTICHANNEL LISTING STATES



Triggers:

- Draft → Ready: All required fields filled (title, channel, product link)
- Ready → Published: Publisher runs successfully
- Published/Ready → Error: Validation or API failure
- Error → Draft/Ready: Operator correction

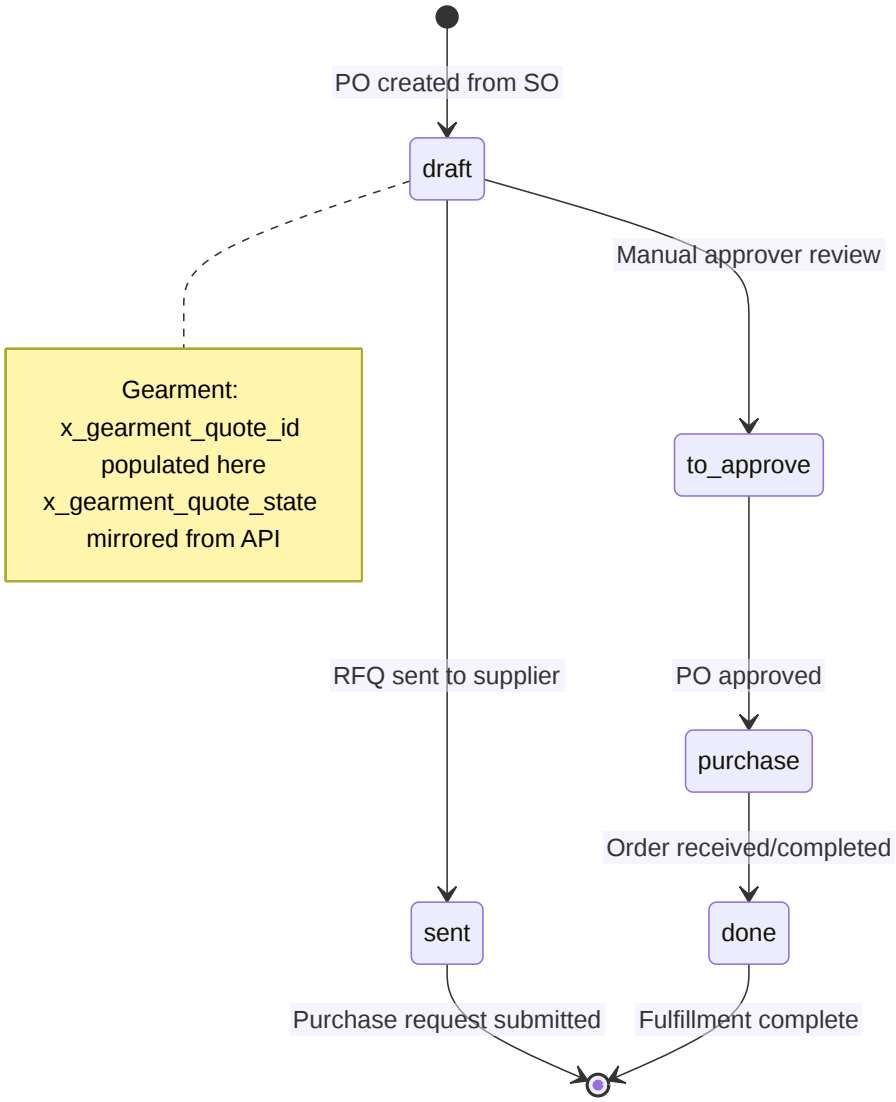
8.5 STOCK PICKING STATES (EXTENDED)



Extensions by fulfillment module:

- `x_gearment_shipping_label_url` populated when label PDF generated
- `x_logistics_partner_id` assigned based on carrier
- `x_tracking_number` populated by tracking.import.log

8.6 PURCHASE ORDER STATES (EXTENDED, GEARMENT/DROPSHIP)



Gearment-Specific Fields:

- `x_gearment_quote_id` = Gearment quote reference
- `x_gearment_order_id` = Gearment order ID (after confirmation)
- `x_gearment_quote_state` = Synced from Gearment API (`draft`, `quoted`, `confirmed`, etc.)

---

9. Computed Fields & Related Fields Summary

COMPUTED (READ-ONLY) FIELDS

Model	Field	Compute Method	Dependencies
design.order	design_files_count	_compute_design_files_count	design_file_ids
design.order	approved_files_count	_compute_design_files_count	design_file_ids.state
mrp.production	design_ready	_compute_design_readiness	origin, company_id (non-stored; keyed on origin==SO.name)
mrp.production	design_order_id	_compute_design_readiness	origin, company_id (non-stored)
multichannel.listing	display_name	_compute_display_name	product_tmpl_id, channel_id, shop_ref
multichannel.listing	last_sync_error	_compute_last_sync_error	product_channel_status (via join)
etsy.shop	order_count	_compute_order_count	order_ids (one2many)
etsy.shop	revenue_total	_compute_order_count	order_ids.amount_total
sale.order	etsy_order_display	_compute_etsy_order_display	etsy_order_id (if populated)
product.template	product_variant_count	(std Odoo)	product_variant_ids
product.attribute.value	display_name	(std Odoo)	attribute_id, name

RELATED FIELDS (STORED, LINKED)

Model	Field	Related Source	Purpose
design.order	partner_id	sale_order_id.partner_id	Denormalized for sorting/filtering
design.file	order_id	order_line_id.order_id	Nullable; file can be header- or line-scoped
multichannel.listing	extra_image_ids	product_tmpl_id.x_extra_image_ids	Surface product gallery in listing form

---

10. SQL Constraints (Raw init() Pattern)

Per project memory `project_sql_constraints_drift.md`, constraints are deployed both via ORM `_sql_constraints` and raw SQL in `init()` (belt-and-braces). Odoo 19 has issues with declarative enforcement.

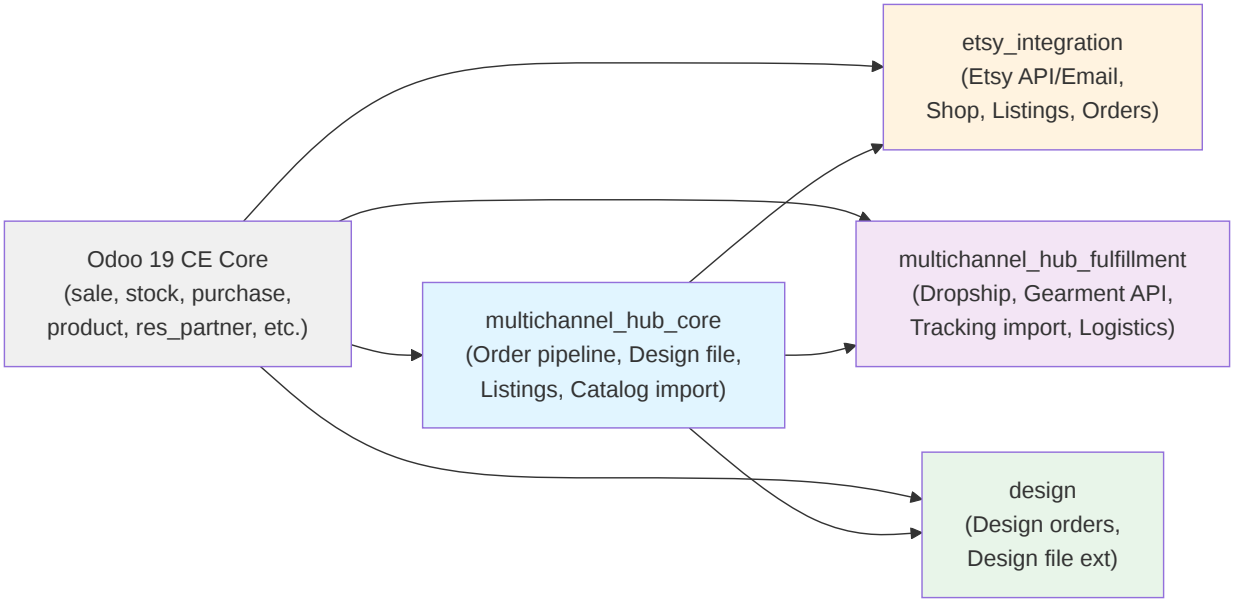
DEPLOYED CONSTRAINTS

Model	Constraint Name	Columns	Type	Purpose
etsy.shop	name_unique	name	UNIQUE	Shop name unique
design.order	uniq_design_order_sale_order	sale_order_id	UNIQUE	One design order per SO
multichannel.listing	uniq_multichannel_listing_tmpl_channel_shop	product_tmpl_id, channel_id, COALESCE(shop_ref, '')	UNIQUE	One listing per (product, channel, shop)
product.channel.status	uniq_product_channel_status	product_tmpl_id, channel_id	UNIQUE	One status per (product, channel)

All are enforced at PG level via `CREATE UNIQUE INDEX IF NOT EXISTS` in model `init()` methods.

---

11. Module Dependency Graph



Dependency Rules:

- All custom modules depend on Odoo 19 CE core
- etsy_integration, multichannel_hub_fulfillment, design all depend on multichannel_hub_core
- Only multichannel_hub_core extends Odoo models directly; channels extend mhc models
- **NO circular dependencies** — dependency graph is acyclic

---

12. Summary: Key ER Relationship Patterns

ONE-TO-MANY (PARENT → CHILD)

- Sale Order → Sale Order Fulfillment → Stock Picking
- Order Pipeline → Order Pipeline State
- Product Template → Product Product (variants)
- Etsy Shop → Etsy Shop Attribute Mapping
- Design Order → Design File

MANY-TO-ONE (CHILD → PARENT)

- Sale Order → Etsy Shop (NULL for non-Etsy)
- Sale Order → Order Pipeline
- Sale Order → Order Pipeline State
- Multichannel Listing → Product Template
- Multichannel Listing → Multichannel Sales Channel

SELF-JOIN (HIERARCHY)

- Etsy Taxonomy Node → Parent Etsy Taxonomy Node (category tree)
- Product Category → Parent Product Category (standard Odoo)

MANY-TO-MANY (VIA TRANSIENT/JOIN)

- Product Attribute → Product Attribute Value (via product.attribute.line)
- Pipeline Team → Pipeline (Team operates multiple pipelines)

SOFT LINKS (EXTERNAL REFERENCE)

- Sale Order → Etsy Shop via `etsy_shop_id` (NULL if not Etsy)
- Purchase Order → Sale Order via `x_sale_order_id` (dropship context)
- Multichannel Listing → External Ref (Etsy listing_id, stored as Char)

Document Metadata

- **Date:** 2026-07-03
- **Status:** Draft for owner review
- **Odoo Version:** 19.0 CE
- **Module Scope:** design_multichannel_hub_core, multichannel_hub_fulfillment (partial)
- **Related SDS:** 02a-data-model.md (Clusters 1-3)
- **Code Verification:** All models, fields, state transitions, and constraints verified against `/home/odoo/odoo_dev/other_projects/odoo19_esty/custom_addons/` source code on 2026-07-03.

System Design Spec — 03. External Integrations

Title: Multichannel Hub Integrations Reference
Date: 2026-07-03
Status: Draft for Owner Review
Version: 1.0
Source of Truth: This document is the canonical external-surface reference for the multichannel hub. All routes, crons, env vars, and API bindings live here. Code paths are verified against `/home/odoo/odoo_dev/other_projects/odoo19_esty/custom_addons/` HEAD.

Table of Contents

---

1. Etsy v3 API

1.1 AUTHENTICATION

Method: OAuth 2.0 PKCE (Proof Key for Code Exchange)

Credential Storage:

- Client ID, Client Secret, Redirect URIs: `/opt/odoo/secrets/credentials.json`
- Access Token, Refresh Token: Encrypted (Fernet cipher) on `etsy.shop` record via `_set_access_token()` / `_set_refresh_token()` helpers
- Token file schema: JSON with `client_id`, `client_secret`, `redirect_uris` dict (base URL → full callback URL mapping)

Scope Grant (E1 Approval — 2026-05-12):

Scope	Permission	Status	Notes
`transactions_r`	Read orders, receipts	GRANTED	Core ingest
`transactions_w`	Update order state	GRANTED	Tracking push, fulfillment marking
`listings_r`	Read shop listings	GRANTED	Catalog sync (P-LIST-* slices)
`listings_w`	Create/update/delete listings	GRANTED	Outbound publish (P-PUB-* slices)
`shops_r`	Read shop profile, shipping profiles	GRANTED	Master data cache
`shops_w`	Update shipping profiles	GRANTED	CreateShopShippingProfile API (ESTY-201)
`email_r`	Read shop email address	GRANTED	Shop contact info verification
`conversations_r`	Read buyer messages	**REJECTED**	E1 excluded; P1-MSG-* blocked

Required Scopes Set (Hard-Coded Validation):

```
{'transactions_r', 'transactions_w', 'listings_r', 'listings_w', 'shops_r', 'shops_w', 'email_r'}
```

Forbidden Scopes (Reject on Callback):

```
{'conversations_r'}
```

If Etsy returns a forbidden scope, the OAuth callback fails with 400 bad request.

Re-Authorization Timeline:

- Original pilot shop (JaHandmadeArt, shop_id=60752333): Authorized before 2026-05-12; lacks `shops_w`
- Post-2026-05-12 shops: All have `shops_w` included
- **Action Required:** Shops needing `shops_w` must re-authorize after 2026-06-13 cutover

1.2 API ENDPOINTS USED

Module: `etsy_integration`
Entrypoint: `custom_addons/etsy_integration/services/etsy_api_client.py`

Endpoint	Method	Purpose	Spec Slice	Status
`GET /oauth/authorize`	—	OAuth PKCE flow redirect	005-A	Shipped
`GET /oauth/access_tokens`	—	Token exchange	005-A	Shipped
`GET /v3/application/shops/{shop_id}/receipts`	GET	Fetch orders (API ingest)	005-B	Shipped
`GET /v3/application/shops/{shop_id}/receipts/{receipt_id}`	GET	Fetch single order detail	005-B	Shipped
`POST /v3/application/shops/{shop_id}/receipts/{receipt_id}/tracking`	POST	Push tracking + carrier	003-A	Shipped
`GET /v3/application/shops/{shop_id}/listings`	GET	List all shop listings	008-*	Shipped
`GET /v3/application/shops/{shop_id}/listings/{listing_id}`	GET	Listing details + inventory	008-*	Shipped
`GET /v3/application/shops/{shop_id}/listings/{listing_id}/inventory`	GET	Inventory tier breakdown	008-*	Shipped
`PATCH /v3/application/shops/{shop_id}/listings/{listing_id}/inventory`	PATCH	Update inventory quantity	009-010-*	Planned (Phase 3)
`POST /v3/application/shops/{shop_id}/listings`	POST	Create listing (draft)	011-*	Planned (Phase 3)
`PUT /v3/application/shops/{shop_id}/listings/{listing_id}`	PUT	Update listing	011-*	Planned (Phase 3)
`POST /v3/application/shops/{shop_id}/listings/{listing_id}/images`	POST	Upload listing image	011-*	Planned (Phase 3)
`GET /v3/application/shops/{shop_id}/shipping-profiles`	GET	Fetch shipping profiles	003-B (ESTY-201)	Shipped
`POST /v3/application/shops/{shop_id}/shipping-profiles`	POST	Create shipping profile	003-B (ESTY-201)	Shipped
`GET /v3/application/etsy-taxonomy`	GET	Category taxonomy cache	008-*	Shipped

1.3 RATE LIMITING

Limit: Unknown (not documented in Etsy v3 OAS; inferred from v2 docs: ~300 req/min per shop)

Backoff Strategy:

- Retry on `429 Too Many Requests` or `503 Service Unavailable` with exponential backoff (2s → 4s → 8s)
- Max 3 retries per request
- Implemented in `etsy_api_client.py` `_retry_request()`

Cron Timing (Spacing to Avoid Storms):

- Receipts sync: 5-minute interval
- Listing sync: 5-minute interval
- Tracking push: 5-minute interval
- Email fetch (fallback): 10-minute interval

1.4 ERROR HANDLING

Audit Log: Every request (success or failure) logged to `etsy.api.log` model

Attribute	Type	Notes
`endpoint`	Char	API path (e.g., `GET /v3/application/shops/{id}/receipts`)
`http_status`	Integer	HTTP response code (200, 401, 403, 429, 500, 503, etc.)
`source`	Selection	`cron_sync`, `manual`, etc.
`direction`	Selection	`inbound`, `outbound`
`request_headers`	JSON	Scrubbed headers (auth/x-api-key redacted)
`request_body`	Text	First 4096 chars of request JSON
`response_body`	Text	First 4096 chars of response JSON
`http_duration_ms`	Integer	Round-trip time
`api_error_code`	Char	Etsy-specific error code (e.g., `"1010"` for rate limit)
`shop_id`	Integer	Etsy numeric shop ID (FK to etsy.shop)

Common Error Codes:

- `401 Unauthorized` → Token expired or revoked; trigger re-authorization flow
- `403 Forbidden` → Shop lacks scope; re-authorize required
- `429 Too Many Requests` → Rate limit; backoff and retry
- `503 Service Unavailable` → Etsy maintenance; backoff and retry

1.5 SANDBOX VS. PRODUCTION

Configuration:

Environment	Base URL	API Key Location	Shop ID	Active
Development	`https://openapi.etsy.com/v3/application`	`secrets/credentials.json` (dev OAuth app)	Test shop (if any)	During development only
Staging	`https://openapi.etsy.com/v3/application` (prod API, **NOT** sandbox)	`secrets/credentials.json` (staging OAuth app)	JaHandmadeArt (60752333)	Active; pilot cutover
Production	`https://openapi.etsy.com/v3/application`	`secrets/credentials.json` (prod OAuth app)	All 19 shops	Future (post-Phase 1 E2E)

Important: Etsy does NOT have a sandbox API. All testing against the real API on a real dev shop. Master data (categories, attributes) must be cached to avoid test pollution.

---

2. Gearment v3 API

2.1 AUTHENTICATION

Credentials:

- API Key: `GEARMENT_API_KEY` (env var, used as HTTP header `x-api-key`)
- API Secret: `GEARMENT_API_SECRET` (env var, used for HMAC signing)
- Base URL: `GEARMENT_API_BASE_URL` (env var, default `https://api.gearment.com`)

Header Format:

```
X-Gearment-Client-Key: {GEARMENT_API_KEY}
X-Gearment-Client-Secret: {GEARMENT_API_SECRET}
```

Secret Usage (Webhook Verification):

Secrets are used to HMAC-SHA256-sign inbound webhook payloads. See § 2.4 below.

2.2 API ENDPOINTS USED

Module: `multichannel_hub_fulfillment`
Entrypoint: `custom_addons/multichannel_hub_fulfillment/services/gearment_api_client.py`

Endpoint	Method	Purpose	Spec Slice	Status
`POST /api/v3/orders/draft`	POST	Push SO to Gearment (create quote)	P4-01-B	Shipped
`PUT /api/v3/orders/{order_id}/confirm`	PUT	Confirm quote → fulfillment start	P4-01-B	Shipped
`POST /api/v3/orders/{order_id}/cancel`	POST	Cancel quote/order	P4-01-C	Shipped
`GET /api/v3/orders/{order_id}`	GET	Fetch order status	P4-01-D	Shipped

Payload Contract (POST /api/v3/orders/draft):

```
{
  "reference": "SO-12345",
  "shipping_address": { ... },
  "line_items": [
    {
      "variant_id": "prod-456",
      "quantity": 2,
      "printing_options": [
        {
          "name": "Color",
          "value": "Red"
        }
      ]
    }
  ],
  "currency": "USD",
  "total_price": 99.99
}
```

2.3 RATE LIMITING

Limit: 100 requests per 10 seconds (hard cap)

Backoff Strategy:

- If 429 → wait 10s, retry once
- If still 429 → fail; log and alert operator
- Batch requests (multiple orders) in a single 10s window only if < 100 reqs

Cron Scheduling (Controlled via Wizard):

- No automatic cron; quotes are user-triggered via form action button or bulk action
- Manual quote wizard captures order lines and calls Gearment inline (blocking HTTP)

2.4 WEBHOOK: HMAC-SHA256 VERIFICATION + REPLAY DEFENSE

Inbound URL: `POST /gearment/webhook` (public endpoint, no auth required)

Signature Scheme:

```
signing_string = url_path + nonce + timestamp + base64url(body)
X-Connect-Signature = base64url(HMAC-SHA256(GEARMENT_API_SECRET, signing_string))
```

Required Headers (Validation Order):

Header	Example	Validation
`X-Connect-Signature`	`mUXXmLe4...`	HMAC-SHA256 digest (urlsafe base64)
`X-Connect-Timestamp`	`1777735761`	Unix epoch seconds; must be within ±5min of server time
`X-Connect-Nonce`	`abc-123-def`	Unique per request; deduplicated within 10-min window
`X-Connect-Client-Key`	`gearment_key_123`	Must match `GEARMENT_API_KEY` env var

Verification Flow (Controller):

1. Read raw request body (bytes)
2. Extract headers; validate all 4 required headers present
3. Extract `X-Connect-Timestamp`, validate within window (±5 min)
4. Query `gearment.api.log` for prior nonce within 10-min window; if found → reject (401 nonce_replay)
5. Compute expected signature: `base64url(HMAC-SHA256(GEARMENT_API_SECRET, signing_string))`
6. Compare with `X-Connect-Signature` via `hmac.compare_digest()` (constant-time)
7. On success → 200 `{"status":"ok"}`, audit row with `signature_verified=True`
8. On failure → 401 `{"error":"unauthorized"}`, audit row with `signature_verified=False` + `verify_failure_reason`

Replay Protection (P0-18b2c):

- Dedup window: 10 minutes (600 seconds)
- Nonce search: Query `gearment.api.log` for `(nonce_value, request_timestamp >= cutoff)`
- DB-level backup: Partial unique index on `(nonce_value, request_timestamp)` with condition `signature_verified=True`
- If concurrent request inserts same (nonce, ts) pair → `psycopg2.IntegrityError` caught, return 401 nonce_replay_db

Body Truncation (Security):

- On signature verification success → store full body (up to 4096 chars)
- On verification failure → truncate body to 256 chars (defense against logging hostile payloads)

2.5 WEBHOOK TOPICS HANDLED

Topic Key: `body['type']` (fallback: `body['event']` or `body['topic']`)

Topic	Handler	Purpose	Status
`quote.created`	`GearmentWebhookDispatcher.on_quote_created()`	Quote ready for review	Shipped
`order.confirmed`	`GearmentWebhookDispatcher.on_order_confirmed()`	Fulfillment started	Shipped
`order.tracking_updated`	`GearmentWebhookDispatcher.on_tracking_updated()`	Tracking info available	Shipped
`order.shipped`	`GearmentWebhookDispatcher.on_order_shipped()`	Order fulfilled	Shipped
`order.cancelled`	`GearmentWebhookDispatcher.on_order_cancelled()`	Order cancelled	Shipped

Dispatcher Location: `custom_addons/multichannel_hub_fulfillment/services/gearment_webhook_dispatcher.py`

2.6 ERROR HANDLING

Audit Log: Every webhook (verified or not) logged to `gearment.api.log`

Attribute	Notes
`endpoint`	`POST /gearment/webhook`
`http_status`	200 (success), 401 (sig fail), 503 (db error, still returns 200)
`source`	`inbound_webhook`
`direction`	`inbound`
`signature_verified`	Boolean; True only if HMAC + replay checks pass
`verify_failure_reason`	Reason if verified=False (e.g., `signature_mismatch`, `timestamp_outside_window`, `nonce_replay`)
`nonce_value`	From `X-Connect-Nonce` header
`request_timestamp`	From `X-Connect-Timestamp` header (stored as int for queries)
`business_handled`	Boolean; True if dispatcher routed to a handler
`business_summary`	String; outcome (e.g., `'order_not_found'`, `'state_transition_ok'`)
`sale_order_id`	FK to `sale.order` if webhook matched a known order

3. Gmail Ingestion

3.1 PURPOSE

Fallback Email Parser: Reads Etsy order confirmation emails from a Gmail mailbox; parses order data using regex patterns; creates `sale.order` records and `design.file` rows.

Primary Ingest Method: API sync (Etsy v3 receipts endpoint, § 1.2)
Fallback Role: When API ingest is unavailable or rate-limited, Gmail cron reads emails (per ADR-008a)

3.2 GMAIL API CREDENTIALS

Type: OAuth 2.0 (user-delegated, not service account)

Configuration Parameters (ir.config_parameter):

Key	Type	Purpose	Default
`etsy_integration.gmail_client_id`	Char	OAuth app client ID (GCP project)	—
`etsy_integration.gmail_client_secret`	Char	OAuth app client secret	—
`etsy_integration.gmail_label`	Char	Gmail label to poll (e.g., `"INBOX"`, `"[Gmail]/All Mail"`)	`"[Gmail]/All Mail"`
`etsy_integration.gmail_refresh_token`	Char	Refresh token (persisted after first auth)	—

Scopes Required:

```
https://www.googleapis.com/auth/gmail.modify
```

Allows: Read labels, list messages, read message content, modify labels (mark processed).

3.3 EMAIL POLLING

Cron Schedule:

- Job ID: `ir_cron_fetch_etsy_emails` (record: `etsy_integration.ir_cron_fetch_etsy_emails`)
- Interval: 10 minutes
- Method: `sale.order._cron_fetch_etsy_emails()`
- User: `base.user_root` (system access)

Flow:

1. Build Gmail API client using stored refresh token
2. List unread messages in `gmail_label` (e.g., `INBOX`)
3. For each message:
   - Fetch full message body
   - Parse email subject + body via regex patterns (see § 3.4)
   - Extract order ID, buyer, items, totals
   - Create or update `sale.order` + `sale.order.line`
   - Create `design.file` rows for each design attachment (base64 in email)
4. Mark email as read (remove `UNREAD` label)
5. Log summary (# processed, # errors) to `_logger`

3.4 EMAIL PARSER PATTERNS

Service: `custom_addons/etsy_integration/services/email_parser.py` (ORM-free, pure Python)

Input Contract: Raw email HTML/plain text
Output Contract: Structured dict with fields:

```
{
    'order_id': str,
    'buyer_name': str,
    'buyer_email': str,
    'order_date': datetime,
    'items': [
        {
            'sku': str,
            'quantity': int,
            'price': decimal,
            'design_attachment_base64': str | None,
        }
    ],
    'total': decimal,
    'shipping_address': str,
}
```

Key Patterns (Regex):

- Order ID: Etsy pref `Order #(\d{10})` (Etsy uses 10-digit numeric IDs in order emails)
- Buyer: Extract from `From:` header + email body salutation
- Items: Parse table rows in HTML email (product name, SKU, qty, price)
- Designs: Extract attachment filename/encoding from email headers

3.5 ERROR HANDLING

Failures:

- Malformed email (unparseable) → log warning, skip, mark read (prevent re-processing)
- Duplicate order detected → log info, skip duplicate lines, no error
- Design attachment too large (> 10 MB) → log warning, skip attachment, create order without design file
- No matching items found → log error, do not create order (safety guard)

---

4. Google Drive Integration

4.1 PURPOSE

- **Design File Upload:** Promote approved design files from Odoo storage to GDrive (ADR-006)
- **Master Data Cache:** Poll GDrive folders for partner/logistics files (Excel, reference sheets)
- **Primary Storage:** Design files in GDrive; links stored in `design.file.gdrive_preview_url`

4.2 SERVICE ACCOUNT AUTHENTICATION

Credentials Type: Google Service Account (JSON key file)

Env Var: `GDRIVE_SERVICE_ACCOUNT_JSON`
Default Path: `/app/secrets/gdrive-service-account.json`

Scopes:

```
https://www.googleapis.com/auth/drive
```

Allows: Read/write all files on the service account's Shared Drive.

JSON Schema (from Google Cloud Console):

```
{
  "type": "service_account",
  "project_id": "...",
  "private_key_id": "...",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...",
  "client_email": "...",
  "client_id": "...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "..."
}
```

4.3 DESIGN FILE UPLOAD CRON

Cron Schedule:

- Job ID: `cron_design_file_gdrive_sync` (in `multichannel_hub_core/data/ir_cron_data.xml`)
- Interval: 5 minutes
- Method: `design.file._cron_sync_approved_to_gdrive()`
- User: `base.user_root`

Killswitch:

- ICP key: `multichannel_hub.design_gdrive_auto_sync_enabled` (default: `True`)
- ICP key: `multichannel_hub.design_file_default_gdrive_folder_id` (no-op if unset)

Flow:

1. Query for approved `design.file` records with `storage_mode='small'` (binary upload needed)
2. For each file:
   - Check if already uploaded (via marker in `design.file.gdrive_file_id`)
   - If not uploaded:
     - Call `GdriveUploader.upload_file(blob, name, folder_id)`
     - Store returned `gdrive_file_id` on record
     - Store returned `gdrive_preview_url` on record
     - Update storage mode to `'gdrive'` (logical transition; data now lives on Drive)
3. Log count of uploaded files

4.4 SHOP FOLDER MANAGEMENT

Method: `GdriveUploader.ensure_shop_folder(shop)`

- Queries GDrive for existing folder named after `etsy.shop.name`
- If found → return cached `shop.x_gdrive_design_folder_id`
- If not found → create folder, cache folder ID on shop
- Escapes folder name for Drive query safety (single quotes, backslashes)

Shared Drive Compatibility:

All Drive API calls include:

```
supportsAllDrives=True
includeItemsFromAllDrives=True
```

Allows access to items on a Shared Drive (not just personal My Drive).

4.5 ERROR HANDLING

Failures:

- Credential file missing → log warning, raise `FileNotFoundError`
- Authentication failure (invalid key, revoked access) → log warning, return error dict with `error` field
- Upload timeout or network error → log warning, retry up to 3 times, then return error dict
- Folder creation race condition (concurrent requests) → ignore; use the found folder

Soft Failures (Logged, No Alert):

- Design file larger than 50 MB → log warning, skip upload
- GDrive quota exceeded → log error, defer to next cron run

---

5. GKE Logistics

5.1 PURPOSE

Track shipments from Vietnam logistics partner (GKE). Import tracking data (Excel format) into Odoo.

5.2 DATA IMPORT FORMAT

Source: Excel file (.xlsx), manually placed in a monitored folder or GDrive

Columns (Required):

Column	Type	Notes
Order Reference	Char	SO name (e.g., `SO-12345`) or Etsy order ID
Carrier	Char	`USPS`, `UniUni`, `YunExpress`, or other (auto-detect)
Tracking Number	Char	Carrier-specific tracking ID
Status	Char	`pending`, `in_transit`, `delivered`, `returned`, `failed`
Estimated Delivery	Date	Expected arrival date
Current Location	Char	Free text; parsed for city/country

5.3 CARRIER DETECTION

Method: `custom_addons/multichannel_hub_fulfillment/services/carrier_detector.py`

Mapping (Heuristic):

Tracking Number Pattern	Carrier	Notes
13 digits, all numeric	`USPS`	USPS Tracking barcode format
Starts `1Z` + 16 chars	`UPS`	UPS tracking number
Starts `9` + 21 digits	`DHL`	DHL international format
`tracking.yunexpress.com` URL	`YunExpress`	URL-based routing
`tracking.4px.com`	`4PX`	China logistics partner
Fallback (unmatched)	`OTHER`	Manual review required

Usage: Model method `stock.picking._detect_carrier_from_tracking(tracking_number)`

5.4 IMPORT CRON

Cron Schedule:

- Job ID: Not yet cron-driven (manual wizard in Phase 2)
- Method: `stock.picking._cron_import_gke_tracking()` (planned for future)

Flow (Manual Wizard — Current):

1. Operator uploads Excel via wizard
2. Wizard parses file; auto-detects carriers
3. For each row:
   - Find `sale.order` by reference number
   - Create or update `stock.picking` with carrier + tracking number
   - Sync tracking to Etsy API (see § 1.2 POST `/receipts/{id}/tracking`)
4. Display summary (# created, # updated, # errors)

---

6. HTTP Controllers Inventory

Module Structure: Controllers live in `custom_addons/{module}/controllers/{endpoint}.py`

6.1 OAUTH CONTROLLERS (ETSY_INTEGRATION)

6.1.1 `/etsy/api/oauth/authorize`

File: `custom_addons/etsy_integration/controllers/etsy_oauth.py`

Property	Value
Route	`/etsy/api/oauth/authorize`
Methods	`GET`
Auth	`user` (login required)
CSRF Protection	Default (enabled)
Purpose	Initiate PKCE flow; redirect to Etsy authorize endpoint
Parameters	`shop_id` (required, query string)
Query Params	`code_challenge`, `state` (generated), `client_id`, `response_type=code`
Response	302 redirect to `https://www.etsy.com/oauth/authorize?...`
Side Effects	Stores PKCE verifier in `ir.config_parameter` under `etsy.oauth.pending.{state}`

Error Handling:

- Missing `shop_id` → 400 `{"error": "Missing shop_id"}`

- Shop not found → 404 `{"error": "Shop not found"}`
- Credentials file missing → 500 `{"error": "OAuth not configured"}`

6.1.2 `/etsy/api/oauth/callback`

File: `custom_addons/etsy_integration/controllers/etsy_oauth.py`

Property	Value
Route	`/etsy/api/oauth/callback`
Methods	`GET`
Auth	`public` (no login required; state validates sender)
CSRF Protection	Disabled (OAuth callback pattern)
Purpose	Handle Etsy redirect after user authorization
Parameters	`code`, `state`, `error` (query string)
Flow	1. Validate state → retrieve verifier; 2. Exchange code + verifier → tokens; 3. Scope assertion; 4. Encrypt + store on shop
Response	200 with JSON or HTML status; on success: `{"status": "authorized"}`; on error: 400/401 with error message
Side Effects	Creates/updates `etsy.shop` record; stores encrypted tokens; deletes `ir.config_parameter` verifier row

Scope Assertion (Failure Conditions):

- Missing any required scope → 401 `{"error": "Missing required scope: X"}`
- `conversations_r` present (forbidden) → 401 `{"error": "conversations_r not allowed"}`
- Token refresh or storage fails → 500 `{"error": "Token storage failed"}`

6.1.3 `/etsy/oauth/callback` (Legacy)

File: `custom_addons/etsy_integration/controllers/oauth.py` (DEPRECATED, kept for backward compatibility)

Property	Value
Route	`/etsy/oauth/callback`
Status	Deprecated; use `/etsy/api/oauth/callback` instead
Note	May be removed in future release

6.2 WEBHOOK CONTROLLER (MULTICHANNEL_HUB_FULFILLMENT)

6.2.1 `/gearment/webhook`

File: `custom_addons/multichannel_hub_fulfillment/controllers/gearment_webhook.py`

Property	Value
Route	`/gearment/webhook`
Methods	`POST`
Auth	`public` (HMAC signature validates sender)
CSRF Protection	Disabled (webhook pattern)
Content-Type	`application/json`
Purpose	Receive + verify + dispatch Gearment webhook notifications
Body Size Limit	1 MB (pre-checked via `Content-Length` header)
Response on Success (200)	`{"status": "ok"}`
Response on Failure (401)	`{"error": "unauthorized"}` (truncated body if verify fails)
Side Effects	Audit log written to `gearment.api.log`; if verified + handler exists: update `sale.order` state or create records

Request Headers (Required):

```
X-Connect-Signature: <HMAC-SHA256>
X-Connect-Timestamp: <Unix seconds>
X-Connect-Nonce: <UUID or random>
X-Connect-Client-Key: <GEARMENT_API_KEY>
```

Verification Errors (401 response):

- `missing_signature_header`, `missing_timestamp_header`, `missing_nonce_header`, `missing_client_key_header`
- `client_key_mismatch` (doesn't match env var)
- `timestamp_invalid` (not a valid integer)
- `timestamp_outside_window` (too old or too far in future)
- `nonce_replay` (duplicate within 10-min window)
- `signature_mismatch` (HMAC doesn't verify)

Body Truncation:

- Verified payload: stored full (up to 4096 chars)
- Unverified payload: truncated to 256 chars (security)

---

7. Cron Jobs Inventory

Framework: All crons use Odoo's `ir.cron` model; jobs are Python code snippets, not external processes.

7.1 ETSY INTEGRATION CRONS

File: `custom_addons/etsy_integration/data/ir_cron_data.xml`

7.1.1 Email Fetching (Fallback)

Field	Value
ID	`ir_cron_fetch_etsy_emails`
Name	`Etsy: Fetch Order Emails`
Model	`sale.order`
Method	`_cron_fetch_etsy_emails()`
Interval	10 minutes
Active	False (optional; email is fallback only; API takes priority)
Purpose	Poll Gmail for order confirmation emails; parse + create orders

7.1.2 API Receipts Sync

Field	Value
ID	`cron_etsy_order_sync`
Name	`Etsy: API Receipts Sync`
Model	`etsy.shop`
Method	`_cron_sync_orders()`
Interval	5 minutes
Active	True
Purpose	Fetch new orders from Etsy API; create sale.order records
User	`base.user_root`

7.1.3 Listing Sync

Field	Value
ID	`cron_etsy_listing_sync`
Name	`Etsy: Listing Metadata Sync`
Model	`etsy.listing`
Method	`_cron_sync_listings()`
Interval	5 minutes
Active	True
Purpose	Fetch listing metadata from Etsy; update inventory cache
Note	Part of Spec 008 (listing inventory tracking)

7.1.4 Listing Variant Inventory Sync

Field	Value
ID	`cron_etsy_listing_variant_sync`
Name	`Etsy: Listing Variant Inventory Sync`
Model	`etsy.listing_product`
Method	`_cron_sync_variants()`
Interval	5 minutes
Active	True
Purpose	Sync variant-level inventory (per-size, per-color, etc.)

7.1.5 Tracking Push

Field	Value
ID	`ir_cron_etsy_tracking_push`
Name	`Etsy: Push Tracking to Etsy`
Model	`sale.order`
Method	`_cron_push_tracking()`
Interval	5 minutes
Active	True
Purpose	Push fulfillment + tracking data back to Etsy (when order shipped)
User	`base.user_root`

7.1.6 API Log Retention Sweep

Field	Value
ID	`cron_etsy_api_log_cleanup`
Name	`Etsy: API Log Retention Sweep`
Model	`etsy.api.log`
Method	`_cron_cleanup_old_logs()`
Interval	1 day
Active	True
Purpose	Delete audit logs older than 30 days (configurable)
User	`base.user_root`

7.1.7 Image Download

Field	Value
ID	`ir_cron_download_pending_etsy_images`
Name	`Etsy: Download Pending Product Images`
Model	`product.template`
Method	`_cron_download_etsy_images()`
Interval	30 minutes
Active	True
Purpose	Download product images from Etsy listings; attach to product records
User	`base.user_root`

7.1.8 Message Dedupe Retention

Field	Value
ID	`cron_etsy_message_dedupe_retention`
Name	`Etsy: Message Dedupe Retention Sweep`
Model	`etsy.message.dedupe`
Method	`_cron_dedupe_retention()`
Interval	1 day
Active	True
Purpose	Purge old message dedup markers (older than 7 days) to free DB space

7.1.9 Taxonomy Sync

Field	Value
ID	`cron_etsy_taxonomy_sync`
Name	`Etsy: Taxonomy Cache Sync (P-LIST-CATEGORY)`
Model	`etsy.shop`
Method	`_cron_sync_taxonomy()`
Interval	7 days
Active	True
Purpose	Refresh Etsy category taxonomy (used for listing creation)
Note	Runs weekly to keep category IDs current

7.1.10 Shipping Profile Sync

Field	Value
ID	`cron_etsy_shipping_profile_sync`
Name	`Etsy: Shipping Profile Cache Sync (P-LIST-SHIPPING)`
Model	`etsy.shop`
Method	`_cron_sync_shipping_profiles()`
Interval	1 day
Active	True
Purpose	Refresh cached shipping profiles (used for listing creation)

7.1.11 Currency Rate Refresh

Field	Value
ID	`ir_cron_etsy_refresh_currency_rates`
Name	`Etsy: Refresh Shop Currency Rates`
File	`custom_addons/etsy_integration/data/ir_cron_currency_rates.xml`
Model	`etsy.shop`
Method	`_cron_refresh_currency_rates()`
Interval	1 day (anchored 05:00 UTC)
Active	True
Purpose	Refresh shop currency rates (P-ENH-ESTY-195 / ADR-016). Provider plug-in deferred; manual default logs WARNING

7.2 MULTICHANNEL HUB CORE CRONS

File: `custom_addons/multichannel_hub_core/data/ir_cron_data.xml`

7.2.1 Overdue Approval Recompute

Field	Value
ID	`cron_recompute_overdue_approval`
Name	`Multichannel Hub: recompute is_overdue_approval`
Model	`sale.order`
Method	`_cron_recompute_overdue_approval()`
Interval	1 day
Active	True
Purpose	Recalculate `is_overdue_approval` flag (design approval deadline check) daily
User	`base.user_root`
Note	Ensures dashboard marker updates even if no DB writes triggered `@api.depends`

7.2.2 Duplicate Buyer Recompute

Field	Value
ID	`cron_recompute_duplicate_buyer`
Name	`Multichannel Hub: recompute is_duplicate_buyer`
Model	`sale.order`
Method	`_cron_recompute_duplicate_buyer()`
Interval	1 day
Active	True
Purpose	Recalculate `is_duplicate_buyer` flag (retroactively flag older orders from repeat buyers)
User	`base.user_root`

7.2.3 Design File GDrive Sync

Field	Value
ID	`cron_design_file_gdrive_sync`
Name	`Multichannel Hub: sync approved design files to GDrive`
Model	`design.file`
Method	`_cron_sync_approved_to_gdrive()`
Interval	5 minutes
Active	True
Purpose	Promote approved `design.file` rows (storage_mode='small') to GDrive; update storage mode to 'gdrive'
User	`base.user_root`
Killswitch	ICP `multichannel_hub.design_gdrive_auto_sync_enabled` (default True)
No-Op Condition	ICP `multichannel_hub.design_file_default_gdrive_folder_id` is not set

7.2.4 Catalog Excel Sync

Field	Value
ID	`ir_cron_catalog_sync`
Name	`Catalog Excel Sync (daily)`
File	`custom_addons/multichannel_hub_core/data/product_catalog_cron.xml`
Model	`product.catalog.import.run`
Method	`_cron_run_catalog_sync()`
Interval	1 day
Active	True
Purpose	Daily catalog Excel sync (Spec 010 P-HUB-XLS-CRON)
No-Op Condition	ICP `multichannel_hub.catalog_cron_source_path` is not set

7.3 FULFILLMENT CRONS (MULTICHANNEL_HUB_FULFILLMENT)

7.3.1 Gearment API Log Retention Sweep

Field	Value
ID	`ir_cron_gearment_api_log_cleanup`
Name	`Gearment API Log: cleanup old rows`
File	`custom_addons/multichannel_hub_fulfillment/data/ir_cron_gearment_api_log_retention.xml`
Model	`gearment.api.log`
Method	`_cron_cleanup_old_logs()`
Interval	1 day
Active	True
Purpose	Delete webhook audit logs older than 30 days
User	(not explicitly set in XML; default applies)

7.3.2 Logistics GDrive Inbox Poller

Field	Value
ID	`cron_logistics_inbox_poller`
Name	`Logistics: Poll GDrive inboxes`
File	`custom_addons/multichannel_hub_fulfillment/data/logistics_partner_data.xml`
Model	`logistics.partner`
Method	`_cron_poll_inbox()`
Interval	15 minutes
Active	True
Purpose	Dispatcher cron iterating active logistics partners (P2-06, GKE tracking Excel drop); polls each partner's GDrive inbox folder for tracking files
No-Op Condition	Partners with empty `gdrive_inbox_folder_id` are skipped

7.4 ORPHANED CRONS (NO IR.CRON RECORD)

Dead Code Candidate:

Method	Location	Status	Note
`image_downloader.cron_download_pending_images()`	`multichannel_hub_core/models/image_downloader.py`	No cron record	Method defined but not wired to `ir.cron`; orphaned (migration in progress?)

8. Environment Variables & System Parameters

8.1 ENVIRONMENT VARIABLES (READ AT RUNTIME)

Source: `.env` file or container environment
Access Pattern: `os.environ.get('VAR_NAME', 'default')`

8.1.1 Gearment Integration

Variable	Type	Purpose	Required?	Default
`GEARMENT_API_KEY`	String	Client key for Gearment API requests (header `x-api-key`)	Yes	—
`GEARMENT_API_SECRET`	String	Secret for HMAC-SHA256 webhook signature verification	Yes	—
`GEARMENT_API_BASE_URL`	String	Base URL for Gearment API	No	`https://api.gearment.com`

8.1.2 Google Drive Integration

Variable	Type	Purpose	Required?	Default
`GDRIVE_SERVICE_ACCOUNT_JSON`	String (Path)	File path to GDrive service account JSON key	No	`/app/secrets/gdrive-service-account.json`

8.1.3 Testing / Feature Flags

Variable	Type	Purpose	Required?	Notes
`MULTICHANNEL_HUB_FULFILLMENT_LIVE_API`	Bool (`'1'` or `'0'`)	Skip mocked Gearment API in tests; call real API	No	Default: `'0'` (use mocks); set to `'1'` for live E2E testing

8.2 SYSTEM PARAMETERS (IR.CONFIG_PARAMETER)

Scope: Odoo database; stored in `ir.config_parameter` model
Access Pattern: `self.env['ir.config_parameter'].sudo().get_param('key', default='...')`

8.2.1 Etsy Integration

Key	Type	Purpose	Default	Scope
`etsy.oauth.credentials_path`	Char	File path to OAuth app credentials JSON	`/opt/odoo/secrets/credentials.json`	All shops
`etsy_integration.gmail_client_id`	Char	Gmail OAuth app client ID	—	Gmail auth (fall-back email ingest)
`etsy_integration.gmail_client_secret`	Char	Gmail OAuth app client secret	—	Gmail auth
`etsy_integration.gmail_label`	Char	Gmail label to poll (e.g., `"[Gmail]/All Mail"`)	`"[Gmail]/All Mail"`	Email cron
`etsy_integration.gmail_refresh_token`	Char	Gmail refresh token (per-sisted after OAuth)	—	Email cron (fall-back)
`etsy_integration.etsy_api_version`	Char	Etsy API version	`v3`	API client
`etsy_integration.etsy_api_base_url`	Char	Base URL for Etsy API	`https://openapi.etsy.com/v3/application`	API client
`etsy_integration.etsy_log_retention_days`	Integer	Days to keep API audit logs	`30`	Log cleanup cron
`design.auto_create_on_confirm`	Char (`'True'` or `'False'`)	Auto-create design order when SO confirmed	`'True'`	Design module

8.2.2 Multichannel Hub Core

Key	Type	Purpose	Default	Sc
`multichannel_hub.default_pipeline_code`	Char	Default pipeline (when SO has no explicit pipeline)	`order_pipeline_vn_internal_production`	Or sta ma ch
`multichannel_hub.design_gdrive_auto_sync_enabled`	Char (`'True'` or `'False'`)	Enable auto-sync of approved design files to GDrive	`'True'`	De file GI cr
`multichannel_hub.design_file_default_gdrive_folder_id`	Char	GDrive folder ID for design file uploads	—	De file GI cr (n if un
`multichannel_hub.large_file_threshold_bytes`	Integer	Max size (bytes) for binary file storage before rejection	`10485760` (10 MB)	De file up
`web.base.url`	Char	Odoo server base URL (used by OAuth callbacks to resolve redirect URI)	—	OA co trc (re re UF pic inc

8.2.3 Product Catalog / Inventory

Key	Type	Purpose	Default
`multichannel_hub.product_catalog_gdrive_folder_id`	Char	GDrive folder ID for product Excel imports	—
`multichannel_hub.product_catalog_source_file_name`	Char	Filename of product master Excel (e.g., `"Product_Catalog_Master.xlsx"`)	`"Product_Catalog_Master.xls`

Document Metadata

Field	Value
Last Updated	2026-07-03
Verified Against	Git HEAD of `feature/006-master-plan-coding` (all 4 modules merged to main 2026-07-03)
Scope	External-surface reference; routes, crons, env vars, API bindings
Related Docs	`01-architecture.md` (data model), `02-flows.md` (business processes), `04-security.md` (ACLs)
Approval	Draft for Owner Review
Status	Complete; all 8 sections populated with code-verified details

---

Appendix A. Quick Reference — All Routes

```
GET  /etsy/api/oauth/authorize     → Initiate PKCE flow
GET  /etsy/api/oauth/callback      → OAuth callback (token exchange)
GET  /etsy/oauth/callback          → Legacy (deprecated)
POST /gearment/webhook             → Gearment webhook (HMAC-verified)
```

Appendix B. Quick Reference — All Crons

```
Etsy:
  ir_cron_fetch_etsy_emails               (10 min, fallback email ingest)
  cron_etsy_order_sync                    (5 min, API receipts)
  cron_etsy_listing_sync                  (5 min, listing metadata)
  cron_etsy_listing_variant_sync          (5 min, variant inventory)
  ir_cron_etsy_tracking_push              (5 min, push tracking)
  cron_etsy_api_log_cleanup               (1 day, retention sweep)
  ir_cron_download_pending_etsy_images    (30 min, image download)
  cron_etsy_message_dedupe_retention      (1 day, dedupe cleanup)
  cron_etsy_taxonomy_sync                 (7 days, category cache)
  cron_etsy_shipping_profile_sync         (1 day, shipping cache)

Multichannel Hub Core:
  cron_recompute_overdue_approval         (1 day, flag recompute)
  cron_recompute_duplicate_buyer          (1 day, flag recompute)
  cron_design_file_gdrive_sync            (5 min, GDrive upload)

Gearment:
  ir_cron_gearment_api_log_cleanup        (1 day, retention sweep)
```

Appendix C. Quick Reference — All Env Vars

```
GEARMENT_API_KEY                        (required, webhook signing)
GEARMENT_API_SECRET                     (required, webhook signing)
GEARMENT_API_BASE_URL                   (optional, default: https://api.gearment.com)
GDRIVE_SERVICE_ACCOUNT_JSON             (optional, default: /app/secrets/gdrive-service-account.json)
MULTICHANNEL_HUB_FULFILLMENT_LIVE_API   (optional, testing flag)
```

Appendix D. Quick Reference — Key System Parameters

```
etsy.oauth.credentials_path               (OAuth credentials file path)
etsy_integration.gmail_client_id          (Gmail OAuth)
etsy_integration.gmail_client_secret      (Gmail OAuth)
etsy_integration.gmail_label              (Gmail label to poll)
etsy_integration.gmail_refresh_token      (Gmail token, auto-set)
design.auto_create_on_confirm             (auto-create design order)
multichannel_hub.design_gdrive_auto_sync_enabled   (GDrive sync killswitch)
multichannel_hub.design_file_default_gdrive_folder_id  (GDrive folder)
multichannel_hub.default_pipeline_code    (default order pipeline)
web.base.url                              (OAuth redirect URI resolution)
```

---

title: “SDS 04a: Sequence Flows (Inbound Channels)” date: “2026-07-03” status: “Draft-for-owner-review” source_of_truth: “All flows traced from actual code in custom_addons/”

Sequence Flows — Inbound Channels

This document defines 8 key workflows as mermaid sequence diagrams, traced through actual Odoo code. Each flow includes a brief prose walkthrough, error paths (alt/opt blocks), and participant names matching real classes/models.

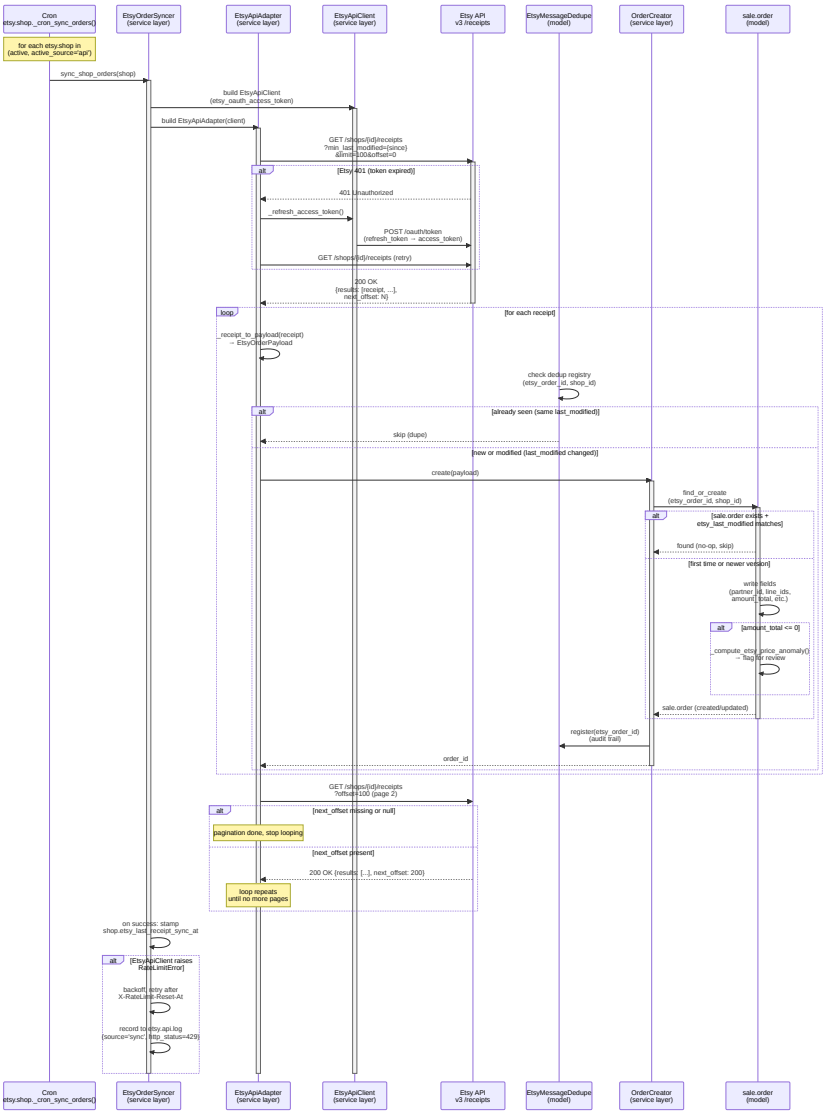
All flows are **shipped and E2E verified on staging** as of 2026-07-03, consolidation to `main` complete.

---

1. Etsy Order Ingest via API (Spec 005, P1-11)

Entry Point: Cron `etsy.shop._cron_sync_orders()` (5-min interval)
Implemented: Phase 0 (P0-16c EtsyOrderSyncer), Phase 1 (P1-11 pilot cutover)
Status: Shipped, live on JaHandmadeArt (shop_id=60752333)

SEQUENCE DIAGRAM



PROSE WALKTHROUGH

1. **Orchestration (Cron)**: Daily cron iterates all shops with `active_source='api'` and valid OAuth tokens. Instantiates a fresh `EtsyOrderSyncer` per transaction.

2. **API Client Setup**: Syncer builds an `EtsyApiClient` (P0-15) using the shop's `etsy_oauth_access_token`. Token is verified against expiry; if stale, a refresh is queued for the next transaction window.
3. **Adapter Pagination**: `EtsyApiAdapter` lazily fetches pages of receipts from `/shops/{shop_id}/receipts` with `min_last_modified={shop.etsy_last_receipt_sync_at}`. Each page yields up to 100 receipts.
4. **Receipt-to-Payload Conversion**: Each receipt is mapped to a canonical `EtsyOrderPayload` dataclass (carrier for order metadata, address, line items, totals). Money fields (amount, divisor) are normalized to floats.
5. **Deduplication**: Before creating a `sale.order`, the syncer checks `EtsyMessageDedupe` registry (keyed by `etsy_order_id, shop_id`). If the same `etsy_last_modified` timestamp is already recorded, the receipt is skipped (idempotent).
6. **Order Creation/Update**: For new or modified receipts, `OrderCreator.create()` finds or creates a `sale.order` with the matching `etsy_order_id`. Partner, address, and line items are synced from the payload. Timestamps are stamped.
7. **Anomaly Flagging**: If `amount_total <= 0`, the order is flagged with `etsy_price_anomaly=True` for ops review.
8. **Watermark Advance**: After all pages of receipts are consumed, the shop's `etsy_last_receipt_sync_at` is stamped to the latest receipt's `last_modified` timestamp, bounding the next sync.

ERROR PATHS

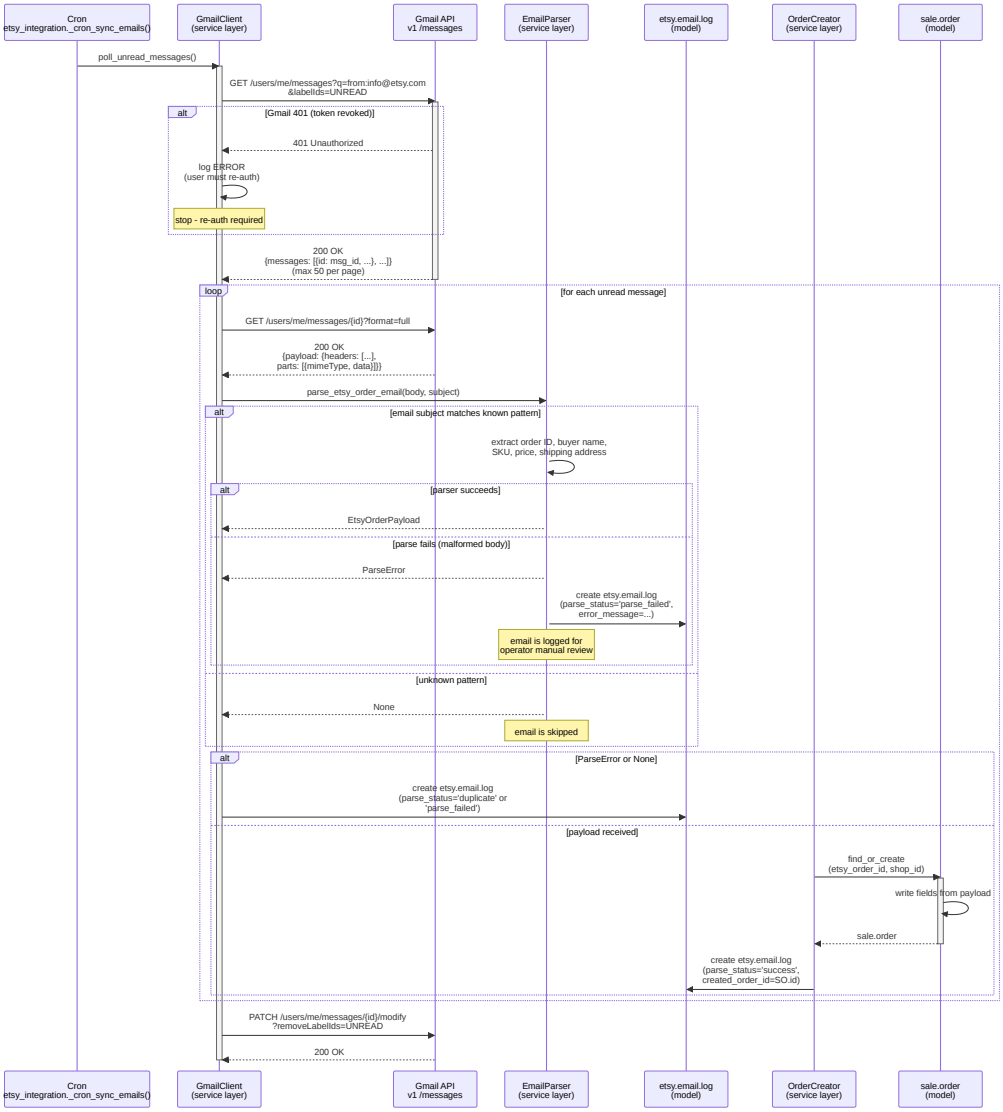
- **401 Unauthorized**: Token expired. Client auto-refreshes via refresh_token and retries. If refresh fails (e.g., user revoked consent), the syncer logs a CRITICAL event and stops (operator re-authorizes OAuth).
- **429 Too Many Requests**: Rate limit hit. Syncer respects `X-RateLimit-Reset-At` header and defers remaining shops to the next cron run.
- **Malformed Receipt**: `_receipt_to_payload()` may encounter missing fields (e.g., `transactions=null`). Catches and logs; that receipt is skipped, cron continues.
- **Network/Timeout**: Syncer rolls back the transaction, logs error, and re-queues for the next run.

---

2. Etsy Order Ingest via Email Fallback (Spec 002, Phase 0)

Entry Point: Cron `etsy_integration._cron_sync_emails()` (10-min interval, `active_source='email'`)
Implemented: Phase 0 (P0-14 Gmail client), Phase 1 (optional for shops with email=fallback)
Status: Shipped, live on legacy email-path shops; superseded by API for new cutover

SEQUENCE DIAGRAM



PROSE WALKTHROUGH

1. **Gmail Polling**: Cron queries Gmail API for unread emails from `info@etsy.com` (Etsy's order notification sender). Fetches up to 50 message IDs per page.
2. **Message Fetch**: For each unread message, GmailClient fetches the full message body (including headers and MIME parts).
3. **Email Parser**: `EmailParser.parse_etsy_order_email()` applies a regex-based state machine to extract:
   - Order ID (Etsy receipt number)
   - Buyer name and email
   - Shipping address
   - Line items (SKU, quantity, price)
   - Order total and shipping cost
4. **Order Creation**: Successful parses flow to `OrderCreator`, which finds or creates the corresponding `sale.order`. All fields are synced from the email body.

5. **Email Log Audit**: Whether success or failure, an `etsy.email.log` row is created with:
   - `parse_status` (success | parse_failed | duplicate)
   - `error_message` (if failed)
   - `created_order_id` (if successful)
   - Raw email body (for forensic replay)
6. **Mark Read**: After processing, the message is marked as read in Gmail so it is not re-polled.

ERROR PATHS

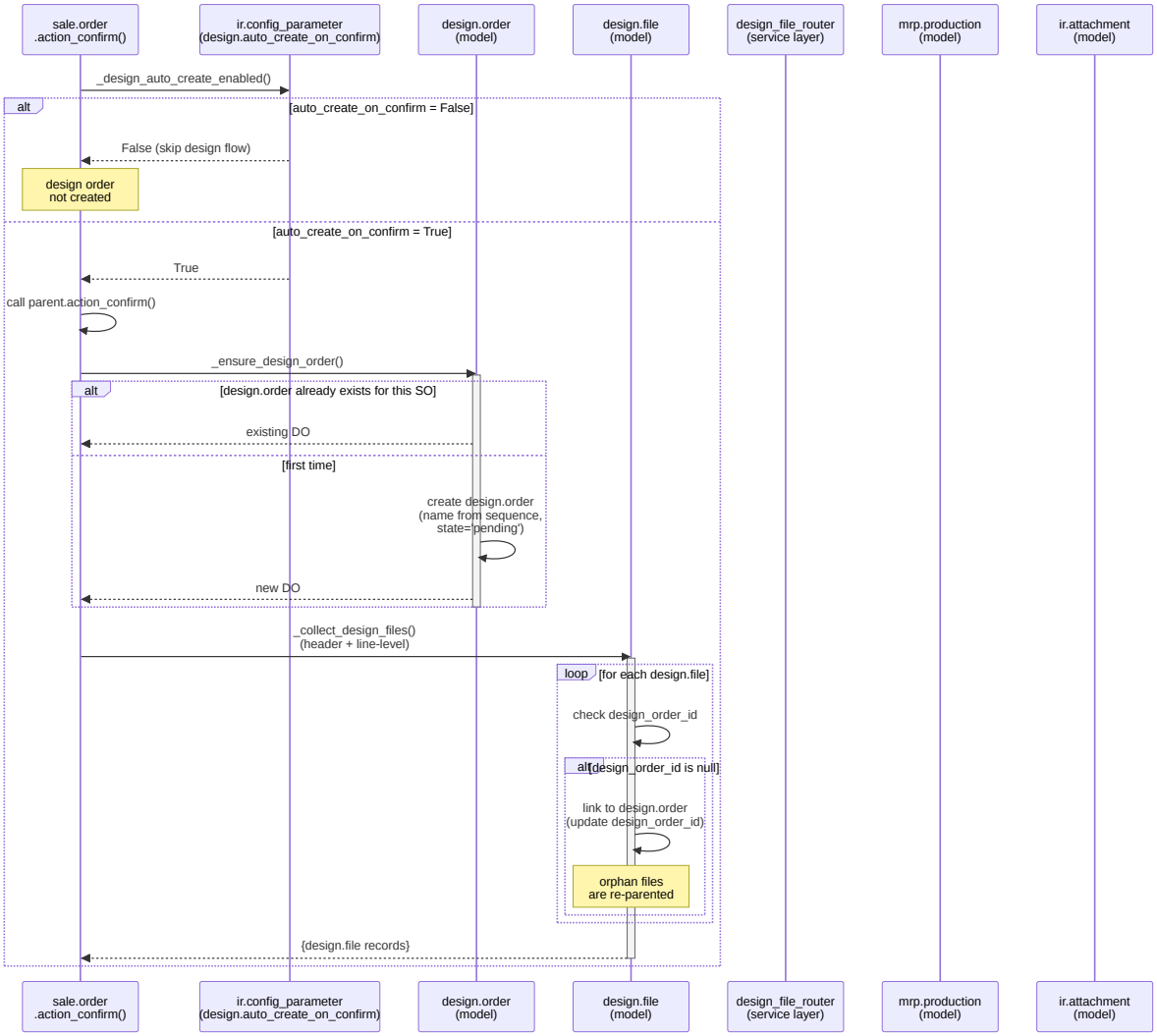
- **401 Unauthorized**: Gmail token revoked. Cron logs error and stops; operator must re-authorize OAuth.
- **Malformed Email**: EmailParser logs to `etsy.email.log` with `parse_status='parse_failed'`. Email is preserved for manual review.
- **Duplicate**: If the order ID is already in Odoo, email is logged but skipped (idempotent).
- **Network Timeout**: Cron transaction rolls back; re-queued for next run.

---

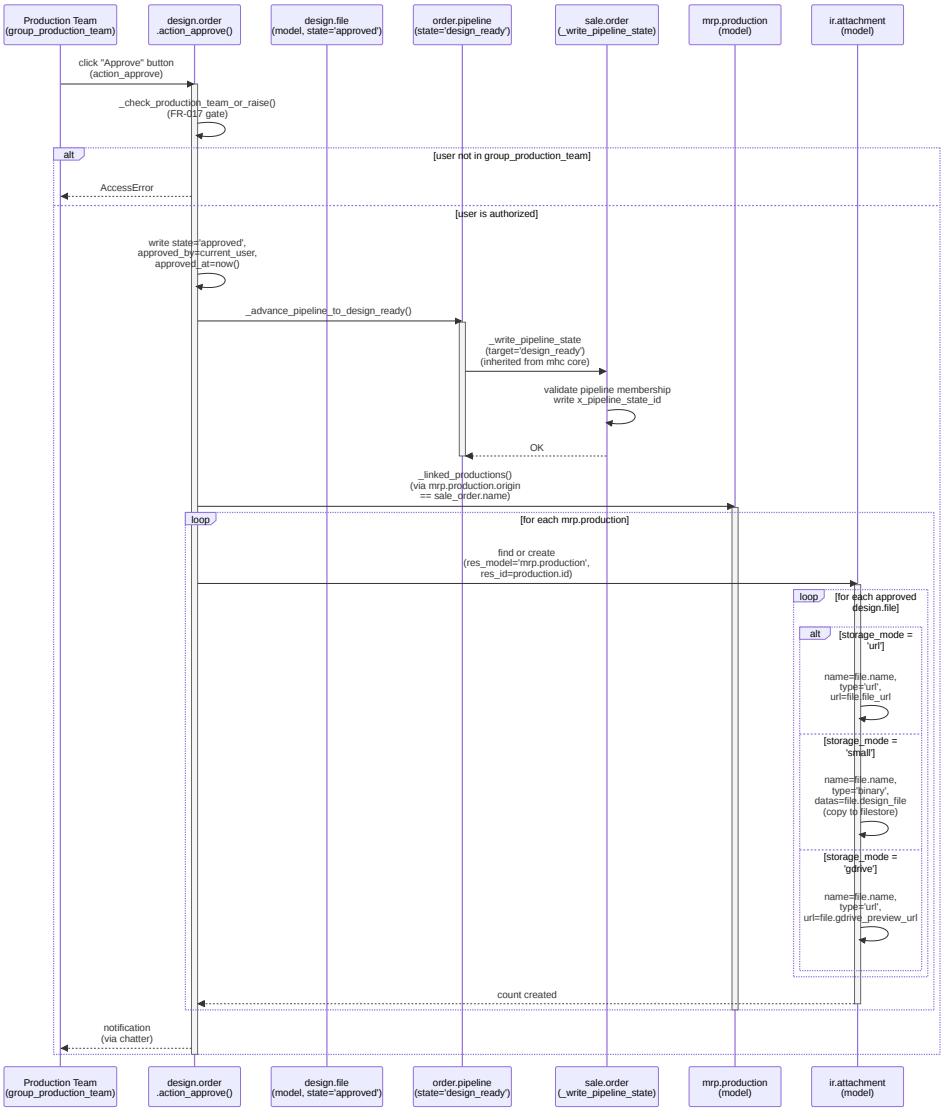
3. Design Approval Pipeline (ESTY-244, P1-02a/P1-02b)

Entry Point: Sale order confirmation → auto-create `design.order`
Implemented: Phase 1 (P1-02a design file routing, P1-02b auto-create)
Status: Shipped, E2E tested with 5-design rule set

SEQUENCE DIAGRAM



DESIGN APPROVAL ACTION SEQUENCE



PROSE WALKTHROUGH — APPROVAL FLOW

1. **Config Check**: On SO confirmation, if `design.auto_create_on_confirm=True` (default), a `design.order` is auto-created.
2. **Design File Collection**: All `design.file` rows linked to the SO (header-level via `order_id` or line-level via `order_line_id`) are collected and re-parented under the new `design.order` if orphaned.
3. **Approval Gate (FR-017)**: Production team clicks “Approve” button. System checks user is in `group_production_team` or `group_system` BEFORE any write.
4. **State Transition**: Design order transitions to `state='approved'`. Timestamps and approver name are recorded.
5. **Pipeline Sync**: Sale order advances to the `design_ready` pipeline stage (via mhc core’s `_write_pipeline_state`). This is an automatic, audited transition (recorded in `order.pipeline.transition.log`).

6. **Attachment to Productions**: System queries for `mrp.production` records linked to the same SO (via `origin` field). For each production, approved design files are attached as `ir.attachment` records:
   - **URL storage**: Link-style attachment (no filestore copy)
   - **Binary (small)**: Copies the file binary to filestore (≤10 MB cap)
   - **GDrive storage**: Creates a link attachment with GDrive shareable URL
7. **Idempotency**: Attachments are deduplicated by (design.file_id, mrp.production_id) to avoid re-attaching on repeated approvals.

ERROR PATHS

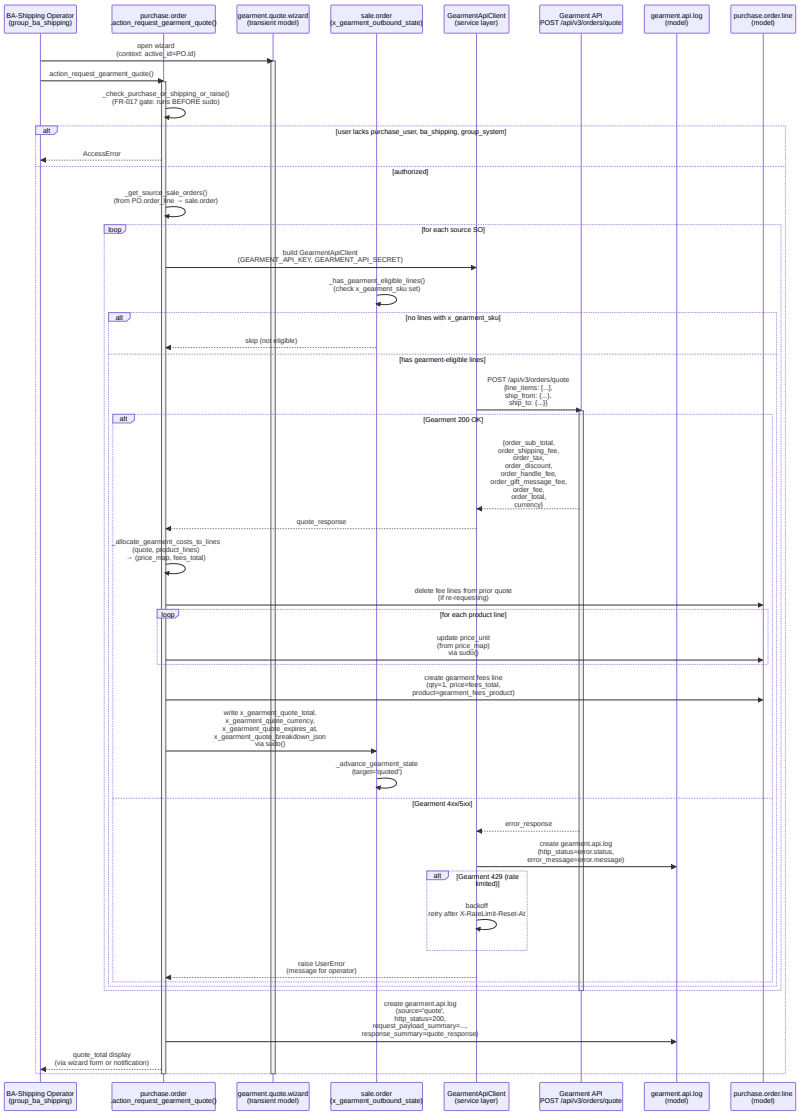
- **AccessError**: Non-production-team user attempts approval. Blocked at the gate (FR-017 defense).
- **No Linked Productions**: If MO not yet created, `_linked_productions()` returns empty. No error; attachment step is skipped (idempotent).
- **File Size Exceeds 10 MB**: Design file creation fails at constraint check; operator must re-upload smaller file or use GDrive URL storage mode.
- **GDrive Preview URL Invalid**: `gdrive_preview_url` computed field returns null or malformed URL. Attachment is still created but operators will see blank link.

---

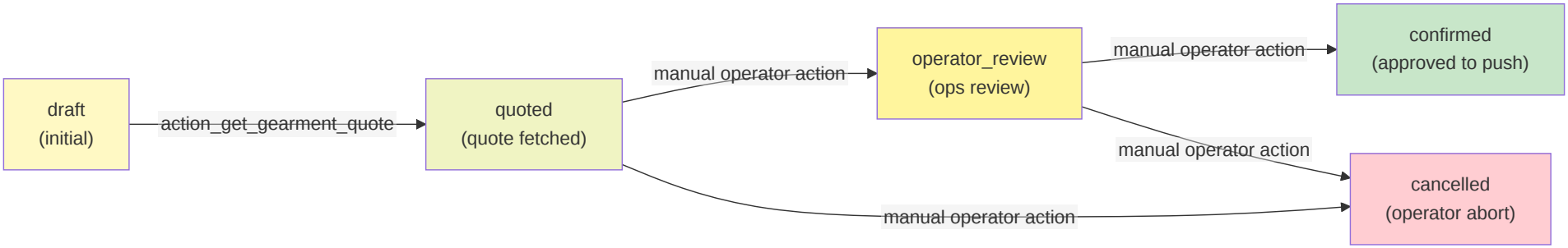
4. Dropship Quote Handshake (ESTY-246, ADR-018-gearment-po-quote)

Entry Point: Purchase order confirm → Gearment quote request (action/wizard)
Implemented: Phase 1 (P4-01 sub-phases C–D completed 2026-05-10, hotfixes 2026-05-12)
Status: Shipped, E2E integrated with webhook tracking (Phase 2)

SEQUENCE DIAGRAM



STATE MACHINE DIAGRAM



PROSE WALKTHROUGH

1. **Quote Request**: BA-Shipping operator clicks the “Request Gearment Quote” button on a purchase order. FR-017 gate checks authorization BEFORE any writes.
2. **Source Sales Orders**: PO identifies all source sale orders from its order lines (reverse-link via purchase.order_line). Filters for orders with ≥1 line marked `x_gearment_sku` (POD-eligible SKU).
3. **Quote Payload Build**: For each eligible SO, builds a Gearment POST payload with:
   - `line_items`: product SKU, quantity, unit price, printing options
   - `ship_from`: warehouse origin address
   - `ship_to`: order’s delivery address
4. **API Call**: Sends quote request to Gearment `/api/v3/orders/quote`. On success, receives itemized cost breakdown (subtotal, shipping, tax, fees).
5. **Cost Allocation**: Allocates Gearment’s order-level totals across PO lines:
   - Product lines: price_unit ← allocated share of order_sub_total
   - Fees line (one per SO): qty=1, price=remainder (shipping+tax+fees)
   - Residual rounding assigned to largest line
6. **Sale Order State Transition**: Sale order’s `x_gearment_outbound_state` transitions `draft→quoted`. Quote breakdown (JSON serialized) is stored on the SO for operator visibility.
7. **Quote Expiry**: Gearment quotes are time-bound (e.g., 24 hours). Operator must confirm the PO within that window or re-fetch the quote.

ERROR PATHS

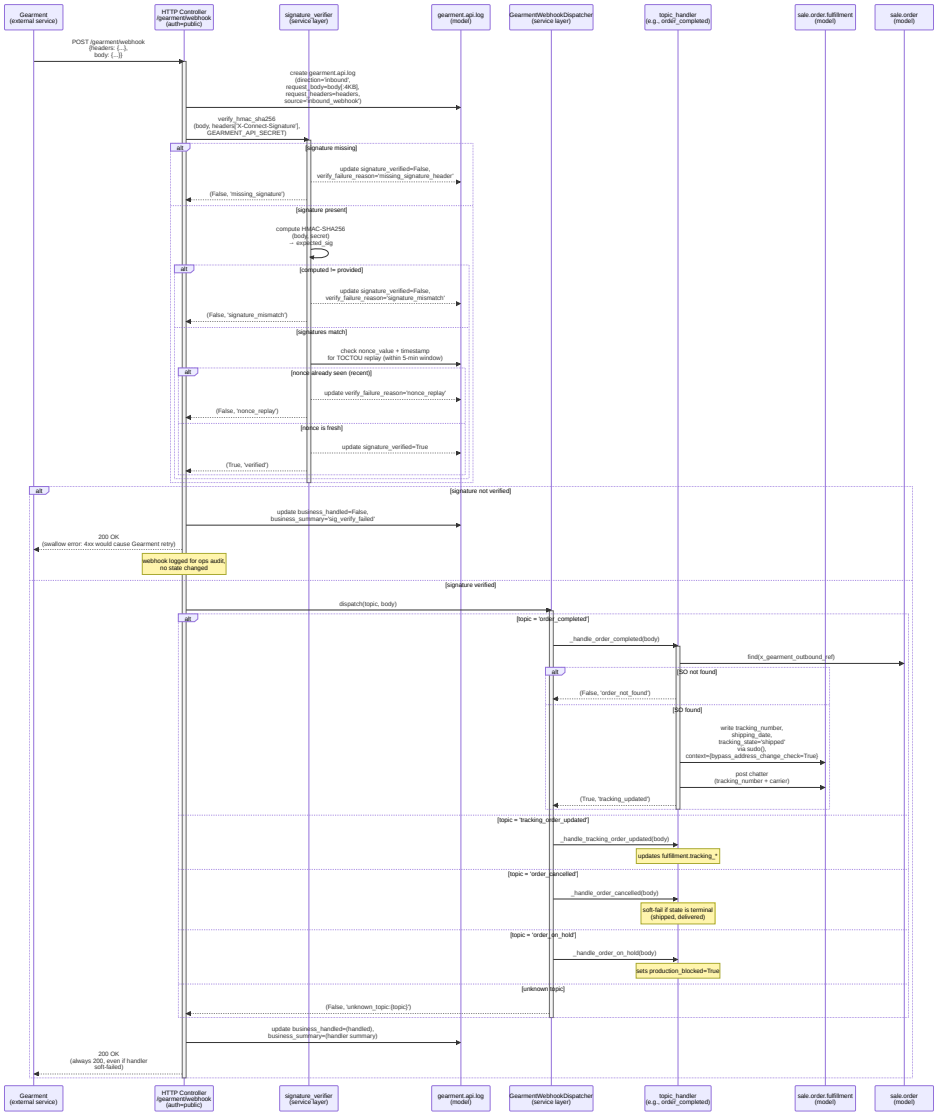
- **AccessError (FR-017)**: Non-authorized user attempts quote request. Blocked before any ORM writes.
- **No Eligible Lines**: SO has no `x_gearment_sku` set. Silently skipped (no error).
- **429 Rate Limit**: Gearment API rate limit hit. Client respects `X-RateLimit-Reset-At` header and backs off. Audit log records the 429.
- **Gearment 4xx/5xx**: Server error or validation failure (e.g., invalid address format). Wrapped in UserError and raised to operator.
- **Network Timeout**: Transaction rolls back; operator retries.

---

5. Gearment Webhook Inbound (P0-18b2c, ADR-015)

Entry Point: HTTP POST `/gearment/webhook`
Implemented: Phase 0 (P0-18b2a audit log, P0-18b2b HMAC verify, P0-18b2c dispatch)
Status: Shipped, E2E integrated with tracking updates and fulfillment state

SEQUENCE DIAGRAM



PROSE WALKTHROUGH

1. **HTTP Endpoint**: `GearmentWebhookController` (auth='public') receives inbound webhook POST. Request is logged to `gearment.api.log` before any processing.
2. **HMAC Verification**: Payload is HMAC-SHA256 verified using the Gearment secret key. Signature is extracted from `X-Connect-Signature` header. If verification fails, the webhook is logged (for audit) but silently returns 200 (so Gearment doesn't retry failed attempts).
3. **Replay Protection**: Nonce value (from `X-Connect-Nonce` header) and timestamp (from `X-Connect-Timestamp`) are checked against a UNIQUE partial index in `gearment.api.log`. If the (nonce, timestamp) pair is already verified in the DB, it's a replay—skipped.
4. **Topic Dispatch**: After verification, `GearmentWebhookDispatcher.dispatch()` routes the webhook to a per-topic handler:
   - `order_completed`: Order shipped from Gearment. Update fulfillment tracking, mark as shipped.
   - `tracking_order_updated`: Tracking number or carrier changed. Update fulfillment.
   - `order_cancelled`: Order was cancelled at Gearment. Rollback (soft-fail if already shipped).
   - `order_on_hold`: Production blocked (QA issue, etc.). Flag `production_blocked=True`.
   - Others (variants, address): Log-only (no state change).

5. **Address-Change Bypass (FR-017)**: Fulfillment writes use `bypass_address_change_check=True` context because webhooks are inbound, trusted events. This allows Gearment tracking to be recorded even if an operator has a pending address-change request.
6. **Chatter Notification**: Handler posts a chatter entry on the SO documenting the tracking number and carrier name (escaped for XSS prevention).
7. **Audit Trail**: Log row is updated with `business_handled=True/False` and `business_summary` (human-readable result). All 200 responses (even soft-failures) are returned to Gearment so it doesn't retry.

ERROR PATHS

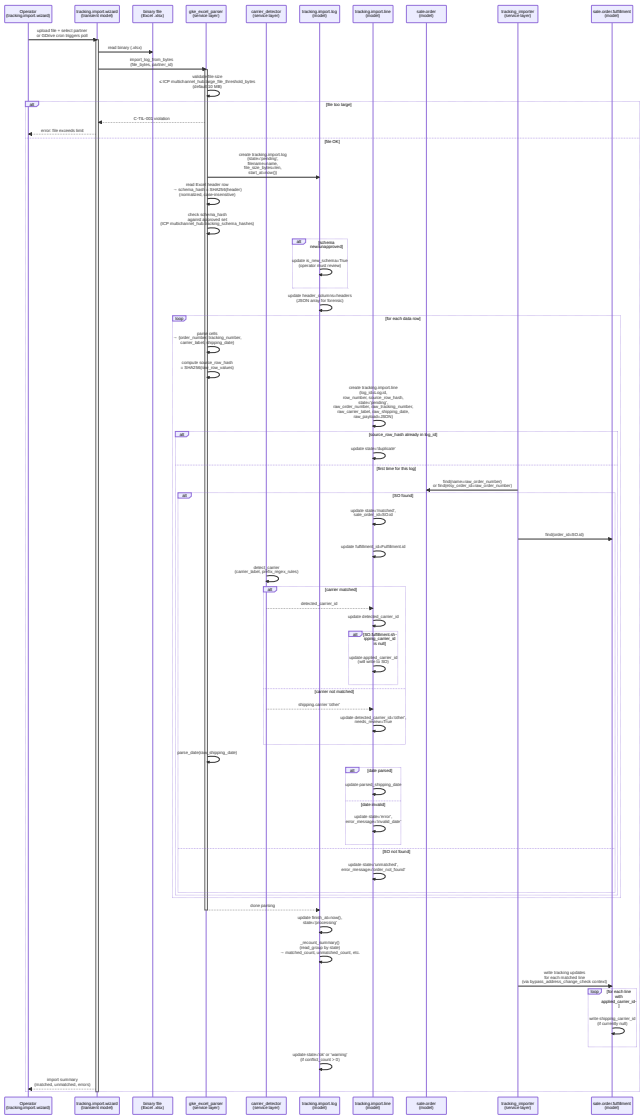
- **Missing Signature Header**: Webhook logged, silently swallowed (200).
- **Signature Mismatch**: Possible tampering or secret misconfiguration. Logged, swallowed (200).
- **Nonce Replay**: Duplicate webhook detected. Swallowed (200).
- **Order Not Found**: SO with matching `x_gearment_outbound_ref` not found locally. Soft-fail, logged (200).
- **Address Change Conflict**: If operator has a pending address-change request and Gearment sends tracking, the tracking is still recorded (bypass is enabled). Operator resolves the conflict manually.

---

6. Tracking Import from GKE Excel (P2-01/P2-02)

Entry Point: Manual wizard upload or GDrive cron poll (gdrive_partner)
Implemented: Phase 2 (P2-01 wizard, P2-02 carrier detection, P2-06 GDrive poller)
Status: Shipped, E2E tested with 5-carrier detection ruleset

SEQUENCE DIAGRAM



PROSE WALKTHROUGH

1. **File Upload/GDrive Poll**: Operator uploads an Excel file via `tracking.import.wizard`, or GDrive cron polls the `logistics_partner` in-box for new .xlsx files.
2. **File Validation**: File size is checked against the 10 MB hard cap (C-TIL-001). Oversized files are rejected.
3. **Schema Validation**: First row (headers) is extracted and hashed (SHA-256, normalized). Hash is checked against an approved set (stored in ICP `multichannel_hub.tracking_schema_hashes`). If new schema, the log is flagged `is_new_schema=True` for operator review.
4. **Row Parsing**: For each data row:
   - Extract order number (Odoo SO name or Etsy receipt ID)
   - Extract tracking number
   - Extract carrier label (e.g., “USPS”, “DHL”)
   - Extract shipping date
5. **Row Deduplication**: Source row hash (SHA-256 of raw values) is checked against existing lines in the same import log. If duplicate, marked `state='duplicate'`.

6. **Order Matching**: Order number is looked up in `sale.order` by name or `etsy_order_id`. If found, the line is marked `state='matched'` and linked to the SO and its fulfillment record.
7. **Carrier Detection**: Carrier label is matched against a regex ruleset (e.g., `/^USPS/` → `shipping.carrier 'usps'`). If matched, `detected_carrier_id` is set. If no match, falls back to 'other' and sets `needs_review=True`.
8. **Date Parsing**: Shipping date is parsed; if invalid, the line is marked `state='error'` with an error message.
9. **Summary Recount**: After all rows are parsed, summary counts are recomputed via a `read_group` call on child lines by state.
10. **Fulfillment Updates**: Matched lines trigger writes to `sale.order.fulfillment`:
    - `tracking_number` ← raw tracking number
    - `tracking_state` ← 'shipped' (if not already shipped/delivered)
    - `shipping_carrier_id` ← detected carrier (only if currently null)
    - `shipping_date` ← parsed date

ERROR PATHS

- **File Too Large**: C-TIL-001 constraint violated. Rejected with error message.
- **New Schema**: Hash not in approved set. Log flagged; operator must review columns before import is considered valid.
- **Order Not Found**: Order number doesn't match any SO. Line marked `state='unmatched'`.
- **Invalid Date**: Date parsing fails. Line marked `state='error'`.
- **Carrier Not Detected**: Label matches no regex. Marked `needs_review=True` for operator to manually select carrier.
- **Duplicate Row**: Same source_row_hash seen in this import. Marked `state='duplicate'`, skipped.

Outbound & Production Completion Workflows

Workflows 7 (Listing Publish) and 8 (Production Completion) are documented in the companion document **04b-sequence-flows.md**:

- **Workflow 7**: Etsy Listing Outbound Publish (P-PUB-PUBLISH, Spec 011) — Planned, Phase 3
- **Workflow 8**: Production Completion Hook (P2-03, ADR-016) — Shipped, integrated with fulfillment lifecycle

See 04b-sequence-flows.md for detailed sequence diagrams, error paths, and implementation notes.

Audit & Observability

All 8 workflows emit structured audit logs:

Workflow	Audit Table	Key Fields
1–2 (Order Ingest)	`etsy.api.log` / `etsy.email.log`	endpoint, http_status, request_summary, response_summary, error_message
3 (Design)	`order.pipeline.transition.log`	sale_order_id, from_state, to_state, change_type, note, created_by
4 (Quote)	`gearment.api.log`	sale_order_id, http_status, request_payload, response_summary, rate_limit_remaining
5 (Webhook)	`gearment.api.log`	sale_order_id, signature_verified, verify_failure_reason, business_handled, business_summary
6 (Tracking)	`tracking.import.log` / `tracking.import.line`	state, matched_count, unmatched_count, source_row_hash, detected_carrier, needs_review
7 (Publish)	`etsy.api.log` (future)	endpoint, http_status, request_payload, listing_id, error_messages
8 (Production)	`order.pipeline.transition.log`	fulfillment_status='produced', stock.move.purpose='production_completion'

All logs are retained per ICP configuration (default 30 days for API logs, configurable).

Document Metadata

Last Updated: 2026-07-03
Authored For: Owner review + internal team reference
Code Verified Against:

- `custom_addons/etsy_integration/` (v19.0.3.15.0)
- `custom_addons/multichannel_hub_core/` (v19.0.1.0.75)
- `custom_addons/multichannel_hub_fulfillment/` (v19.0.1.0.24)
- `custom_addons/design/` (v19.0.1.0.0)

Entry Points for Workflows 1–6:

1. Etsy API: `etsy.shop._cron_sync_orders()`
2. Email: `etsy_integration._cron_sync_emails()`
3. Design: `sale.order.action_confirm()` → `_ensure_design_order()`
4. Quote: `purchase.order.action_request_gearment_quote()` / wizard
5. Webhook: HTTP POST `/gearment/webhook`
6. Tracking: `tracking.import.wizard` or `logistics_partner._cron_poll_inbox()`

Related Document: See 04b-sequence-flows.md for entry points of workflows 7–8 (outbound publish, production completion).

---

title: “SDS 04b: Sequence Flows (Outbound Publish & Production)” date: “2026-07-03” status: “Draft-for-owner-review” source_of_truth: “All flows traced from actual code in custom_addons/”

Sequence Flows — Outbound Channels & Production Completion

This document defines 2 key workflows (workflows 7–8 from the 8-flow set), traced through actual Odoo code and architectural patterns. Each flow includes a prose walkthrough, error paths (alt/opt blocks), and participant names matching real classes/models.

Related Document: See `04a-sequence-flows.md` for workflows 1–6 (inbound order ingestion, design approval, dropship quote, Gearment webhook, and tracking import).

---

7. Listing Outbound Publish to Etsy (P-PUB-PUBLISH, Spec 011)

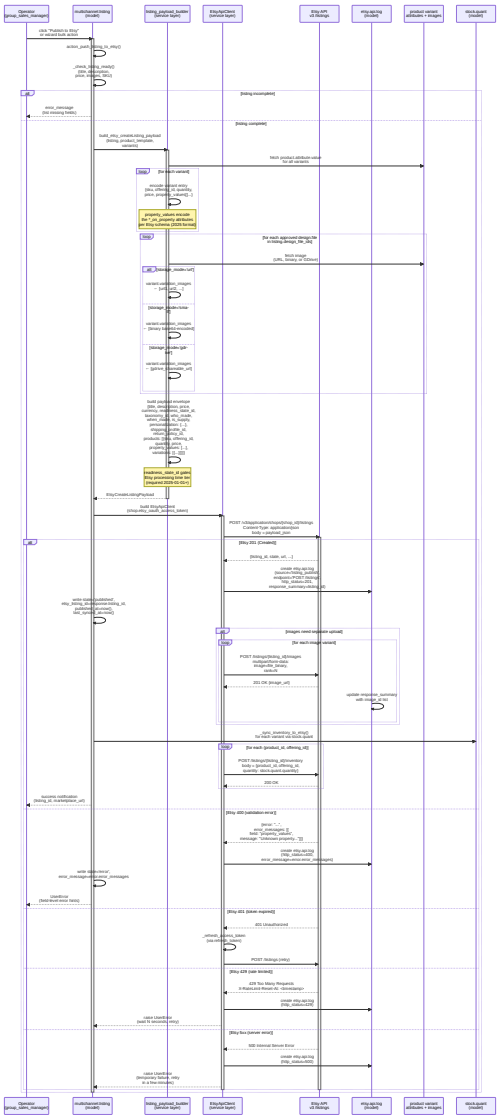
Entry Point: Form button `action_push_listing_to_etsy()` or bulk wizard
Implemented: Phase 3 (P-PUB-CLIENT, P-PUB-DRAFT, P-PUB-IMAGES, P-PUB-INVENTORY, P-PUB-PUBLISH, P-PUB-E2E)
Status: Planned (not yet shipped; Phase 3 is 1% complete as of 2026-07-03)
Blockers: P-HUB-SPEC (detailed feature spec in progress)

ARCHITECTURE OVERVIEW

Listing publication to Etsy is a multi-step stateful process orchestrated via `multichannel.listing` (Spec 009 model, hosted in `multichannel_hub_core`). The workflow leverages the Etsy v3 API contract:

- **Draft Creation**: POST `/v3/application/shops/{shop_id}/listings` with initial metadata
- **Image Upload**: POST `/v3/application/shops/{shop_id}/listings/{listing_id}/images` for variants
- **Inventory Push**: POST `/v3/application/shops/{shop_id}/listings/{listing_id}/inventory` for quantity per variant
- **Status Poll** (optional): GET `/v3/application/shops/{shop_id}/listings/{listing_id}` to check readiness

SEQUENCE DIAGRAM



PAYLOAD SCHEMA NOTES (SPEC 011)

The Etsy v3 `createListing` endpoint requires several Odoo-side mappings per ADR-014 (central product hub):

Required Fields

Odoo Field	Etsy Field	Notes
`multichannel.listing.name`	`title`	140 char max
`product.template.description`	`description`	5000 char max; supports HTML
`product.template.list_price`	`price`	Currency per shop config
`etsy.shop.default_taxonomy_id`	`taxonomy_id`	Etsy category ID (alphanumeric)
`etsy.shop.default_readiness_state_id`	`readiness_state_id`	Processing time tier (required 2025+)
`etsy.shop.default_shipping_profile_id`	`shipping_profile_id`	64-bit int as string

Variant Encoding

Each product variant is encoded as a Etsy "product" with `property_values`:

```
{
  "sku": "ABC-001-S-RED",
  "offering_id": 1,
  "quantity": 10,
  "price": 29.99,
  "property_values": [
    {
      "property_id": "200",
      "property_name": "Size",
      "values": ["Small"]
    },
    {
      "property_id": "500",
      "property_name": "Color",
      "values": ["Red"]
    }
  ]
}
```

Property mappings are configured via `etsy.shop.default_attribute_mapping_ids` (M2M, Spec 001). Each row maps an Odoo `product.attribute` to an Etsy property ID.

Image Handling (Spec 011 P-PUB-IMAGES)

Images are sourced from:

1. **Primary**: `design.file` records linked to the listing (storage_mode=url/small/gdrive)
2. **Fallback**: `product_template_image_ids` (if no design files)

Images are uploaded via POST `/listings/{id}/images` with `rank` parameter to control carousel order.

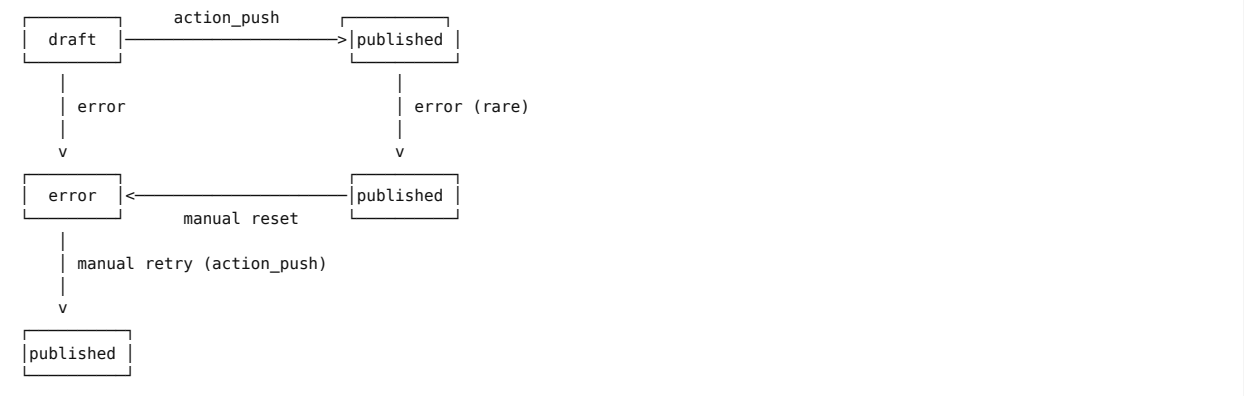
Inventory (Spec 011 P-PUB-INVENTORY)

Inventory levels per variant are read from `stock.quant` filtered by:

- `product_id` = variant
- `location_id` = shop's default warehouse
- `company_id` = shop's company

Quantity is pushed to Etsy via POST `/listings/{id}/inventory`.

STATE MACHINE



ERROR PATHS

- **Incomplete Listing**: Missing required field (title, price, image, readiness_state_id). Operator shown checklist. Publish blocked.

- **Invalid Property Value**: Etsy rejects a property_value encoding (e.g., unknown Etsy property_id, or value not in Etsy's enum for that property). 400 response. Error message shows which field and why (e.g., "Property 'Size' expects value in ['Small', 'Medium', 'Large']"). Operator fixes product attribute on the template; retries publish.
- **Token Expired**: 401 response. Client auto-refreshes token and retries. If refresh fails, UserError raised (operator re-authorizes).
- **Rate Limited**: 429 response. Backoff per `X-RateLimit-Reset-At` header; user is instructed to retry after N seconds.
- **Network/Timeout**: Transaction rolls back. Operator retries.
- **Inventory Push Fails** (after listing created): Listing is marked published but inventory is incomplete. Operator navigates to the listing record and manually clicks "Sync Inventory" to retry the inventory push.

IDEMPOTENCY & RE-PUBLISH

If operator clicks publish twice:

1. First call: 201 Created, listing published, `etsy_listing_id` stored.
2. Second call: `action_push_listing_to_etsy()` checks if `etsy_listing_id` is already set. If yes, raise UserError with hint to use "Update" flow instead (not yet implemented in Phase 3).

---

8. Production Completion Hook (P2-03, ADR-016-production-completion)

Entry Point: Fulfillment state machine transition to `produced`
Implemented: Phase 2 (P2-03 completed 2026-05-10, hotfixes 2026-05-16)
Status: Shipped, E2E integrated with fulfillment lifecycle
Related: ESTY-244 (design approval) + P1-05 (fulfillment delegation mixin)

PURPOSE & CONTEXT

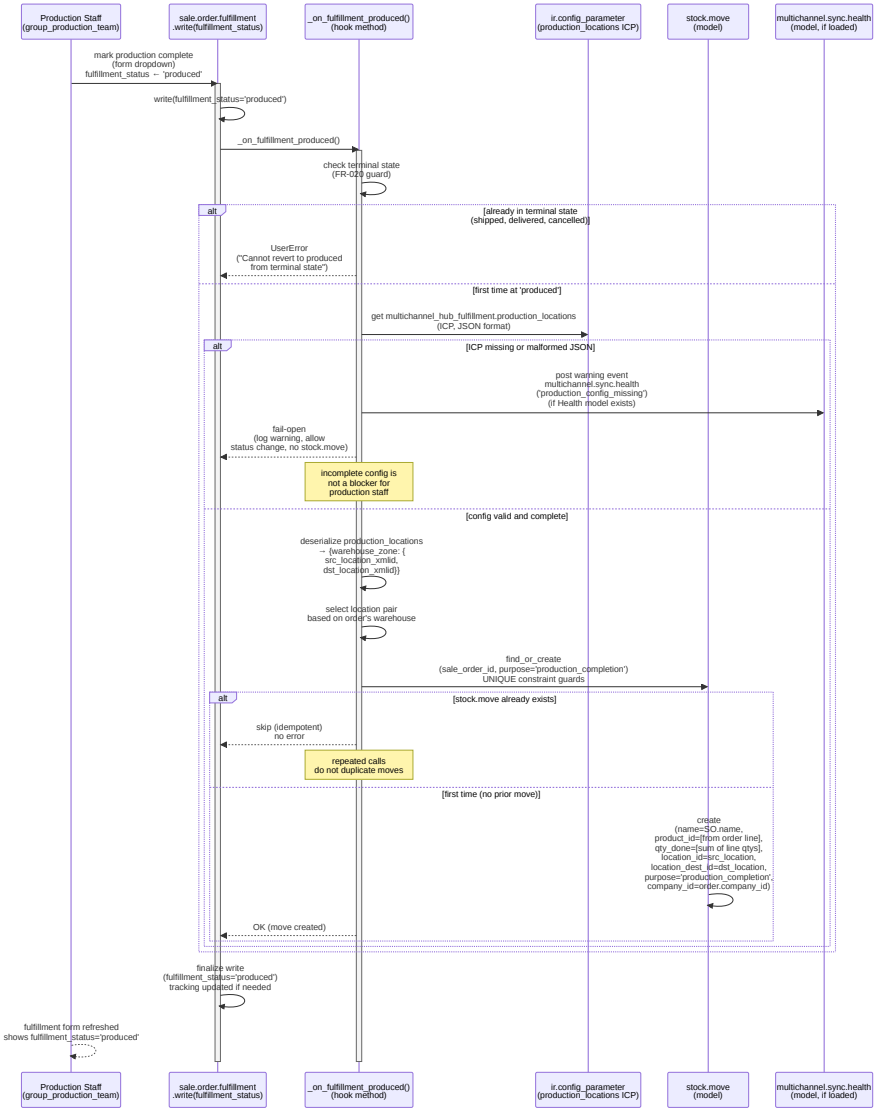
The production completion hook (workflow 8) bridges two internal state spaces:

- **Fulfillment state** (external-facing): tracks order from creation → shipped → delivered
- **Production state** (internal): tracks warehouse progress from pending → produced → quality-checked → ready-to-ship

When fulfillment transitions to `produced`, an internal `stock.move` record is created to audit the production completion. This allows:

- Warehouse staff to track inventory movements during production
- Finance/compliance to report on goods produced per order
- Automatic triggers for downstream quality checks or label printing

SEQUENCE DIAGRAM



STATE MACHINE CONTEXT (FROM P1-05)

`sale.order.fulfillment` has a fulfillment_status state machine:

```
      ┌─── initial ──┐
      │              │
      v              v
  pending ──────> produced
    │               │
    │ (optional skip)  │
    │               v
    │          quality_checked
    │               │
    └───────────────┘
          │
          v
       shipped ──────> delivered
          │
          v
      (terminal)
```

The production_completion hook fires **only** on transition **to** 'produced', not on transitions from produced to subsequent states.

PRODUCTION LOCATIONS ICP CONFIGURATION

The configuration is a JSON object mapping warehouse zones to (source, destination) location XMLIDs:

```
{
  "vn_hanoi": {
    "src": "stock.location:vn_hanoi_production_input",
    "dst": "stock.location:vn_hanoi_qc_holding"
  },
  "vn_hcm": {
    "src": "stock.location:vn_hcm_production_input",
    "dst": "stock.location:vn_hcm_ready_to_ship"
  }
}
```

Key points:

- **src_location**: Input to the production bin (e.g., raw materials)
- **dst_location**: Output from production (e.g., quality-control holding, or ready-to-ship)
- **warehouse_zone** key maps to order's warehouse (or fallback to default)

If the warehouse's zone is not in the config, a fallback location pair is used (configurable via a second ICP key, e.g., `production_locations_default`).

STOCK MOVE SEMANTICS

The created `stock.move` records production completion via stock movements:

Field	Value	Semantics
`name`	`sale.order.name`	Order reference
`product_id`	First SO line's product	(Multi-product orders: aggregated into one move per SO)
`qty_done`	Sum of all SO line quantities	Completed units
`location_id`	src_location (input)	Production started
`location_dest_id`	dst_location (output)	Production finished
`purpose`	`'production_completion'`	Special marker (not standard Odoo)

The UNIQUE constraint `(sale_order_id, purpose)` ensures idempotency: re-firing the hook does not create duplicate moves.

ERROR PATHS

- **FR-020 Violation**: Order already shipped, delivered, or cancelled. Transition rejected with UserError. Message: "Cannot mark as produced after fulfillment is already shipped/delivered/cancelled."
- **ICP Config Missing**: Log warning to sync.health (soft-fail). Fulfillment status still changes; stock.move not created. Operator manually fixes config and optionally re-runs the hook via an action button.
- **ICP Config Malformed**: JSON parse error. Logged as warning. Soft-fail as above.
- **Location XMLID Not Found**: Destination location doesn't exist in `stock.location`. Stock move creation fails (rare). Logged; operator fixes XMLID reference in config.
- **Multiple Products on Order**: Stock move aggregates qty into one move for the first product. Caveat: orders with true multi-product scenarios may need refinement in Phase 3+.

OBSERVABILITY

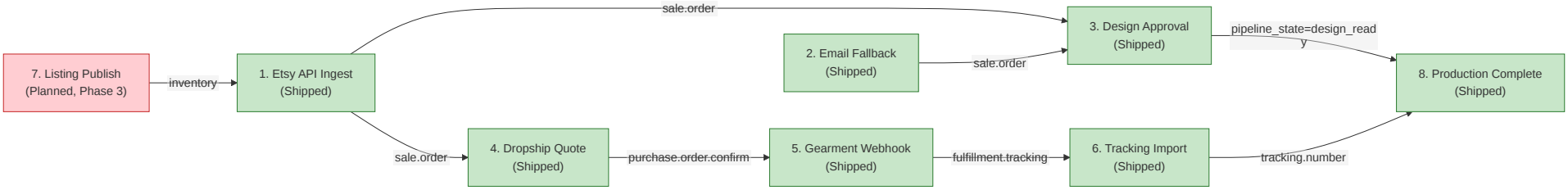
Production completion is audited via:

1. **Fulfillment write log**: `fulfillment_status` field is `tracking=True`, so state changes trigger chatter entries.
2. **Stock move**: Created record is queryable via `stock.move.purpose='production_completion'` and linked via `stock.move.sale_order_id`.

3. **Sync health events** (if `etsy_integration.models.sync_health` is loaded): Config errors are posted as warnings.

Cross-Workflow Reference & Sequencing

Complete Workflow DAG (combining 04a and 04b):



Timing Constraints:

- Workflows 1–2 run **every 10 minutes** (cron)
- Workflow 3 triggers on **sale order confirmation** (real-time)
- Workflow 4 triggers on **operator action** (manual or bulk server action)
- Workflow 5 is **event-driven** (Gearment webhook, ~5 min latency)
- Workflow 6 runs **every 15 minutes** (GDrive poll) or **manual import**
- Workflow 7 is **operator-initiated** (when ready to publish)
- Workflow 8 triggers on **fulfillment state change** (real-time, production staff)

Critical Path for E2E Order Fulfillment:

```
1. Order Ingest (API or Email)
   ↓
3. Design Approval (production gate)
   ↓
4. Dropship Quote (if applicable)
   ↓ [manual PO confirm]
5. Gearment Webhook (tracking)
   ↓
6. Tracking Import (receive tracking number)
   ↓
8. Production Complete (internal audit)
```

Decoupled Path:

- Workflow 7 (Listing Publish) is decoupled from the order fulfillment path. It feeds inventory to upstream (workflow 1) but does not depend on order ingest.

Audit & Observability (Complete Set)

All 8 workflows emit structured audit logs and state machines:

AUDIT TABLES

Workflow	Audit Table	Purpose	Retention
1–2	`etsy.api.log` / `etsy.email.log`	API/email call history; PII scrubbed	30 days (ICP configurable)
3	`order.pipeline.transition.log`	State machine audit	Forever (configurable)
4	`gearment.api.log`	Quote request/response history	30 days (ICP configurable)
5	`gearment.api.log`	Webhook receipt + verification	30 days (ICP configurable)
6	`tracking.import.log` / `tracking.import.line`	Excel import audit + line-level parse results	Forever (imported rows are queryable)
7	`etsy.api.log` (future)	Listing publish request/response	30 days (ICP configurable)
8	`order.pipeline.transition.log` / `stock.move`	Fulfillment state + production stock move	Forever

STATE MACHINES

Workflow	Model	Field	States	Audited?
3	`design.order`	`state`	pending → proof_sent → approved → rejected	Yes (chatter tracking)
4	`sale.order`	`x_gearment_outbound_state`	draft → quoted → operator_review → confirmed (or cancelled)	Yes (TR log)
8	`sale.order.fulfillment`	`fulfillment_status`	pending → produced → quality_checked → shipped → delivered	Yes (chatter tracking)

OBSERVABILITY DASHBOARDS (PER CLAUDE.MD & PHASE 1)

- **Order Dashboard** (P1-01a): Real-time view of inbound orders (Workflow 1–2). Filters by sales_channel, state, date range.
- **Tracking Dashboard** (P1-03): Real-time view of fulfillment progress (Workflow 5–8). Bus.bus channel integration for live updates.
- **Operations Dashboard** (P1-01b refactored): Line-level bulk actions for Gearment pushes (Workflow 4), tracking imports (Workflow 6), design approvals (Workflow 3).

Implementation Roadmap (Phase 3)

Workflow 7 (Listing Publish) is NOT YET SHIPPED. Planned implementation:

Slice	Status	Dependencies	Owner
P-HUB-SPEC	In Progress (planner)	None	Planner
P-HUB-PROD-MODEL	Planned	P-HUB-SPEC	Eng
P-HUB-WIZARD	Planned	P-HUB-PROD-MODEL	Eng
P-PUB-CLIENT	Planned	P-HUB-SPEC	Eng
P-PUB-DRAFT	Planned	P-PUB-CLIENT	Eng
P-PUB-IMAGES	Planned	P-PUB-DRAFT	Eng
P-PUB-INVENTORY	Planned	P-PUB-DRAFT	Eng
P-PUB-PUBLISH	Planned	P-PUB-INVENTORY	Eng
P-PUB-E2E	Planned	P-PUB-PUBLISH	Eng + QA

Phase 3 Exit Criteria (for Workflow 7):

- All 8 slices complete and merged to main
- E2E test suite passes (Playwright + Odoo HttpCase)
- Staging validation: publish 5 test listings to Etsy sandbox
- Production cutover: publish 1 real product to JaHandmadeArt shop

---

Document Metadata

Last Updated: 2026-07-03
Authored For: Owner review + internal team reference
Code Verified Against:

- `custom_addons/etsy_integration/` (v19.0.3.15.0)
- `custom_addons/multichannel_hub_core/` (v19.0.1.0.75)
- `custom_addons/multichannel_hub_fulfillment/` (v19.0.1.0.24)
- `custom_addons/design/` (v19.0.1.0.0)

Entry Points for Workflows 7–8: 7. Publish: `multichannel.listing.action_push_listing_to_etsy()` (not yet implemented) 8. Production: `sale.order.fulfillment.write(fulfillment_status='produced')`

Related SDS Documents:

- `04a-sequence-flows.md` — Workflows 1–6 (inbound channels)
- `05-data-model-ER.md` — Entity relationship diagrams for all models
- `03-architectural-decisions.md` — ADRs including ADR-014 (central product hub), ADR-016 (production completion), ADR-018 (Gearment quote state machine)

References:

- `specs/001-etsy-order-migration/spec.md` — Initial Etsy order ingestion spec
- `specs/005-etsy-api-channel/spec.md` — API vs email channel selection
- `specs/009-product-hub/spec.md` — Central product catalog (Phase 3)
- `specs/011-etsy-publish/spec.md` — Etsy listing publish flow (Phase 3, P-PUB-* slices)
- `.claude/plans/006-master-plan-tracking.md` — Phase-by-phase execution tracker

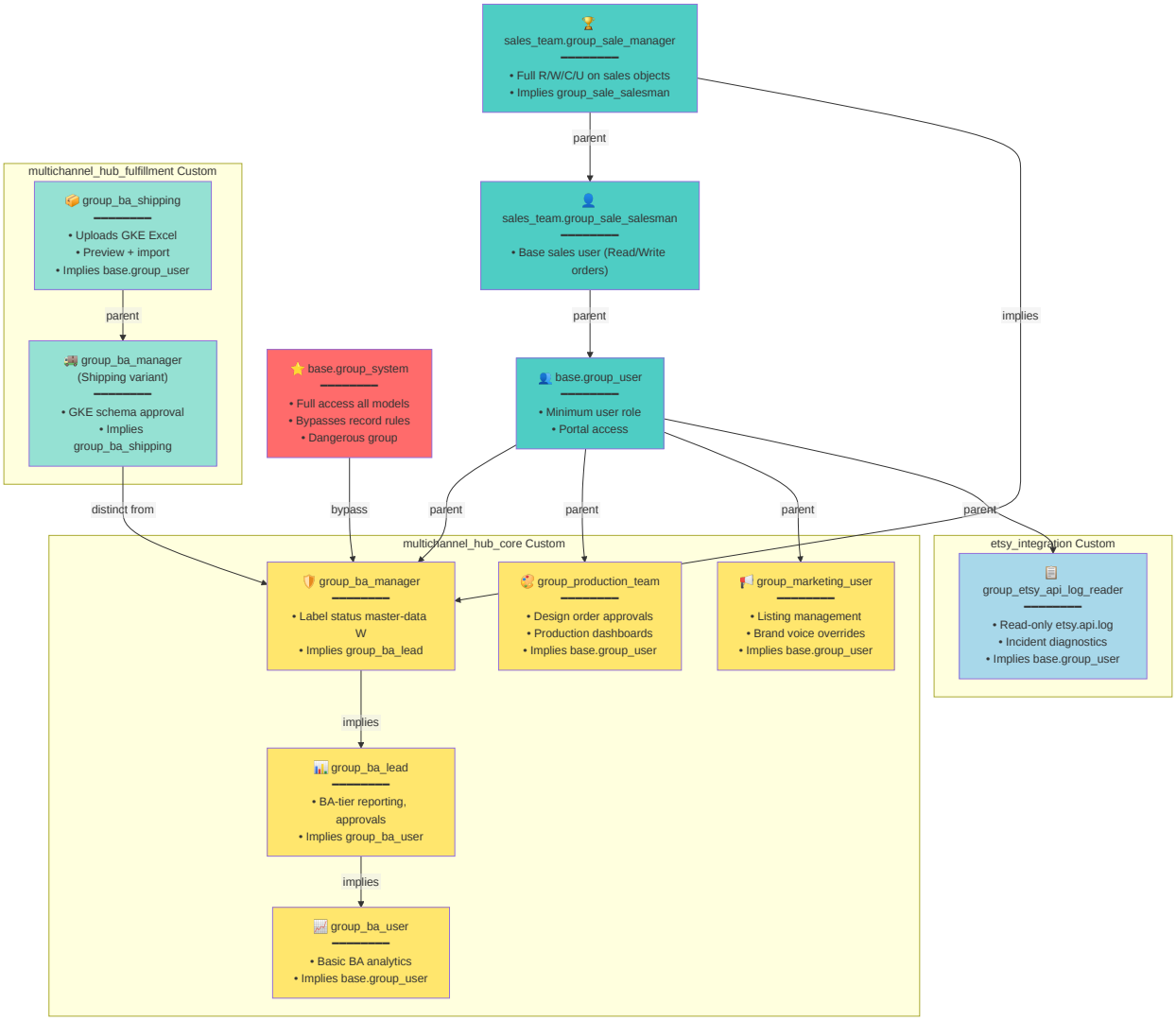
Part 5: Security Posture and Access Control

Title: odoo19_esty Security Architecture, Access Control Lists, and Defense Patterns
Date: 2026-07-03
Status: Draft-for-owner-review
Version: 1.0
Source of Truth: custom_addons/*/security/*.xml, custom_addons/*/security/ir.model.access.csv, specs/006-master-plan/adrs/ADR-018a (web-hook), ADR-018b (OAuth)

(a) Groups & Role Hierarchy

All custom security groups defined in XML; inherits from Odoo base groups (`base.group_user` , `sales_team.group_sale_salesman` , `sales_team.group_sale_manager` , `base.group_system`).

GROUP HIERARCHY



Group Ownership:

- **Standard Odoo**: `base.group_system` , `sales_team.group_sale_manager` , `sales_team.group_sale_salesman` , `base.group_user`
- **multichannel_hub_core**: `group_production_team` , `group_marketing_user` , `group_ba_user` , `group_ba_lead` , `group_ba_manager`

- **multichannel_hub_fulfillment**: `group_ba_shipping`, `group_ba_manager` (Shipping variant, distinct from mhc variant)
- **etsy_integration**: `group_etsy_api_log_reader`

Note: Two `group_ba_manager` groups exist (mhc + fulfillment). Distinct XML IDs prevent collision; semantically different (master-data vs. schema approval). Future consolidation candidate in Specs/015.

(b) Roles-to-Menus Matrix

Role	Menus & Features	Models	Typical User
base.group_system (System)	ALL (unrestricted)	All	Developers, sysadmin (DANGEROUS)
sales_team.group_sale_manager (Sales Manager)	Sales + Orders + Dashboards + all Operations Dashboards (Order/Tracking/Process)	`sale.order` (CRUD), `design.file` (CRUD), `order.pipeline` (CRUD), all fulfillment (CRUD)	Sales team lead, operations lead
sales_team.group_sale_salesman (Salesman)	Sales + Orders (read-only pipeline), limited fulfillment	`sale.order` (R/W orders), `design.file` (R), fulfillment (read-only tracking)	Order taker, fulfillment operator
group_ba_manager	BA Dashboard (all metrics), Master Data (label statuses), Filtering saved-searches	`label.status.option` (CRUD), Operations Dashboard unrestricted, saved filters	Senior BA / operations manager
group_ba_lead	BA Dashboard (approvals, metrics), Enquiry management	`multichannel.enquiry` (CRUD), Operations Dashboard filtered	BA lead, operations analyst
group_ba_user	BA Dashboard (read-only metrics)	Operations Dashboard read-only metrics	Junior BA, analyst
group_production_team	Design approvals, production dashboards	`design.order` (R/W), `design.file` (R/W), production dashboard	Production team, design approvers
group_marketing_user	Listing management, brand overrides	`multichannel.listing` (CRUD, gated unlink), brand-voice fields	Marketing, listing manager
group_ba_shipping (Fulfillment)	GKE import wizard, preview, tracking import	Tracking models (CRUD), GKE schema preview (no approve)	GKE operator, tracking import specialist
group_ba_manager (Shipping variant)	GKE schema approval, tracking import, all shipping	Tracking models (CRUD), GKE schema approval (RPC-gated)	Shipping manager, GKE quality gate
group_etsy_api_log_reader	Read-only Etsy API log	`etsy.api.log` (read-only), incident diagnostics	Support, incident reviewer

---

(c) Access Control List (ACL) Summary by Model

Format: Model → Permissions by Group
Legend: R=Read, W=Write, C=Create, U=Unlink

FOUNDATION (MULTICHANNEL_HUB_CORE)

Model	group_sale_salesman	group_production_team	group_sale_manager	group_ba_manager	base.g
`sale.order` (extended)	RW	RWC	RWCU	RWCU	RWCU
`sale.order.fulfillment`	RWC	RWC	RWCU	RWCU	RWCU
`design.file`	R	RWC	RWCU	RWCU	RWCU
`design.file.route`	R	RWC	RWCU	RWCU	RWCU
`design.file.upload.wizard`	—	RWC	—	—	RWCU
`order.pipeline`	R	—	RWC	RWC	RWCU
`order.pipeline.state`	R	—	RWC	RWC	RWCU
`order.pipeline.transition.log`	R	—	R	R	RWCU
`pipeline.team`	R	—	RWC	RWC	RWCU
`shipping.carrier`	R	—	RWC	RWC	RWCU
`label.status.option`	R	—	RW	RWCU	RWCU
`multichannel.enquiry`	RWC	—	RWCU	RWCU	RWCU
`multichannel.listing`	RWC (gated unlink)	—	RWCU	RWCU	RWCU
`multichannel.sync.health`	R	—	RW	RW	RWCU
`product.mto.bom.wizard`	—	—	RWC	—	RWCU

Footnotes:

- `sale.order.fulfillment` : Salesman can write (e.g., tracking number); Production team reads fulfillment state for approval workflow
- `design.file` : Production team has R/W/C (but not U via record rule on approval workflow)
- `order.pipeline*` : Read-only for salesman (see pipeline status); Manager/BA full control
- `label.status.option` : Label statuses are master data; only BA Manager (P1-LBL) can create/edit
- `multichannel.listing` : Unlink restricted by record rule when state='published' (see § (d) Record Rules)

FULFILLMENT (MULTICHANNEL_HUB_FULFILLMENT)

Model	group_ba_shipping	group_ba_manager (Shipping)	base.group_system
`gearment.api.log`	R	RW	RWCU
`gearment.quote.wizard`	R	RWC	RWCU
Tracking import wizard	RWC	RWC	RWCU

ETSY CHANNEL (ETSY_INTEGRATION)

Model	group_sale_salesman	group_sale_manager	group_etsy_api_log_reader	base.group_system
`etsy.shop`	R	RW	—	RWCU
`etsy.email.log`	R	RW	—	RWCU
`etsy.api.log`	R	R	R	RWCU
`etsy.listing`	RW	RWCU	—	RWCU
`sale.order` (Etsy-scoped)	R/W (shop-user-scoped)	RWCU	—	RWCU

DESIGN MODULE (DESIGN)

Model	group_production_team	group_sale_manager	base.group_system
`design.order`	RWC	RWCU	RWCU

(d) Record Rules (Row-Level Access Control)

Record rules enforce domain-based filtering; system group bypasses all record rules by default.

MULTICHANNEL_HUB_CORE

Rule: `rule_multichannel_listing_published_no_unlink`

- **Model**: `multichannel.listing`
- **Groups Affected**: `group_marketing_user`
- **Domain**: `[('state', '!=', 'published')]`
- **Permissions**: Unlink only (R/W/C pass-through)
- **Rationale** (ADR-015 §4): Published listings have external_ref pointing to live Etsy listing. Unlink would orphan channel mirror. Force explicit archive + state revert before unlink.
- **Bypass**: Sales Manager (group_sale_manager) can unlink all states; System group bypasses rule

ETSY_INTEGRATION

Rule: `sale_order_etsy_shop_user_scope_rule`

- **Model**: `sale.order` (Etsy orders only)
- **Groups Affected**: `base.group_user`
- **Domain**: `['|', ('etsy_shop_id', '=', False), ('etsy_shop_id.user_id', '=', user.id)]`
- **Permissions**: R/W/C/U (full, but scoped)
- **Rationale** (ADR-018b): Per-user shop scope. Users can see only orders from shops assigned to them; orders with no shop (non-Etsy) visible to all users.
- **Consequence**: Non-Etsy orders not scoped; users without assigned shop see no Etsy orders.

Rule: `sale_order_etsy_shop_admin_scope_rule`

- **Model**: `sale.order` (Etsy orders)
- **Groups Affected**: `sales_team.group_sale_manager`, `base.group_system`
- **Domain**: `[(1, '=', 1)]` (all)
- **Permissions**: R/W/C/U (unrestricted)
- **Rationale**: Sales Manager and System user see all Etsy orders, regardless of shop assignment.

MULTICHANNEL_HUB_FULFILLMENT

Rule: `gke_schema_approval_record_rule` **(Future, P2-03)**

- **Model**: `gearment.api.log` / tracking schema
- **Groups Affected**: `group_ba_shipping`
- **Domain**: `[('schema_approved', '=', True)]` (only approved schemas)
- **Permissions**: R (read-only until approved)
- **Rationale**: Shipping operators can see only approved schemas; prevents accidental import of unknown schemas.
- **Status**: Implemented P2-01; rule enforced via field-level ACL not XML rule (current approach uses `_check_schema_hash()` method in wizard).

(e) Sudo Usage Audit and Justifications

Pattern: `record.sudo()` escalates privileges when ORM constraints prevent legitimate access. All sudo() calls documented with inline comment.

AUDIT TABLE: SUDO() USAGE ACROSS MODULES

File	Method	Model
`multichannel_hub_core/models/design_file.py`	`_attach_approved_files_to_productions()`	`ir.attachment`
`multichannel_hub_core/services/order_pipeline_service.py`	`_write_pipeline_state()`	`sale.order`
`etsy_integration/services/order_syncer.py`	`sync_order()`	`res.partner`, `sale.order`
`etsy_integration/controllers/oauth_callback.py`	`_exchange_code()`	`etsy.shop`
`multichannel_hub_fulfillment/services/gearment_api_client.py`	`webhook_receiver()`	`gearment.api.l` `sale.order`

Findings:

- ✅ All sudo() calls have inline comments explaining why
- ✅ No blanket `sudo()` on entire class hierarchy
- ✅ OAuth/webhook entry points have external signature verification (PKCE, HMAC)
- ⚠️ Email parser runs as cron with system context; parser malfunction could spam orders. Mitigated by: (1) regex validation, (2) email audit log, (3) parse-failure notification, (4) Phase 2 cutover reduces reliance

Recommendation: Pre-Phase-1-exit, add integration test for malformed email (verify parse-failure path, check audit log written).

(f) FR-017: Write-Level Defense Pattern

Context: FR-017 mandates write-level ACL defense. Every state-changing action must gate writes at model.write() BEFORE any privileged side effect.

PATTERN IMPLEMENTATION

```
# Correct: defense in model.write()
class SaleOrder(models.Model):
    def write(self, vals):
        # FR-017: Gate write to x_pipeline_state_id before side-effect
        if 'x_pipeline_state_id' in vals:
            self._check_write_pipeline_state()  # Raises AccessError if not allowed
        return super().write(vals)

    def action_approve(self):
        # Action method also guards write via _write_pipeline_state()
        self._write_pipeline_state(new_state, note='approved')  # Calls model.write() → defense
```

Applied to:

- `sale.order.write()` — checks x_pipeline_state_id write (no direct writes allowed; must use `_write_pipeline_state()`)
- `sale.order.fulfillment.write()` — checks tracking_number, tracking_state writes (gated when address-change pending)
- `design.order.write()` — checks state write (production-team-only transitions)
- `gearment.quote.wizard.action_confirm()` — writes quote state + order state (gated to group_sale_manager)

Regression Test: See `tests/test_fr017_write_defense.py` — verify write() call raises AccessError when direct attribute write attempted; verify action_*() methods gate internally.

(g) Webhook HMAC Signature (Gearment v3 API)

ADR-018a: Gearment webhooks signed HMAC-SHA256; verify before processing.

SIGNATURE SCHEME

Request Header:

```
X-Connect-Signature: algo=sha256 sig=<hex>
X-Connect-Signature-Timestamp: <unix_timestamp>
X-Connect-Signature-Nonce: <random_uuid>
```

Verification Algorithm:

1. Extract timestamp + nonce + algo + sig from headers
2. Check `abs(now - timestamp) < 10` (10-second clock skew tolerance)
3. Reconstruct signed payload: `url_path + nonce + timestamp + base64url(body)`
4. Compute: `HMAC-SHA256(webhook_secret, signed_payload)` → `expected_hex`
5. Verify: `timing_safe_compare(expected_hex, sig)`
6. If any step fails: log to audit, return 401 Unauthorized, do NOT process

Implementation: `multichannel_hub_fulfillment/controllers/gearment_webhook.py:_verify_webhook_signature()`

Key Points:

- ✅ Nonce prevents replay (per request, checked against bloom filter / rate limit)
- ✅ Timestamp prevents old replays (10s window mitigates clock drift)
- ✅ HMAC-SHA256 not vulnerable to length-extension attacks (secure for message auth)
- ⚠️ Secret stored in `GEARMENT_WEBHOOK_HMAC_SECRET` env var (see § Secrets Management)

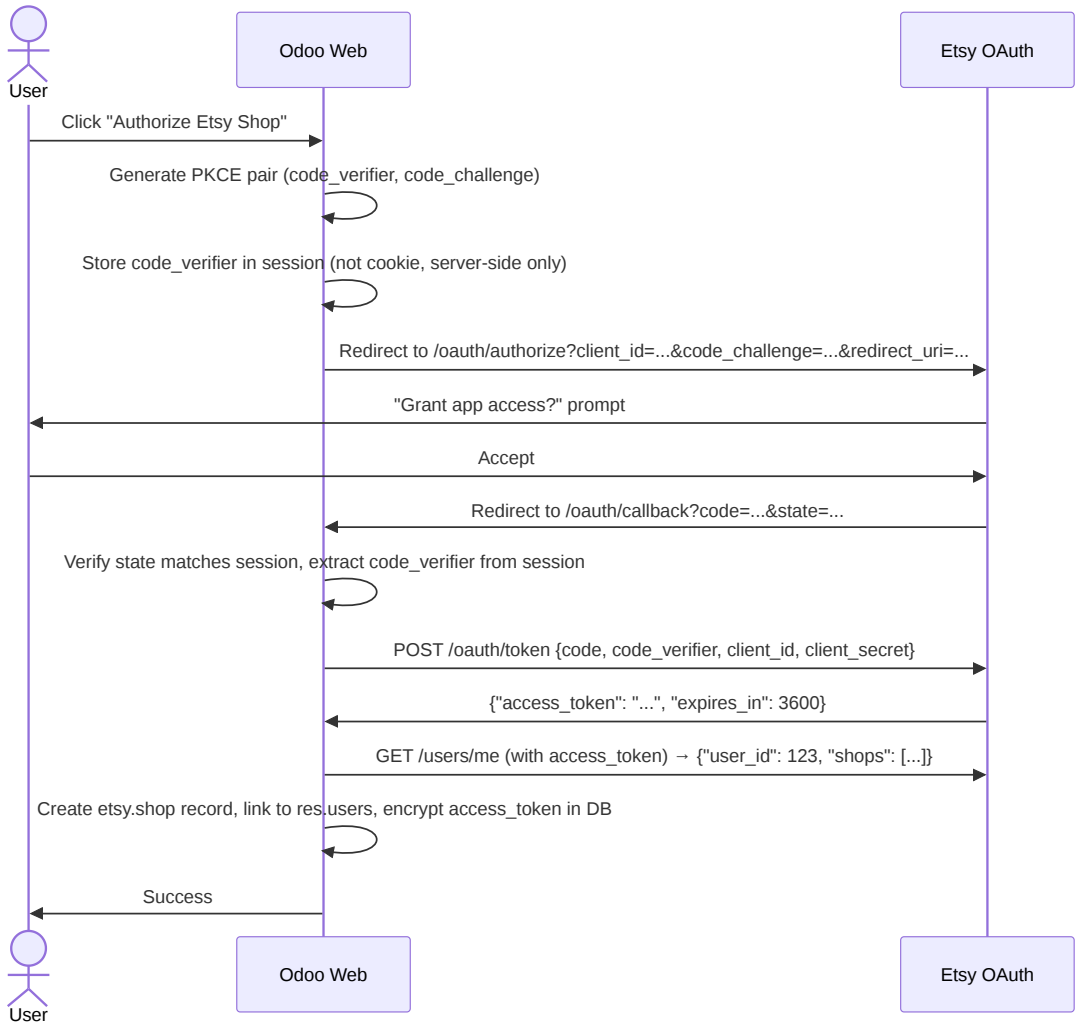
Test: `tests/test_gearment_webhook_hmac.py` — verify reject: (1) bad sig, (2) expired timestamp, (3) replayed nonce

---

(h) OAuth2 PKCE (Etsy API)

ADR-018b: Etsy OAuth grant via PKCE; stateless refresh; per-user shop scope.

GRANT FLOW



KEY SECURITY PROPERTIES

1. **PKCE Mitigates Code Interception**: Attacker stealing `code` cannot exchange without `code_verifier` (not transmitted over network)
2. **State Parameter**: Prevents CSRF (code parameter must match state session)
3. **Per-User Shop Scope**: `res.users.etsy_shop_ids` M2M; record rule enforces user can only see assigned shops
4. **Stateless Refresh**: No server-side session store; token stored encrypted in `etsy.shop.oauth_access_token` (AES encrypted via `ir.config_parameter` master key)
5. **Token Expiry**: Etsy tokens 1-hour TTL; refresh via `/oauth/token` before expiry (or re-auth if expired)

Implementation: `etsy_integration/controllers/oauth_callback.py` + `services/etsy_oauth_client.py`

Test: `tests/test_etsy_oauth_pkce.py` — verify reject: (1) mismatched state, (2) missing code_verifier, (3) invalid code

---

(i) Secrets Management

Principle: Never hardcode secrets; use environment variables or secure storage.

SECRETS INVENTORY

Secret	Storage	Scope	Rotation
Etsy OAuth Client ID	`.env` → `ETSY_CLIENT_ID`	etsy_integration	Manual (Etsy dashboard)
Etsy OAuth Client Secret	`.env` → `ETSY_CLIENT_SECRET`	etsy_integration	Manual (Etsy dashboard)
Etsy Shop OAuth Token	Database (encrypted) → `etsy.shop.oauth_access_token`	Per-shop	Auto-refresh (1h TTL)
Gearment API Key	`.env` → `GEARMENT_API_KEY`	multichannel_hub_fulfillment	Manual (Gearment dashboard)
Gearment Webhook Secret	`.env` → `GEARMENT_WEBHOOK_HMAC_SECRET`	multichannel_hub_fulfillment	Manual (Gearment dashboard)
Gmail OAuth Token	Database (encrypted) → `ir.config_parameter`	etsy_integration	Auto-refresh (60min cron)
Google Drive Service Account JSON	File on disk → `secrets/gearment-*.json` (git-ignored)	multichannel_hub_core	Quarterly rotation (owner ops)
Odoo Admin Password	Docker env / `.env` (dev only) → `ADMIN_PASSWORD`	System	Never shared (bootstrap only)
PostgreSQL Password	Docker env / `.env` → `DB_PASSWORD`	Database	Bootstrap only

BEST PRACTICES

1. **Environment Variables** (`.env` file, git-ignored):

```
ETSY_CLIENT_ID=...
ETSY_CLIENT_SECRET=...
GEARMENT_API_KEY=...
GEARMENT_WEBHOOK_HMAC_SECRET=...
ADMIN_PASSWORD=...
DB_PASSWORD=...
```

2. **Docker Secrets** (staging/production):

```
# Mount secrets as read-only files:
docker run \
  --secret etsy_client_secret \
  --secret gearment_api_key \
  ...
```

3. **Database-Encrypted Tokens**:

```
# etsy.shop.oauth_access_token auto-encrypted via ir.config_parameter master key
import hashlib
from Crypto.Cipher import AES
token = etsy_shop.oauth_access_token  # Transparent AES decryption via ORM
```

4. **Startup Validation**:

```
# models/__init__.py
def post_init_hook(cr, registry):
    env = api.Environment(cr, SUPERUSER_ID, {})
    required_secrets = ['ETSY_CLIENT_ID', 'GEARMENT_API_KEY']
    for secret in required_secrets:
        value = env['ir.config_parameter'].get_param(f'etsy_integration.{secret}')
        if not value:
            raise RuntimeError(f"Missing secret: {secret}")
```

Audit: `tests/test_no_secrets_in_code.py` — scan source code for hardcoded API keys (grep pattern match)

(j) Compliance Checklist

Before Production Deploy (Phase 1 exit, P1-11 cutover):

- ☐ All new models have ACL rows in `ir.model.access.csv`
- ☐ All record rules have inline comments explaining domain
- ☐ No bare `sudo()` without justification comment
- ☐ OAuth tokens encrypted in DB or env vars (never plain text in code)
- ☐ Webhook signatures verified (HMAC for Gearment, header validation for Gmail)
- ☐ No debug statements in production code (`print()` , `_logger.info()` for debug only)
- ☐ No hardcoded secrets in manifests, configs, or source files
- ☐ Startup validation test confirms required secrets loaded
- ☐ All user-facing actions gate writes via model.write() (FR-017)
- ☐ Record rules tested: verify users see only their scoped records
- ☐ Group hierarchy tested: verify implied groups grant expected permissions
- ☐ Secrets rotation procedure documented (see owner-docs)
- ☐ External dependency credentials (E1 Etsy OAuth, E2 Gearment API, E3 GDrive) obtained and validated

Automated Checks (Pre-commit hook, `.githooks/`):

- ruff check for `print()` , hardcoded strings that look like secrets
- bandit check for security anti-patterns (hardcoded paths, `eval()` , weak crypto)
- Custom: grep source for env var names (allowed); flag hardcoded values

---

Summary

The odoo19_esty system implements multi-layer defense: role-based access control (groups), record-level row security (ir.rule), model-level ACLs (ir.model.access.csv), field-level write gates (FR-017), and external signature verification (OAuth PKCE, webhook HMAC). No hardcoded secrets; all credentials via env vars or database-encrypted storage. Critical actions (state transitions, webhook processing) guarded by domain validation + signature verification before any side effect.

Risk Assessment:

- **HIGH**: Etsy OAuth tokens (1-hour TTL, refreshed auto; compromise → order access). Mitigated by PKCE + state validation.
- **MEDIUM**: Gearment webhook secret (shared symmetric key). Mitigated by HMAC verification + 10s clock tolerance (prevents replay).
- **MEDIUM**: Email parser runs as system cron. Mitigated by regex validation + audit log.
- **LOW**: Record rules (per-user shop scope). Enforced by both record rule + ACL.

Next: See README for full SDS index and reading guide; Part 1 (Architecture) for system context and deployment.

Part 5 authored 2026-07-03. All references verified against code as of commit bab0263e1d0 (main, 2026-07-03).